Noted Insights

SYSTEM ADMINISTRATION USING LINUX

BICTE 8th Semester
Complete Notebook

AJAY MAHATO

Contact Us:
www.notedinsights.com
ajaymahato@notedinsights.com

Table of Contents

Unit 1: Introduction to Linux and CLI Commands

1.1 Introduction to Open-Source Software

The term **Open-source** is closely related to Open-source software (OSS). Open-source software is a type of computer software that is released under a license, but the source code is made available to all the users. The copyright holders of such software allow the users to use it and do some valuable modifications in its source code to add some new features, to improve the existing features, and to fix bugs if there are any. Because of this reason only Open-source software is mostly developed collaboratively.

1.1.1 Some famous examples of Open-source products are:

- **Operating systems –**
  Android, Ubuntu, Linux
- **Internet browsers –**
  Mozilla Firefox, Chromium
- **Integrated Development Environment (IDEs) –**
  Vs code (Visual Studio Code), Android Studio, PyCharm, Xcode

1.1.2 Open-source community and Contributions:

The **open-source community** is a worldwide community of programmers and software developers who are continuously working on various open-source projects to make our lives better. This community is self-governing and self-organizing, there are no executives to take the decisions solely. This community plays a very crucial role in the sustainability of various open-source organizations.

The contributions made in any open-source project which improves its usability are called **open-source contributions**. These contributions can be of any form not only some software codes like we can work on improving its **documentation**, improving its **UI/UX (user interface and design)**, organize meetups, or find new collaborators.

1.1.3 Benefits of Open-source contributions:

- We code for real-world open-source projects.
- It refines our existing knowledge of programming and also helps us to learn new skills.
- Many open-source projects offer mentorship programs to guide and help us through our first few contributions.
- We need not develop the whole thing from scratch, we just have to fork our favorite projects and start experimenting with them.
- After making any open-source contribution, we get immediate feedback regarding our developmental work.

- While doing open-source contributions, we interact with like-minded developers from all over the world and build connections along the way.
- As we get more closer to the open-source community, we get to know much more about our field of interest and other related fields.

1.2 Introduction to UNIX System

UNIX is an innovative or groundbreaking operating system which was developed in the 1970s by Ken Thompson, Dennis Ritchie, and many others at AT&T Laboratories. It is like a backbone for many modern operating systems like Ubuntu, Solaris, Kali Linux, Arch Linux, and also POSIX. Originally, It was designed for developers only, UNIX played a most important role in the development and creation of the software and computing environments. Its distribution to government and academic institutions led to its widespread adoption across various types of hardware components. The core part of the UNIX system lies in its base Kernel, which is integral to its architecture, structure, and key functionality making it the heart of the operating system.

The basic design philosophy of UNIX is to provide simple, powerful tools that can be combined to perform complex tasks. It features a command-line interface that allows users to interact with the system through a series of commands, rather than through a graphical user interface (GUI).

1.2.1 Some of the Key Features of UNIX Include

1. **Multiuser support:** UNIX allows multiple users to simultaneously access the same system and share resources.
2. **Multitasking:** UNIX is capable of running multiple processes at the same time.
3. **Shell scripting:** UNIX provides a powerful scripting language that allows users to automate tasks.
4. **Security:** UNIX has a robust security model that includes file permissions, user accounts, and network security features.
5. **Portability:** UNIX can run on a wide variety of hardware platforms, from small embedded systems to large mainframe computers.
6. **Communication:** UNIX supports communication methods using the write command, mail command, etc.
7. **Process Tracking:** UNIX maintains a record of the jobs that the user creates. This function improves system performance by monitoring CPU usage. It also allows you to keep track of how much disk space each user uses, and the use that information to regulate disk space.

Today, UNIX is widely used in enterprise-level computing, scientific research, and web servers. Many modern operating systems, including Linux and macOS, are based on UNIX or its variants.

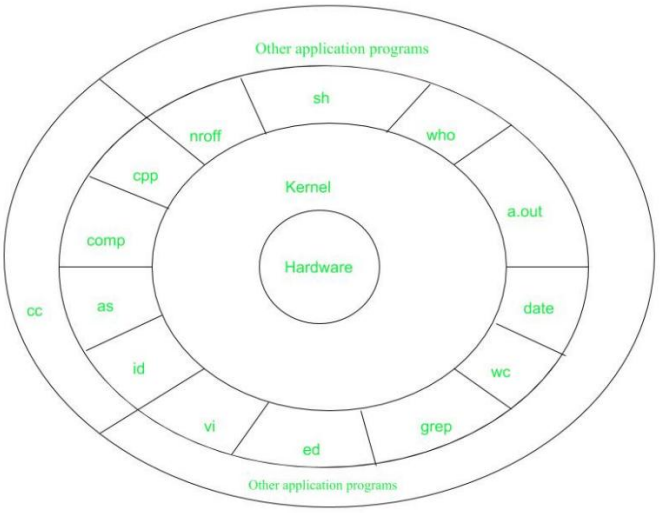


Figure – System Structure

- **Layer-1: Hardware:** It consists of all hardware related information.
- **Layer-2: Kernel:** This is the core of the Operating System. It is a software that acts as the interface between the hardware and the software. Most of the tasks like memory management, file management, network management, process management, etc., are done by the kernel.
- **Layer-3: Shell commands:** This is the interface between the user and the kernel. Shell is the utility that processes your requests. When you type in a command at the terminal, the shell interprets the command and calls the program that you want. There are various commands like cp, mv, cat, grep, id, wc, nroff, a.out and more.
- **Layer-4: Application Layer:** It is the outermost layer that executes the given external applications.

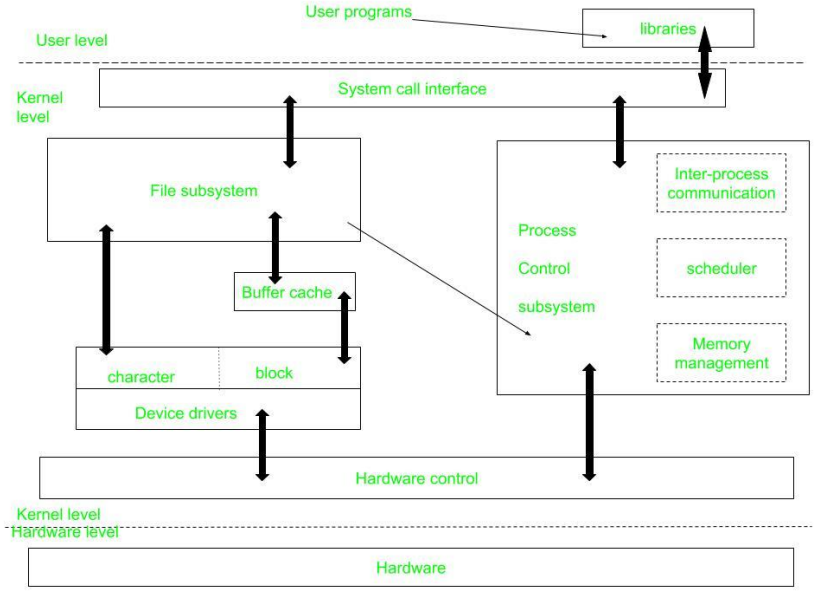


Figure – Kernel and its Block Diagram

1.2.2 This diagram shows three levels: user, kernel, and hardware.

- The system call and library interface represent the border between user programs and the kernel. System calls look like ordinary function calls in C programs. Assembly language programs may invoke system calls directly without a system call library. The libraries are linked with the programs at compile time.
- The set of system calls into those that interact with the file subsystem and some system calls interact with the process control subsystem. The file subsystem manages files, allocating file space, administering free space, controlling access to files, and retrieving data for users.
- Processes interact with the file subsystem via a specific set of system calls, such as open (to open a file for reading or writing), close, read, write, stat (query the attributes of a file), chown (change the record of who owns the file), and chmod (change the access permissions of a file).
- The file subsystem accesses file data using a buffering mechanism that regulates data flow between the kernel and secondary storage devices. The buffering mechanism interacts with block I/O device drivers to initiate data transfer to and from the kernel.
- Device drivers are the kernel modules that control the operator of peripheral devices. The file subsystem also interacts directly with "raw" I/O device drivers without the intervention of the buffering mechanism. Finally, the hardware control is responsible for handling interrupts and for communicating with the machine. Devices such as disks or terminals may interrupt the CPU while a process is executing. If so, the kernel may resume execution of the interrupted process after servicing the interrupt.
- Interrupts are not serviced by special processes but by special functions in the kernel, called in the context of the currently running process.

1.3 Difference Between Unix and Linux

Linux is essentially a clone of Unix. But, basic differences are shown below:

Linux	Unix
The source code of Linux is freely available to its users	The source code of Unix is not freely available general public
It has graphical user interface along with command line interface	It only has command line interface
Linux OS is portable, flexible, and can be executed in different hard drives	Unix OS is not portable
Different versions of Linux OS are Ubuntu, *Linux Mint*, RedHat Enterprise Linux, etc.	Different version of Unix are AIS, HP-UX, BSD, Iris, Solaris, etc.

Linux	Unix
The file systems supported by Linux are as follows: xfs, ramfs, vfat, cramfsm, ext3, ext4, ext2, ext1, ufs, autofs, devpts, ntfs	The file systems supported by Unix are as follows: zfs, js, hfx, gps, xfs, vxfs
Linux is an open-source operating system that was first released in 1991 by Linus Torvalds.	Unix is a proprietary operating system that was originally developed by AT&T Bell Labs in the mid 1960s.
The Linux kernel is monolithic, meaning that all of its services are provided by a single kernel.	The Unix kernel is modular, meaning that it is made up of a collection of independent modules that can be loaded and unloaded dynamically.
Linux has much broader hardware support than Unix.	Unix was originally designed to run on large, expensive mainframe computers, while Linux was designed to run on commodity hardware like PCs and servers.
User Interface of Linux is Graphical or text-based.	User Interface of unix is text-based.
Command Line Interface of Linux is Bash, Zsh, Tcsh.	Command Line Interface of unix is Bourne, Korn, C, Zsh.

1.4 What is Linux Operating System?

Developed by Linus Torvalds in 1991, the Linux operating system is a powerful and flexible open-source software platform. It acts as the basis for a variety of devices, such embedded systems, cell phones, servers, and personal computers. Linux, that's well-known for its reliability, safety, and flexibility, allows users to customize and improve their environment to suit specific needs. With an extensive and active community supporting it, Linux is an appealing choice for people as well as companies due to its wealth of resources and constant developments.

1.4.1 What is a "distribution?"

Linux distribution is an operating system that is made up of a collection of software based on Linux kernel or you can say distribution contains the Linux kernel and supporting libraries and software. And you can get Linux-based operating system by downloading one of the Linux distributions and these distributions are available for different types of devices like embedded devices, personal computers, etc. Around **600 + Linux Distributions** are available and some of the popular Linux distributions are:

- MX Linux
- Manjaro
- Linux Mint
- elementary
- Ubuntu

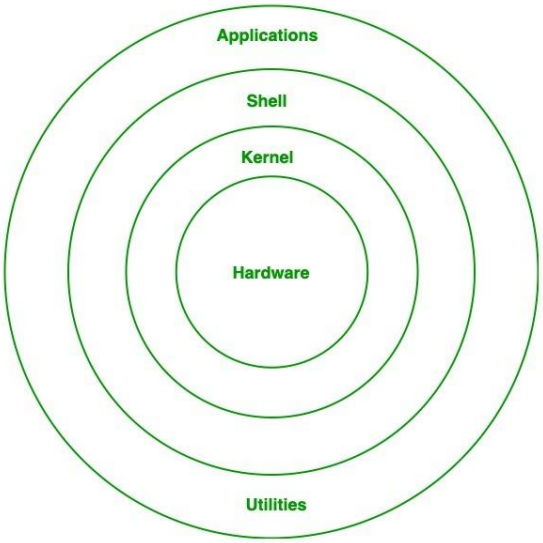
- Debian
- Solus
- Fedora
- openSUSE
- Deepin

1.4.2 Why use Linux?

Because it is free, open-source, and extremely flexible, Linux is widely utilized. For servers and developers, it is the ideal option because it offers strong security, stability, and performance. Generally interoperable hardware, a broad software library, and a vibrant community that offers support and regular updates are the many benefits of Linux. Due to its adaptability, users can customize the operating system according to their own needs, whether they become for personal or large enterprise use.

1.4.3 Architecture of Linux

Linux architecture has the following components:



Linux Architecture

1. **Kernel:** Kernel is the core of the Linux based operating system. It virtualizes the common hardware resources of the computer to provide each process with its virtual resources. This makes the process seem as if it is the sole process running on the machine. The kernel is also responsible for preventing and mitigating conflicts between different processes. Different types of the kernel are:
   - Monolithic Kernel
   - Hybrid kernels
   - Exo kernels
   - Micro kernels

2. **System Library:** Linux uses system libraries, also known as shared libraries, to implement various functionalities of the operating system. These libraries contain pre-written code that applications can use to perform specific tasks. By using these libraries, developers can save time and effort, as they don't need to write the same code repeatedly. System libraries act as an interface between applications and the kernel, providing a standardized and efficient way for applications to interact with the underlying system.
3. **Shell:** The shell is the user interface of the Linux Operating System. It allows users to interact with the system by entering commands, which the shell interprets and executes. The shell serves as a bridge between the user and the kernel, forwarding the user's requests to the kernel for processing. It provides a convenient way for users to perform various tasks, such as running programs, managing files, and configuring the system.
4. **Hardware Layer:** The hardware layer encompasses all the physical components of the computer, such as RAM (Random Access Memory), HDD (Hard Disk Drive), CPU (Central Processing Unit), and input/output devices. This layer is responsible for interacting with the Linux Operating System and providing the necessary resources for the system and applications to function properly. The Linux kernel and system libraries enable communication and control over these hardware components, ensuring that they work harmoniously together.
5. **System Utility:** System utilities are essential tools and programs provided by the Linux Operating System to manage and configure various aspects of the system. These utilities perform tasks such as installing software, configuring network settings, monitoring system performance, managing users and permissions, and much more. System utilities simplify system administration tasks, making it easier for users to maintain their Linux systems efficiently.

1.4.4 How is the Linux Operating System Used

The Linux operating system is widely used across various domains due to its flexibility, security, and open-source nature:

- **Servers and Hosting**: Powers web servers, cloud infrastructure, and database management systems.
- **Development**: Used by developers for coding, debugging, and running applications.
- **Desktop and Personal Use**: Provides secure and customizable desktop environments.
- **Cybersecurity**: Essential for ethical hacking, penetration testing, and security research.
- **Embedded Systems**: Runs lightweight devices like routers, IoT gadgets, and smart appliances.
- **Supercomputers**: Dominates high-performance computing for scientific research and simulations.

- **Education**: A cost-effective tool for teaching programming and system administration.

1.4.5 Which distribution is right for you?

Choosing the right Linux distribution depends on your needs and experience level:

- **For Beginners:** Because of its simple user interface and strong community support, Ubuntu is a wonderful choice for initially Linux users. On the opposite hand, Linux Mint make it straightforward for novices to transition to Linux by offering an experience comparable to Windows out of the box.
- **For Advanced Users:** Advanced users who appreciate customization and direct control might opt for Arch Linux, it is known for its simplistic style and ability to create highly unique systems from the ground up. Another choice is Gentoo, that provides total control of the system but requires manual setup and a lengthy learning process.
- **For Developers:** Fedora was a popular choice among developers due to its focus upon modern technology and software, making it a perfect platform for software testing and development. On the other hand, Debian is well known for its reliability and extensive package repository, which implies it may be used in both production and development environments.
- **For Servers:** For server environments, CentOS is a powerful, community-maintained distribution that matches Red Hat Enterprise Linux (RHEL) quite somewhat. As an alternative, Ubuntu Server offers an extensive list of server applications in addition to strong community support and ease of use.
- **For Lightweight Systems:** Lubuntu is frequently picked by users either like lightweight operating systems or have outdated equipment due to its ability to utilize system resources efficiently while maintaining functionality. Another slim option is Puppy Linux, that is made to run well on outdated hardware while maintaining the essential functions and applications.

1.4.6 Advantages of Linux

1. **Free & Open-Source** – Linux is completely free and anyone can view, change, or share its code.
2. **Safe & Secure** – Linux is less likely to get viruses and doesn't usually need antivirus software.
3. **Fast Updates** – Software updates are quick, frequent, and easy to install.
4. **Stable & Reliable** – It rarely crashes or slows down and doesn't need frequent reboots.
5. **Customizable** – You can install only what you need, making it flexible for any use.
6. **Runs on Any PC** – Works even on old computers and is easy to install from the internet.
7. **Strong Community** – Huge support from global users and developers if you need help.

1.4.7 Disadvantages of Linux

- It is not very user-friendly. So, it may be confusing for beginners.
- It has small peripheral hardware drivers as compared to windows.

1.5 What are Shell Commands in Linux?

A shell **in Linux** is a program that serves as an interface between the **user and the operating system.** It accepts commands from the user, interprets them, and passes them to the operating system for execution. The commands can be used for a wide range of tasks, from **file manipulation** to **system management**.

Some of the essential **basic shell commands** in **Linux** for different operations are:

- **File Management ->** cp, mv, rm, mkdir
- **Navigation ->** cd, pwd, ls
- **Text Processing ->** cat, grep, sort, head
- **System Monitoring ->** top, ps, df
- **Permissions and Ownership ->** chmod, chown, chgrp
- **Networking – >** ping, wget, curl, ssh, scp, ftp
- **Compression and Archiving – >** tar, gzip, gunzip, zip, unzip
- **Package Management – >** dnf, yum, apt-get
- **Process Management ->** kill, killall, bg, killall, kill

1.5.1 Basic Shell Commands for File and Directory Management

Command	Description	Example
ls	Lists files and directories	ls
cd	Changes the current directory	cd /home/user/Documents
pwd	Displays the current directory path	pwd
mkdir	Creates a new directory	mkdir new_directory
rm	Removes files or directories	rm file.txt
cp	Copies files or directories	cp file1.txt file2.txt
mv	Moves or renames files and directories	mv old_name new_name
touch	Creates an empty file or updates file timestamps	touch newfile.txt

Examples:
1. List files in a directory:

```
ls
```

2. Change directory:

```
cd/home/user
```

3. Create a new directory:

```
mkdir new_directory
```

4. Copy a file from one location to another:

```
cp source.txt destination.txt
```

5. Remove a file:

```
rm file.txt
```

1.5.2 Text Processing Commands in Linux

Command	Description	Example
cat	Displays the contents of a file	cat file.txt
grep	Searches for a pattern in a file	grep "error" log.txt
sort	Sorts the contents of a file	sort file.txt
head	Displays the first few lines of a file	head file.txt
tail	Displays the last few lines of a file	tail file.txt
wc	Counts the lines, words, and characters in a file	wc file.txt

Examples:

1. Display the contents of a file:

```
cat file.txt
```

2. Search for a pattern in a file:

```
grep "error" file.txt
```

3. Sort the contents of a file:

```
sort file.txt
```

4. Display the first 10 lines of a file:

```
head file.txt
```

5. Display the last 10 lines of a file:

```
tail file.txt
```

1.5.3 File Permissions and Ownership Commands

Command	Description	Example
chmod	Changes file permissions	chmod 755 file.txt
chown	Changes file owner and group	chown user:group file.txt
chgrp	Changes file group ownership	chgrp group file.txt

Examples:

1. Change permissions of a file:

```
chmod 755 file.txt
```

2. Change the owner of a file:

```
chown user:group file.txt
```

1.5.4 System Monitoring and Process Management Commands

Command	Description	Example
top	Displays real-time system information (CPU, memory)	top
ps	Displays the list of running processes	ps aux
kill	Terminates a process by its ID	kill 1234
df	Displays disk space usage	df -h

Examples:
1. View running processes:
```
ps aux
```
2. Display real-time system statistics:
```
top
```
3. Kill a process by its ID:
```
kill 1234
```
4. Check disk space usage:
```
df -h
```

1.5.5 Networking Shell Commands

Command	Description	Example
ping	Checks the network connection to a server	ping example.com
wget	Retrieves files from the web	wget http://example.com/file.zip
curl	Transfers data from or to a server	curl http://example.com
ssh	Opens SSH client (remote login program)	ssh user@example.com
scp	Securely copies files between hosts	scp file.txt user@example.com:/path/
ftp	Transfers files using the File Transfer Protocol	ftp ftp.example.com

Examples
1. Check the network connection to a server:
- **Command:** ping
- **Example:** ping example.com

2. Retrieve files from the web:
- **Command:** wget
- **Example:** wget http://example.com/file.zip

3. Transfer data from or to a server:
- **Command:** curl
- **Example:** curl http://example.com

4. **Open SSH client (remote login program):**
   - **Command:** ssh
   - **Example**: ssh user@example.com
5. **Securely copy files between hosts:**
   - **Command:** scp
   - **Example:** scp file.txt user@example.com:/path/
6. **Transfer files using the File Transfer Protocol:**
   - **Command:** ftp
   - **Example:** ftp ftp.example.com

1.5.6 Advanced Shell Commands

Command	Description	Example
find	Searches for files and directories	find /home/user -name "*.txt"
tar	Archives files into a tarball (.tar) or extracts them	tar -cvf archive.tar file1.txt file2.txt
ssh	Connects to a remote machine via SSH	ssh user@remote_host

Examples:
1. Find files in a directory:
```
find /home/user -name "*.txt"
```
2. Create a tarball archive:
```
tar -cvf archive.tar file1.txt file2.txt
```
3. Connect to a remote machine using SSH:
```
ssh user@remote_host
```

1.6 Comprehensive Overview of Linux Commands

Linux commands are the keystones of system interaction and management, enabling users to perform a wide array of tasks. This section provides a categorized overview of basic linux commands, aiming to offer both a refresher for seasoned users and a guide for newcomers.

1.6.1 Navigation

Navigating the file system is a fundamental skill for any Linux user. These Linux directory commands in this category allow you to move between directories, list their contents, and display your current location within the filesystem hierarchy. Mastering these commands is the first step toward proficiently managing Linux environments.

Command	Command Syntax	Description
cd	cd /path/to/directory	Change the current directory.
	cd ../	Change to the parent directory of the current directory.
ls	ls -lah	List all files and directories, including hidden ones, with detailed information.

pwd	pwd	Will output the current directory's path.

1.6.2 File Operations

File operations are important for managing the data within your system. This category includes commands for creating, copying, moving, and deleting files. Understanding these operations is essential for organizing and maintaining your file system effectively.

Command	Command Syntax	Description
touch	touch newfile.txt	Create a new file or update the timestamp of an existing file.
cp	cp source.txt destination.txt	Copy files from source to destination.
mv	mv oldname.txt newname.txt	Move or rename files or directories.
rm	rm file.txt	Delete files or directories.

1.6.3 Directory Operations

Beyond individual files, managing directories is equally important. This section covers the creation and removal of directories, providing the tools needed to structure and clean up your file system.

Command	Command Syntax	Description
mkdir	mkdir newdir	Create a new directory.
rmdir	rmdir emptydir	Remove an empty directory.
rm -r	rm –r /path/to/directory/	Remove a directory no matter if it is empty

1.6.4 Viewing Content Commands in Linux

Viewing the content of files is a daily necessity. This category presents commands for displaying file contents in various ways, whether you need to view an entire file or just a specific part of it.

Command	Command Syntax	Description
cat	cat file.txt	Displays the content of files.
less	less file.txt	View content of a file one page at a time.
head	head file.txt	Displays the first few lines of a file.
tail	tail file.txt	Displays the last few lines of a file.

1.6.5 Managing Permissions Commands in Linux

Securing and managing access to files and directories is a critical aspect of Linux administration. This subchapter elucidates commands that modify file permissions and ownership, ensuring that only authorized users can access or modify specific files.

Command	Command Syntax	Description
chmod	chmod [permission] script.sh	Change file mode bits to set permissions.
chown	chown user:group file.txt	Change file owner and group.

1.6.6 Text Manipulation Commands in Linux

Manipulating text files is a common task, whether for programming, configuration, or data processing. This section introduces commands for searching and editing text within files, showcasing the versatility and power of the command line for handling text.

Command	Command Syntax	Description
grep	grep 'pattern' file.txt	Search text using patterns.
awk	awk '/pattern/ {action}' file.txt	Pattern scanning and processing language.
sed	sed 's/original/replacement/' file.txt	Stream editor for filtering and transforming text.

1.6.7 Linux Networking Commands

In today's interconnected world, managing network settings and diagnosing connections are indispensable skills. This subchapter covers commands for examining network configurations and testing connectivity, vital for ensuring your system communicates effectively with other systems and services.

Command	Command Syntax	Description
ping	ping example.com	Check the network connection to a server.
ifconfig	ifconfig	Display or configure the network interface. (Note: ifconfig is deprecated in favor of ip)
netstat	netstat -tuln	Show network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

1.7 Job Control

In the Linux operating system, jobs refer to processes that are running in the background or foreground. Job control refers to the ability to manipulate these processes, including suspending, resuming, and terminating them. This can be useful for managing multiple tasks or for debugging problems with a process.

Job control is made possible by the shell, which is a command-line interface that allows users to interact with the operating system. The most common shell in Linux is the Bourne Again Shell **(BASH)**, but other shells such as the Z Shell **(ZSH)** and the Korn Shell **(KSH)** are also available.

In this article, we will explore the basics of job control in Linux and how to use it to manage processes.

1.7.1 Understanding Processes and Jobs in Linux

In Linux, every program that is running is considered a process. A process can be a standalone program or a part of a larger program.

Each process is assigned a unique identifier called a process ID **(PID)**. The **PID** can be used to refer to the process and perform actions on it, such as suspending or terminating it.

A job is a process that is either running in the foreground or the background. The foreground is the active window in the terminal, and the background is any process that is running but not actively being used in the terminal.

By default, when you run a command in the terminal, it runs in the foreground. You can tell that a process is running in the foreground because it displays output and you cannot enter any more commands until it finishes.

To run a process in the background, you can use the **&** symbol at the end of the command.

Example:

```
$ sleep 30 &
[1] 12345
```

In this example, the **sleep** command causes the process to sleep for 30 seconds. The & symbol causes the process to run in the background, and the output **[1] 12345** indicates that it is job number 1 with a PID of 12345.

1.7.2 Managing Jobs with the fg and bg Commands

The **fg** (foreground) and **bg** (background) commands allow you to move jobs between the foreground and the background.

To bring a background job to the foreground, you can use the **fg** command followed by the job number or the PID.

Example

```
$ fg %1
sleep 30
```

This brings the **sleep** command, which is job number 1, to the foreground and displays its output.

To send a foreground job to the background, you can use the **bg** command followed by the job number or the PID.

Example

```
$ sleep 30
[1] 12345
^Z
[1]+ Stopped      sleep 30
$ bg %1
[1]+ sleep 30 &
```

In this example, the **sleep** command is run in the foreground and then suspended with the **^Z** keyboard shortcut. The **bg** command is then used to resume the job in the background.

1.7.3 Suspend or Resume Jobs in Linux

The **suspend** command allows you to temporarily stop a job, while the **kill** command allows you to permanently terminate a job.

To suspend a job, you can use the **suspend** command followed by the job number or the PID.

Example

```
$ sleep 30 &
[1] 12345
$ suspend %1
[1]+ Suspended
```

This suspends the **sleep** command, which is job number 1. The job can then be resumed with the **fg** command or left suspended in the background.

To terminate a job, you can use the **kill** command followed by the job number or the PID.

Example

```
$ sleep 30 &
[1] 12345
$ kill %1
```

This terminates the **sleep** command, which is job number 1.

1.7.4 View and Manage Jobs with the jobs and ps Commands

The **jobs** command allows you to view a list of all jobs running in the background or suspended in the current shell. The **ps** command allows you to view a list of all processes running on the system.

Example

```
$ sleep 30 &
[1] 12345
$ sleep 60 &
[2] 12346
$ jobs
[1] Running    sleep 30 &
[2] Running    sleep 60 &
```

You can use the **jobs** command to view the status of each job and its job number or PID.

The **ps** command allows you to view a list of all the processes that are currently running on the system. You can use the **-a** flag to show all processes and the **-x** flag to show processes that are not associated with a terminal.

Example

```
$ ps -ax
  PID TTY   STAT    TIME COMMAND
    1 ?    Ss      0:00 /sbin/init
    2 ?    S       0:00 [kthreadd]
    3 ?    S       0:00 [ksoftirqd/0]
    4 ?    S       0:00 [kworker/0:0]
...
```

The **ps** command displays the PID, terminal (TTY), status, time, and command for each process.

1.8 What is the Linux File System

The Linux file system is a multifaceted structure comprised of three essential layers. At its foundation, the Logical File System serves as the interface between user applications and the file system, managing operations like opening, reading, and closing files. Above this, the Virtual File System facilitates the concurrent operation of multiple physical file systems,

providing a standardized interface for compatibility. Finally, the Physical File System is responsible for the tangible management and storage of physical memory blocks on the disk, ensuring efficient data allocation and retrieval. Together, these layers form a cohesive architecture, orchestrating the organized and efficient handling of data in the Linux operating system.

In this article, we will be focusing on the **file system for hard disks on a Linux OS** and discuss which type of file system is suitable. The architecture of a file system comprises three layers mentioned below.

1.8.1 Linux File System Structure

A file system mainly consists of 3 layers. From top to bottom:

1. Logical File System:

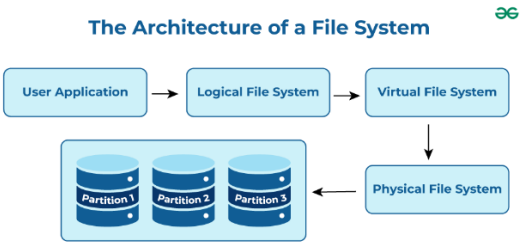
The Logical File System acts as the interface between the user applications and the file system itself. It facilitates essential operations such as opening, reading, and closing files. Essentially, it serves as the user-friendly front-end, ensuring that applications can interact with the file system in a way that aligns with user expectations.

2. Virtual File System:

The Virtual File System (VFS) is a crucial layer that enables the concurrent operation of multiple instances of physical file systems. It provides a standardized interface, allowing different file systems to coexist and operate simultaneously. This layer abstracts the underlying complexities, ensuring compatibility and cohesion between various file system implementations.

3. Physical File System:

The Physical File System is responsible for the tangible management and storage of physical memory blocks on the disk. It handles the low-level details of storing and retrieving data, interacting directly with the hardware components. This layer ensures the efficient allocation and utilization of physical storage resources, contributing to the overall performance and reliability of the file system.



Architecture Of a File System

1.8.2 Characteristics of a File System

- **Space Management**: how the data is stored on a storage device. Pertaining to the memory blocks and fragmentation practices applied in it.

- **Filename**: a file system may have certain restrictions to file names such as the name length, the use of special characters, and case sensitive-ness.
- **Directory**: the directories/folders may store files in a linear or hierarchical manner while maintaining an index table of all the files contained in that directory or subdirectory.
- **Metadata**: for each file stored, the file system stores various information about that file's existence such as its data length, its access permissions, device type, modified date-time, and other attributes. This is called metadata.
- **Utilities**: file systems provide features for initializing, deleting, renaming, moving, copying, backup, recovery, and control access of files and folders.
- **Design**: due to their implementations, file systems have limitations on the amount of data they can store.

1.8.3 Some important terms:

1) Journaling:

Journaling file systems keep a log called the journal, that keeps track of the changes made to a file but not yet permanently committed to the disk so that in case of a system failure the lost changes can be brought back.

2) Versioning:

Versioning file systems store previously saved versions of a file, i.e., the copies of a file are stored based on previous commits to the disk in a minutely or hourly manner to create a backup.

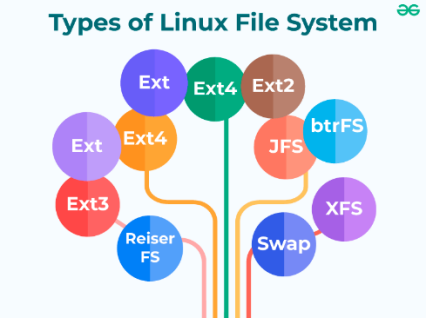
3) Inode:

The index node is the representation of any file or directory based on the parameters – size, permission, ownership, and location of the file and directory.

Now, we come to part where we discuss the various implementations of the file system in Linux for disk storage devices.

1.8.4 Linux File Systems:

Note: *Cluster and distributed file systems will not be included for simplicity.*



Types of File System in Linux

1) ext (Extended File System):

Implemented in 1992, it is the first file system specifically designed for Linux. It is the first member of the ext family of file systems.

2) ext2:

The second ext was developed in 1993. It is a non-journaling file system that is preferred to be used with flash drives and SSDs. It solved the problems of separate timestamp for access, inode modification and data modification. Due to not being journaled, it is slow to load at boot time.

3) Xiafs:

Also developed in 1993, this file system was less powerful and functional than ext2 and is no longer in use anywhere.

4) ext3:

The third ext developed in 1999 is a journaling file system. It is reliable and unlike ext2, it prevents long delays at system boot if the file system is in an inconsistent state after an unclean shutdown. Other factors that make it better and different than ext2 are online file system growth and HTree indexing for large directories.

5) JFS (Journaled File System):

First created by IBM in 1990, the original JFS was taken to open source to be implemented for Linux in 1999. JFS performs well under different kinds of load but is not commonly used anymore due to the release of ext4 in 2006 which gives better performance.

6) ReiserFS:

It is a journal file system developed in 2001. Despite its earlier issues, it has tail packing as a scheme to reduce internal fragmentation. It uses a B+ Tree that gives less than linear time in directory lookups and updates. It was the default file system in SUSE Linux till version 6.4, until switching to ext3 in 2006 for version 10.2.

7) XFS:

XFS is a 64-bit journaling file system and was ported to Linux in 2001. It now acts as the default file system for many Linux distributions. It provides features like snapshots, online defragmentation, sparse files, variable block sizes, and excellent capacity. It also excels at parallel I/O operations.

8) SquashFS:

Developed in 2002, this file system is read-only and is used only with embedded systems where low overhead is needed.

9) Reiser4:

It is an incremental model to ReiserFS. It was developed in 2004. However, it is not widely adapted or supported on many Linux distributions.

10) ext4:

The fourth ext developed in 2006, is a journaling file system. It has backward compatibility with ext3 and ext2 and it provides several other features, some of which are persistent pre-allocation, unlimited number of subdirectories, metadata checksumming and large file size. ext4 is the default file system for many Linux distributions and also has compatibility with Windows and Macintosh.

11) btrfs (Better/Butter/B-tree FS):

It was developed in 2007. It provides many features such as snapshotting, drive pooling, data scrubbing, self-healing and online defragmentation. It is the default file system for Fedora Workstation.

12) bcachefs:

This is a copy-on-write file system that was first announced in 2015 with the goal of performing better than btrfs and ext4. Its features include full filesystem encryption, native compression, snapshots, and 64-bit check summing.

13) Others:

Linux also has support for file systems of operating systems such as NTFS and exFAT, but these do not support standard Unix permission settings. They are mostly used for interoperability with other operating systems.

Note:

XFS, ext4 and btrfs perform the best of all the other file systems. In fact, btrfs looks as if it's almost the best. Despite that, the ext family of file systems has been the default for most Linux distributions for a long time. So, what is it that made the developers choose ext4 as the default rather than btrfs or XFS? Since ext4 is so important for this discussion, let's describe it a bit more.

1.8.5 ext4 in Linux File System

Ext4 was designed to be backward compatible with ext3 and ext2, its previous generations. It's better than the previous generations in the following ways:

- It provides a **large file system** as described in the table above.
- Utilizes **extents** that improve large file performance and reduces fragmentation.
- Provides **persistent pre-allocation** which guarantees space allocation and contiguous memory.
- **Delayed allocation** improves performance and reduces fragmentation by effectively allocating larger amounts of data at a time.
- It uses HTree indices to allow **unlimited number of subdirectories.**

- Performs **journal checksumming** which allows the file system to realize that some of its entries are invalid or out of order after a crash.
- Support for time-of-creation timestamps and **improved timestamps** to induce granularity.
- Transparent encryption.
- Allows cleaning of inode tables in background which in turn speeds initialization. The process is called **lazy initialization**.
- Enables **writing barriers** by default. Which ensures that file system metadata is correctly written and ordered on disk, even when write caches lose power.

There are still some features in the process of developing like metadata checksumming, first-class quota supports, and large allocation blocks.

However, ext4 has some limitations. Ext4 does not guarantee the integrity of your data, if the data is corrupted while already on disk then it has no way of detecting or repairing such corruption. The ext4 file system cannot secure deletion of files, which is supposed to cause overwriting of files upon deletion. It results in sensitive data ending up in the file-system journal.

XFS performs highly well for large filesystems and high degrees of concurrency. So XFS is stable, yet there's not a solid borderline that would make you choose it over ext4 since both work about the same. Unless you want a file system that directly solves a problem of ext4 like having capacity > 50TiB.

Btrfs on the other hand, despite offering features like multiple device management, per-block checksumming, asynchronous replication and inline compression, does not perform the best in many common use cases as compared to ext4 and XFS. Several of its features can be buggy and result in reduced performance and data loss.

1.9 Some HandsOn Example on Linux File System

For example, if our use_case is to set up a server that will first store and serve large multimedia files (videos and audios). In that case we have to prioritize efficient speed and use of storage space.

According to this requirement the XFS file system would be a better choice. Because we know that XFS is optimized for large files and can work on high volumes of data transfer which in general makes it ideal for media servers.

Following steps to use it:

Step 1: Installing XFS utilities package on Linux system.

```
sudo apt-get install xfsprogs
```

```
root@jayesh-VirtualBox:~# sudo apt-get install xfsprogs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
xfsprogs is already the newest version (5.13.0-1ubuntu2).
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
root@jayesh-VirtualBox:~#
```

Installing xfsprogs

Step 2: Create a partition to format as XFS.
For example: `/dev/sda1`
This can be done using tool like `fdisk`.
Step 3: Format the partition as XFS.

```
sudo mkfs.xfs /dev/sda1 -f
```

```
root@jayesh-VirtualBox:~# sudo mkfs.xfs /dev/sda1 -f
meta-data=/dev/sda1              isize=512    agcount=4, agsize=35392 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=0 inobtcount=0
data     =                       bsize=4096   blocks=141568, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=1368, version=2
         =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
root@jayesh-VirtualBox:~#
```

Format the partition

We have formatted partition using XFS filesystem. (Used -f for forcefully to avoid error or warning) .
Step 4: Mount the XFS partition to a directory we want.

```
sudo mount /dev/sda1 /mnt/jayesh_xfs_partition
```

```
root@jayesh-VirtualBox:~# sudo mount /dev/sda1 /mnt/jayesh_xfs_partition
root@jayesh-VirtualBox:~#
```

mounting of XFS partition

We have mounted XFS partition to a directory `/mnt/jayesh_xfs_partition`, (you can create your own directory.)
Step 5: To verify the mount.

```
df -h
```

```
root@jayesh-VirtualBox:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           342M  1.7M  340M   1% /run
/dev/sda3        27G  9.9G   16G  40% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
tmpfs           342M  176K  342M   1% /run/user/1000
/dev/sda1       548M   32M  516M   6% /mnt/jayesh_xfs_partition
root@jayesh-VirtualBox:~#
```

Successful mount

1.10.0 Installing Software on Linux

On Linux, installing software is simple. For Debian-based systems (like Ubuntu), use package managers like apt and sudo apt install package_name; for Fedora, use dnf and sudo dnf install package_name. Software centers are another source for a graphical application installation and searching interface.

1.10 Package Management

Linux distributions use a package manager to install packages. Package management in Linux can be explained using the following; Package is a set of files, metadata, and other information required to install any software, application, or tools on a Linux. These packages are required to install, update, and upgrade these tools. These tools and software and the Linux distribution are upgraded through Package managers. Instructions on how to update and upgrade packages are covered further in the article.

1.10.1 Core Concepts for Package Management

Regarding package management in Linux, package management is the term used to signify the installation and maintenance of Packages in your system. Package managers reduce the requirement for manually downloading and installing various dependencies required for the software.

1. Packages

A package contains all the necessary data required for the installation and maintenance of the software package. These packages are created by someone known as a package maintainer. A package maintainer takes care of the packages. They ensure active maintenance, bug fixes if any, and the final compilation of the package.

2. Repositories

These Packages are present in the Repositories that contain packages specially designed, compiled, and maintained for each Linux version and distribution. These Repositories contain thousands of packages created by the distribution vendors. Sometimes projects may handle their packaging and distribution.

3. Dependencies

Some packages might require some other pre-installed software to function correctly. A resource or software that a package depends on is called its dependency. Dependencies include metadata on how to build the code you depend on and information on where to find the files containing it. The package manager takes care of all these problems for you. It will install, modify, upgrade, update, and remove package files and provide dependency resolution. Resolving a dependency means suppose you have software that requires Python version 3.1. You are trying to install another software that requires version 3.3, which causes a conflict, and this conflict will be required to resolve before proceeding with the installation of this software. The package manager facilitates this. The package manager also ensures we receive the original

and authentic package by verifying their certificates and checksum to ensure they have not been modified.

1.10.2 What is a Package Manager in Linux?

In simple terms, a package manager is a software tool used for package management in Linux i.e. to manage the installation, removal, and updating of various software packages. It can be thought of as a hub for all software packages available for your system. The package manager keeps track of all the installed packages on the system, including their dependencies, and uses this information to resolve conflicts and handle updates.

Using a package manager in Linux can save us a lot of time and effort compared to manually installing software and its dependencies. When we install a package, the package manager automatically checks if any other software is required for it to work correctly and installs these dependencies for us. This relieves us from the problem of figuring out what other softwares are needed and manually installing them. A package manager can also automatically check for updates to installed packages and install them for us. This helps us to keep our system up-to-date and secure.

1.10.3 Most Widely used Package Manager in Linux

1. APT (Advanced Package Tool):

APT (Advanced Package Tool) is a powerful and widely used package manager in the Linux world. It's the default package manager for Debian-based distributions, including Debian itself, Ubuntu, and Linux Mint. APT simplifies software management, ensuring that users can easily install, update, and remove packages while taking care of dependencies. Let's explore APT in more detail, including its commands and real-world examples.

Key Features of APT:

- **Dependency Resolution:** APT is known for its robust dependency resolution capabilities. It automatically detects and installs any necessary libraries or packages required by the software you want to install. This ensures that the software functions correctly without manual intervention.
- **Repository Management:** APT connects to online repositories where software packages are hosted. Users can configure which repositories to use, allowing them to install software from official sources and trusted third-party repositories.
- **Package Management:** It allows users to manage packages on the system, including installation, upgrading, downgrading, and removal. Additionally, APT can lock packages to specific versions, ensuring system stability.
- **Cache Management:** APT maintains a local cache of package information. This cache helps in faster package searches and updates. Users can update this cache using the apt-get update command.

Basic commands in APT:

Installing a package:

```
sudo apt-get install package-name
```

Updating the package list:

```
sudo apt-get update
```

Upgrading packages:

```
sudo apt-get upgrade
```

Removing a package:

```
sudo apt-get remove package-name
```

Searching for packages:

```
apt-cache search package-name
```

2. YUM (Yellowdog Updater, Modified):

YUM (Yellowdog Updater, Modified) is a package manager primarily used in Red Hat-based Linux distributions such as CentOS, Fedora, and RHEL (Red Hat Enterprise Linux). YUM has played a significant role in simplifying package management on these systems, although it's important to note that in recent years, dnf (Dandified YUM) has become the modern and recommended replacement for YUM. Here, we will discuss YUM, its commands, and provide examples of its usage.

Key Features of YUM:

- **Dependency Resolution:** YUM, like APT, is adept at handling complex dependency resolution. It automatically identifies and installs any necessary dependencies for the packages you want to install, ensuring that your software functions correctly.
- **Repository Management:** YUM interacts with software repositories to fetch and install packages. Users can configure these repositories to obtain software from both official sources and trusted third-party repositories.
- **Package Management:** YUM provides a set of commands to manage packages on the system. This includes installing, updating, downgrading, and removing packages, as well as listing installed packages and their information.
- **Cache Management:** YUM maintains a local cache of metadata from the repositories. This cache improves package search and installation speed. You can update this cache using the yum makecache command.

Basic commands in YUM:

Installing a package:

```
sudo yum install package-name
```

Updating the package list:

```
sudo yum makecache
```

Upgrading packages:

```
sudo yum update
```

Removing a package:

```
sudo yum remove package-name
```

3. dnf Package Manager

dnf is the successor to YUM and is now the recommended package manager for Fedora-based distributions. It provides a more modern and user-friendly experience. The syntax and commands for dnf are similar to yum.

Some dnf examples are:
Installing a package:

```
sudo dnf install package-name
```

Updating packages:

```
sudo dnf upgrade
```

4. Pacman:

Pacman is the package manager used in Arch Linux and its derivatives, such as Manjaro. Arch Linux is known for its simplicity, flexibility, and rolling-release model, and Pacman is a vital component that makes managing software on Arch-based systems efficient. In this section, we'll explore Pacman, its commands, and provide examples of its usage.

Key Features of Pacman:

- **Simplicity and Transparency:** Pacman follows a simple and transparent approach to package management. It doesn't hide the details and allows users to understand precisely what's happening during package installation, upgrade, and removal.
- **Rolling Release:** Arch Linux and its derivatives use a rolling-release model, meaning that software is continuously updated rather than waiting for major distribution releases. Pacman plays a crucial role in keeping the system up-to-date with the latest software versions.
- **User-Managed Configuration:** Pacman gives users significant control over their system's configuration files. It provides options to save, overwrite, or merge configuration files during package upgrades, ensuring that the system stays tailored to the user's preferences.
- **AUR Support:** The Arch User Repository (AUR) is a community-driven repository for additional software not available in the official repositories. Pacman can be extended to manage packages from the AUR with AUR helpers like yay.

Basic commands in Pacman:
Installing a package:

```
sudo pacman -S package-name
```

Updating the package list and upgrading packages:

```
sudo pacman -Syu
```

Removing a package:

```
sudo pacman -R package-name
```

Querying package information:

```
pacman -Q package-name
```

5. Zypper:

Zypper is the default package manager used in openSUSE and its derivatives. It plays a critical role in managing software packages on these Linux distributions. Zypper is known for its efficiency and robust dependency resolution capabilities. In this section, we will delve into Zypper, its commands, and provide examples of its usage.

Key Features of Zypper:

- **Dependency Resolution:** Zypper is highly effective at handling dependencies. It automatically detects and resolves package dependencies, ensuring that all required components are installed, which is crucial for software to function correctly.
- **Repository Management:** Zypper interacts with software repositories, making it easy to fetch and install packages. openSUSE provides a wide range of official and community repositories, giving users access to a vast selection of software.
- **Package Management:** Zypper offers a range of package management commands, including installation, upgrading, downgrading, and removal. It provides users with control over their software packages.
- **Transaction Safety:** Zypper is designed with a focus on transaction safety. This means it will attempt to maintain a consistent system state by either completing a transaction successfully or rolling back changes in the event of an error, avoiding a partially updated system.

Basic commands in Zypper:

Installing a package:

```
sudo zypper in package-name
```

Updating packages:

```
sudo zypper up
```

Removing a package:

```
sudo zypper rm package-name
```

Searching for packages:

```
zypper se package-name
```

6. DPKG (Debian Package Manager):

DPKG (Debian Package Manager) is a fundamental package management tool for Debian-based Linux distributions, including Debian itself, Ubuntu, and their derivatives. DPKG is a low-level package manager, and while it directly interacts with individual package files, it is often used in conjunction with higher-level package managers like APT for a more user-

friendly experience. In this section, we'll explore DPKG, its commands, and provide examples of its usage.

Key Features of DPKG:

- **Low-Level Package Management:** DPKG directly handles the installation, removal, and management of individual package files (with the .deb extension). It does not automatically resolve or manage dependencies like higher-level package managers such as APT.
- **Package Installation:** DPKG is responsible for installing software packages on a Debian-based system. Users can install packages from local .deb files or by specifying package names if the packages are already present on the system.
- **Package Removal:** DPKG can remove installed packages from the system, ensuring that configuration files are retained or purged based on user preferences.
- **Package Querying:** Users can query the status and information about installed packages, which is useful for diagnosing issues or verifying package details.

Basic commands in DPKG:

Installing a package from a .deb file:

```
sudo dpkg -i package.deb
```

Removing a package:

```
sudo dpkg -r package-name
```

Querying package information:

```
dpkg -l | grep package-name
```

1.10.4 Comparing Package Managers and Installers

Though the functionalities of package managers and installers may sound similar, there are certain differences between them. To understand these differences, let us go through a brief comparison between package managers and installers-

	Package Managers	**Installers**
Definition	A package manager is a software tool used to manage software packages and their dependencies in a Linux distribution.	An installer is a software tool used to install a specific piece of software on a Linux system.
Function	Package managers are used to install, update, and remove software packages, as well as manage dependencies and repositories.	Installers are used to install a specific piece of software, usually with a graphical user interface (GUI) that guides the user through the installation process.

	Package Managers	Installers
Source of Packages	Package managers typically obtain packages from repositories maintained by the distribution or community, ensuring compatibility and security.	Installers may obtain packages from various sources, such as the developer's website or a third-party repository, which may result in compatibility or security issues.
Updates	Package managers can update multiple packages at once, and can handle dependencies and conflicts automatically.	Installers may require manual updates or may not support updates at all, which can lead to security vulnerabilities.
System Integration	Package managers integrate with the system's package database, ensuring consistency and compatibility with other software components.	Installers may not integrate with the system's package database, which can result in conflicts or version mismatches with other software components.

1.10.5 What is systemd?

Systemd is a system and service manager for Linux operating systems. It is designed to replace traditional init systems like SysV init and Upstart and is responsible for initializing the system, starting services, managing daemons, and monitoring system state. Its approach to managing processes is both efficient and powerful.

Some key features of systemd include:

- Parallel startup of services, which accelerates the boot process.
- Dependency management, ensuring that services are started in the right order.
- Cgroups integration for resource management.
- Enhanced logging through the journal system.
- Socket activation, which allows services to be started on-demand.

1.10.6 Introduction to systemctl

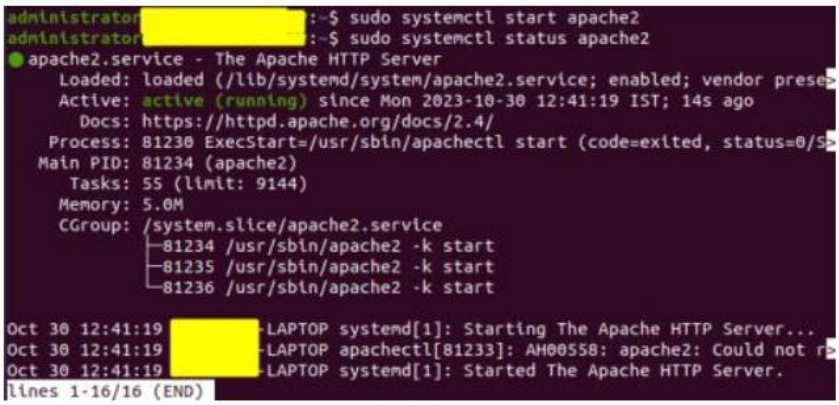
systemctl is the primary command-line interface for interacting with systemd. It provides a wide range of functions for managing services, viewing their status, and controlling the system's behavior. Let's explore some of the most common systemctl commands and their usage.

Basic systemctl Commands

Starting and Stopping Services

To start a service, you use the start command. For example, to start the Apache web server:

```
sudo systemctl start apache2
```



start service

To stop a service, use the stop command:

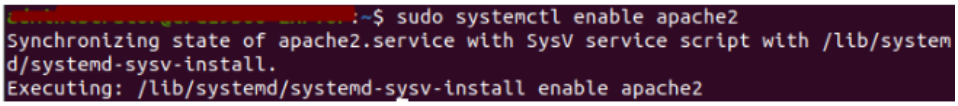
```
sudo systemctl stop apache2
```

Stop service

Enabling and Disabling Services

To ensure a service starts at boot, you enable it using the enable command:

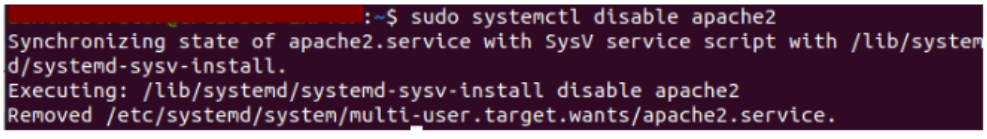
```
sudo systemctl enable apache2
```



Enable service

To disable a service from starting at boot:

```
sudo systemctl disable apache2
```



Disable sevices

Restarting and Reloading Services

To restart a service:

```
sudo systemctl restart apache2
```

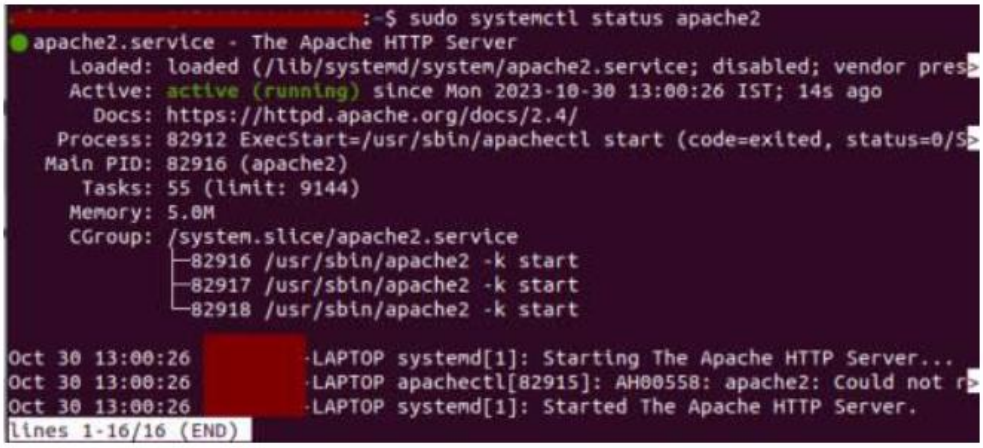
To reload configuration files without stopping the service:

```
sudo systemctl reload apache2
```

Checking Service Status

To view the status of a service, use the status command:

```
systemctl status apache2
```



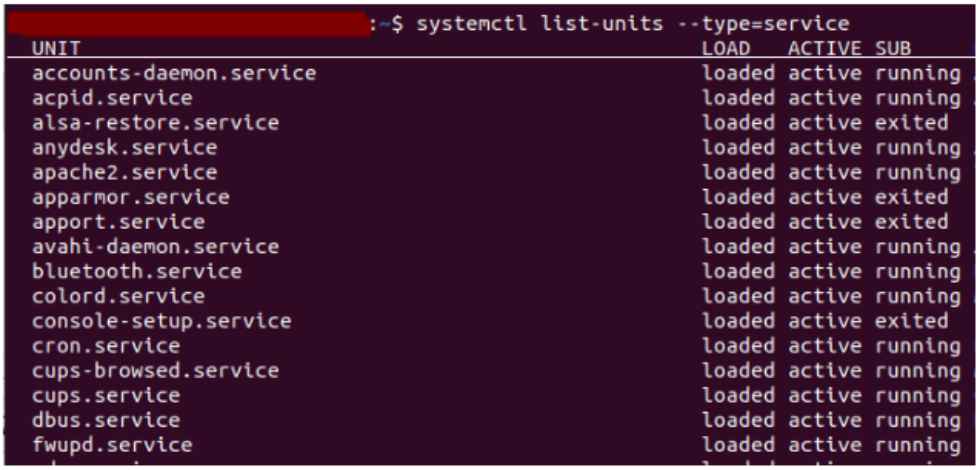
Checking status

This provides information about whether the service is running, its process ID, and recent log entries.

Viewing Active Units

You can list all currently active units (services, sockets, targets, etc.) with:

```
systemctl list-units --type=service
```



currently active units

Q. Why is it important for a system administrator to understand the hierarchy of the Linux file system? Describe the roles of any three critical system directories.

Understanding the hierarchy of the Linux file system is crucial for a system administrator because:

1. **Efficient System Management**: Admins need to know where to find logs, configurations, binaries, and user data to maintain and troubleshoot the system effectively.
2. **Security and Permissions**: Knowing the role of each directory helps in applying correct permissions and avoiding accidental data exposure or deletion.
3. **Backup and Recovery**: Understanding what each directory contains allows for proper backup planning — backing up only what matters and restoring systems quickly when needed.
4. **Package Installation and Configuration**: Most Linux software depends on standard directory structures. Knowing this helps during manual installations or custom setups.

Three Critical System Directories:

1. /etc

- **Role**: Contains system-wide configuration files.
- **Examples**: /etc/passwd (user account info), /etc/ssh/sshd_config (SSH server settings).
- **Why Important**: Any misconfiguration here can make the system unusable or insecure.

2. /var

- **Role**: Stores variable data — data that changes constantly.
- **Examples**: /var/log/ (log files), /var/spool/ (print and mail queues).
- **Why Important**: Crucial for monitoring system behavior and debugging. If /var fills up, services may fail.

3. /home

- **Role**: Contains personal directories for all users.
- **Examples**: /home/ajay, /home/john.
- **Why Important**: This is where user files, preferences, and personal scripts live. It's often a priority in backups.

Unit 2: Installation, Boot Process and User Administration

2.1 How to Install Linux Mint?

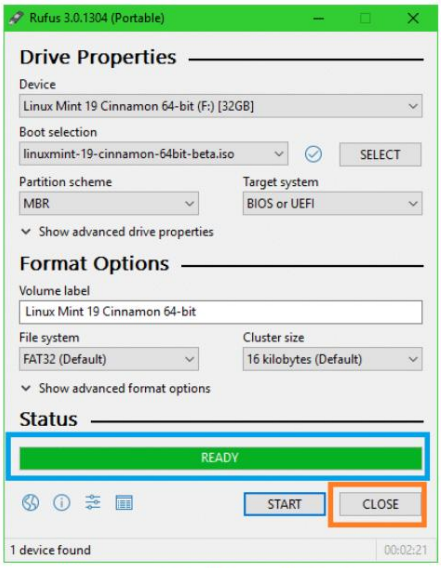
Linux Mint is the second-largest Linux-based distro used in the world. Linux Mint is a community-driven Linux distribution based on Ubuntu which itself is based on Debian and bundled with a variety of free and open-source applications. So here we discuss the installation of Linux mint.

Installation is divided into four parts:

- Creating Installation media
- Booting
- Installing
- Final setup

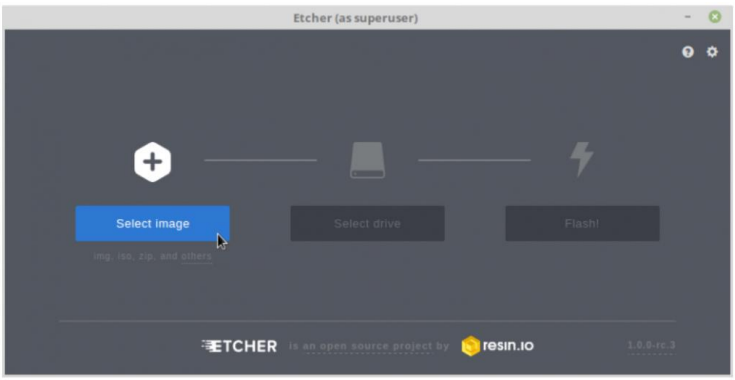
Phase 1: Creating installation media

- First thing is to download the Linux mint ISO file from the official website here. While downloading choose the desktop environment flavor for you.
- The next thing is burning the ISO to a USB flash drive. If you're using **Windows** Machine, We strongly recommend using Rufus.
- In Rufus, select the device like a USB flash device, Now click on the select button in boot selection and select the downloaded ISO file. Click on the start button to start flashing.



RUFUS BURNING LINUX MINT

If you're using a Linux machine, (We strongly recommend using Etcher). Use the same procedure as specified for Rufus.



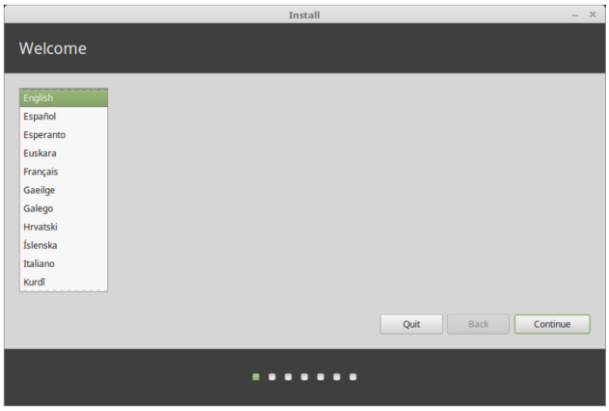
ETCHER

Phase 2: Booting

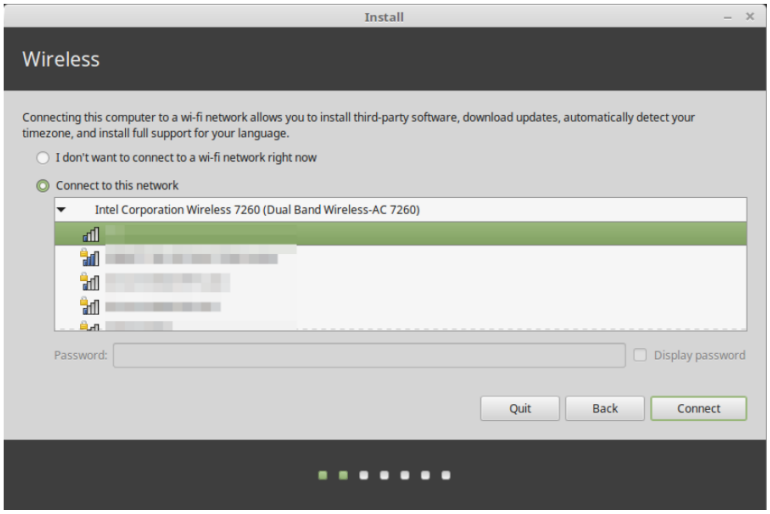
After Flashing, Unplug the USB flash drive and shutdown the target machine. Plugin the USB to the target machine. Turn on the machine and keep pressing the boot key. Boot key for the various machines can be found here. When the boot menu appears select the USB flash drive. The machine starts booting. Wait for some time till then the desktop appears. Double-click on the Install Linux Mint icon on the desktop.

Phase 3: Installation

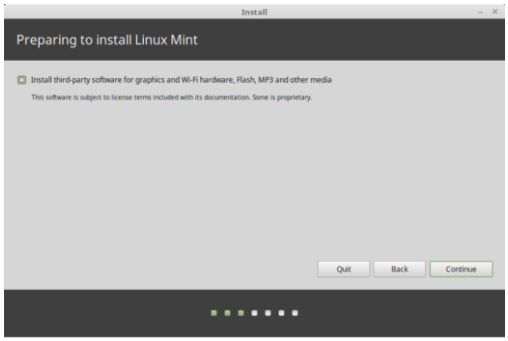
Select your language.



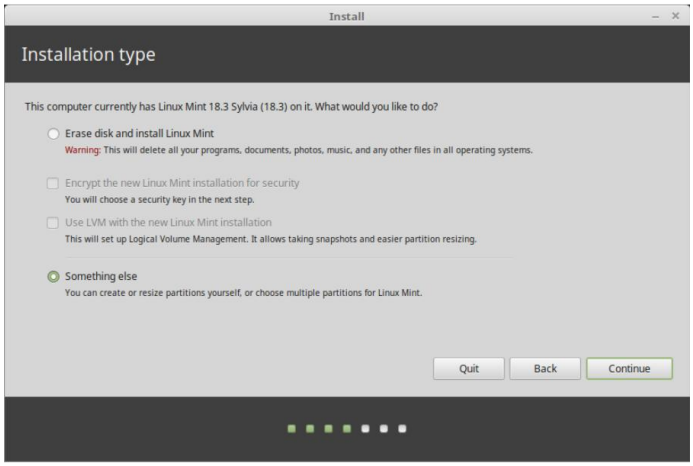
Then connect to the Internet:



If you are connected to the Internet, tick the box to install the multimedia codecs:



Choose an installation type. It is recommended to create a new partition for Linux(10 GB min) in EXT4 type and a swap partition(8 GB).



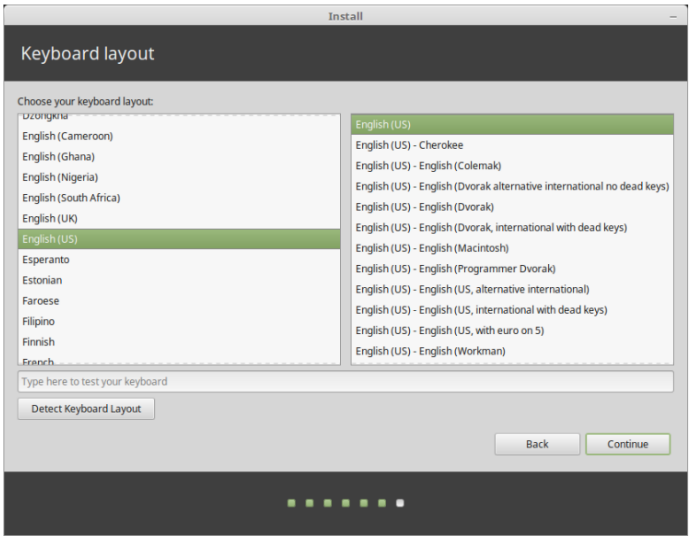
If Linux Mint is the only operating system you want to run on this computer and all data can be lost on the hard drive, choose to Erase disk and install Linux Mint.

If another operating system is present on the computer, the installer shows you an option to install Linux Mint alongside it. If you choose this option, the installer automatically resizes your existing operating system, makes room, and installs Linux Mint beside it. A boot menu is set up to choose between the two operating systems each time you start your computer.

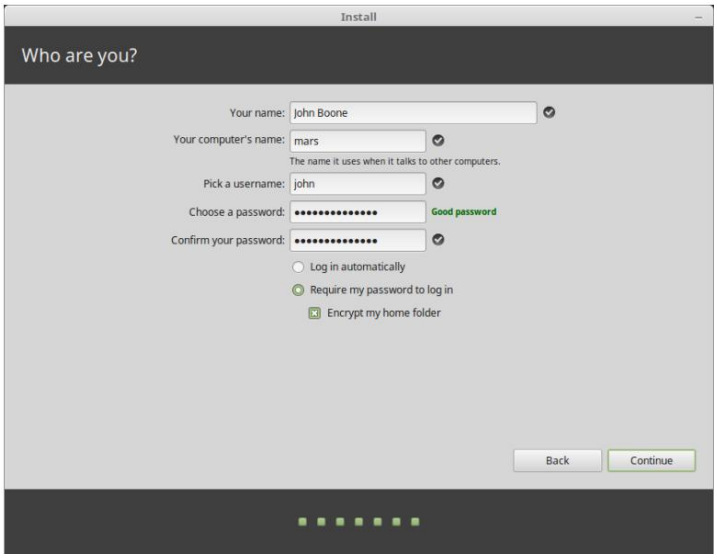
Click continue.

Next Select your timezone:

Select your keyboard layout:



Enter your user details:



Then click continue:

Installation may take a while. When the installation is finished, click Restart Now. The computer will then start to shut down and ask you to remove the USB disk (or DVD). Upon reboot, your computer should show you a boot menu or start your newly installed Linux Mint operating system.

2.2 How to Install Linux Over a Network

1. What is Network Installation?

Network installation allows you to install Linux on client machines **without using a DVD or USB drive**. Instead, the system boots over the network using PXE (Preboot Execution Environment), downloads the installer files, and starts the installation.

Why Use It?

- Ideal for **headless servers** (no GUI).
- Helpful when installing Linux on **multiple systems**.
- Ensures **latest packages** are used directly from the network source.

---

2. Prerequisites

Before starting, make sure you have:

- **Server Machine**: Running RHEL or another Linux distro (IP: 192.168.29.45).
- **Client Machine**: PXE boot enabled (configure in BIOS/UEFI).
- **DHCP Server**: To assign IP addresses to clients.
- **TFTP & NFS Services**: To serve boot and installation files.
- **VMware or Similar**: Use virtualization for testing.

---

3. Steps to Set Up Network Installation Server

Step 1: Install and Configure dnsmasq

dnsmasq is a lightweight DHCP and DNS server. It's simple and works well for local networks.

Install dnsmasq:

```
sudo dnf install dnsmasq
```

Edit Configuration File:

```
sudo nano /etc/dnsmasq.conf
```

Add these lines:

```
interface=eno1
bind-interfaces
dhcp-range=192.168.29.150,192.168.29.240,255.255.255.0,8h
dhcp-option=option:router,192.168.29.1
dhcp-option=option:dns-server,192.168.29.1
dhcp-boot=bootx64.efi,192.168.29.45
```

Explanation:

- interface=eno1: Network interface to listen on.
- dhcp-range: IP address pool for clients.
- dhcp-boot: Specifies the boot file (bootx64.efi) and server IP.

Restart dnsmasq:

```
sudo systemctl restart dnsmasq
```

Step 2: Install and Configure the TFTP Server

The TFTP server delivers boot files to the client machine.

Install TFTP:

```
sudo dnf install tftp-server xinetd
```

Create TFTP root directory:

```
sudo mkdir -p /var/lib/tftpboot
```

Edit TFTP config:

```
sudo nano /etc/xinetd.d/tftp
```

Paste this:

```
service tftp
{
    socket_type     = dgram
    protocol        = udp
```

```
    wait            = yes
    user            = root
    server          = /usr/sbin/in.tftpd
    server_args     = -s /var/lib/tftpboot
    disable         = no
}
```

Restart the service:

```
sudo systemctl restart xinetd
```

Step 3: Set Up the NFS Server

NFS is used to serve the full RHEL installation files to the client.

Install NFS server:

```
sudo dnf install nfs-utils
```

Edit NFS export file:

```
sudo nano /etc/exports
```

Add this line:

```
/var/lib/tftpboot
*(ro,sync,no_root_squash,insecure,no_subtree_check)
```

Export the directory:

```
sudo exportfs -a
```

Step 4: Prepare the RHEL Installation Files

Create a directory and download RHEL ISO:

```
sudo mkdir /var/lib/tftpboot/rhel8

wget https://access.redhat.com/downloads/content/rhel-8.8-x86_64-
dvd.iso -O /var/lib/tftpboot/rhel8/rhel-8.8-x86_64-dvd.iso
```

Mount the ISO temporarily:

```
sudo mkdir /mnt/iso

sudo mount /var/lib/tftpboot/rhel8/rhel-8.8-x86_64-dvd.iso /mnt/iso
-o loop
```

Copy installation files to the TFTP directory:

```
sudo cp -a /mnt/iso/. /var/lib/tftpboot/rhel8/

sudo cp /mnt/iso/images/pxeboot/{vmlinuz,initrd.img}
/var/lib/tftpboot/rhel8/
```

Unmount and optionally delete the ISO:

```
sudo umount /mnt/iso

sudo rm /var/lib/tftpboot/rhel8/rhel-8.8-x86_64-dvd.iso
```

Step 5: Configure GRUB Boot Menu

Create GRUB directory:

```
sudo mkdir /var/lib/tftpboot/grub
```

Create GRUB config file:

```
sudo nano /var/lib/tftpboot/grub/grub.cfg
```

Add this menu entry:

```
menuentry "RHEL 8 - OS" {

    linux /rhel8/vmlinuz ip=dhcp
inst.stage2=nfs:192.168.29.45:/var/lib/tftpboot/rhel8

    echo "Loading Initial Ram Disk..."

    initrd /rhel8/initrd.img

}
```

Explanation:

- ip=dhcp: Client gets IP via DHCP.
- inst.stage2=nfs:...: Tells installer where the RHEL packages are located.

Step 6: Add EFI Boot Files

Download required RPMs:

```
cd /tmp

dnf download shim-x64 grub2-efi-x64 grub2-common
```

Extract and move boot files:

```
# shim (boot loader)

rpm2cpio shim-x64*.rpm | cpio -idmv
./boot/efi/EFI/redhat/shimx64.efi

sudo mv ./boot/efi/EFI/redhat/shimx64.efi
/var/lib/tftpboot/bootx64.efi


# grubx64 (GRUB boot manager)

rpm2cpio grub2-efi-x64*.rpm | cpio -idmv
./boot/efi/EFI/redhat/grubx64.efi

sudo mv ./boot/efi/EFI/redhat/grubx64.efi
/var/lib/tftpboot/grubx64.efi


# Unicode font

rpm2cpio grub2-common*.rpm | cpio -idmv
./usr/share/grub/unicode.pf2

sudo mv ./usr/share/grub/unicode.pf2 /var/lib/tftpboot/unicode.pf2
```

---

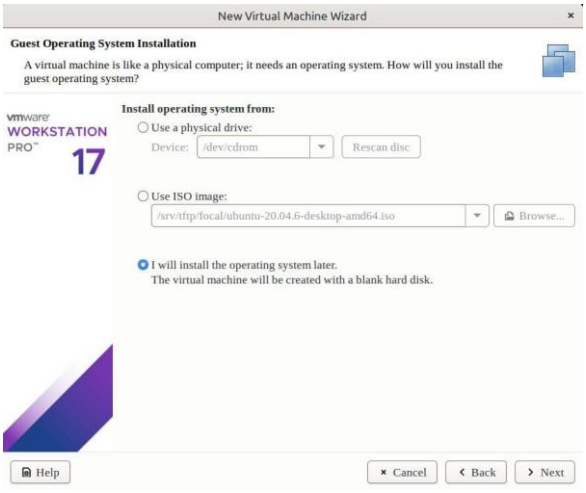
4. Final Step: Boot the Client Machine

1. **Power on the client VM.**
2. **Ensure network boot is enabled in BIOS/UEFI.**
3. The client will:
    - Get IP from dnsmasq (DHCP)
    - Download bootx64.efi via TFTP
    - Load GRUB and show the "RHEL 8 - OS" boot entry
    - Mount RHEL packages via NFS
    - Start installation

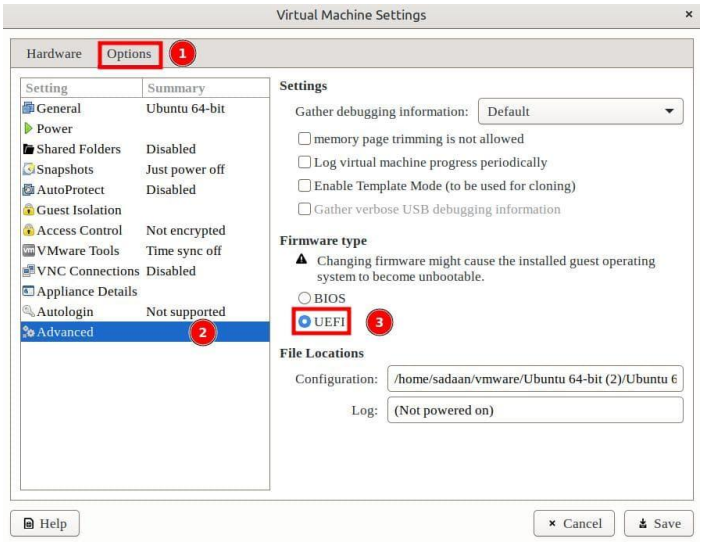
Step:6 Starting Network Installation on Client Machines

To test the above setup, let's create a virtual machine in VMware.

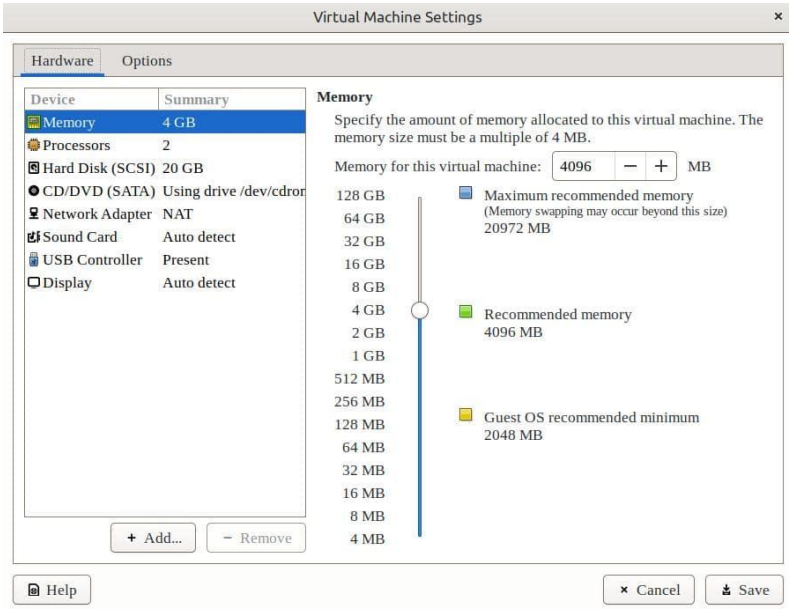
We can set up a new VM with the option to later install the OS:



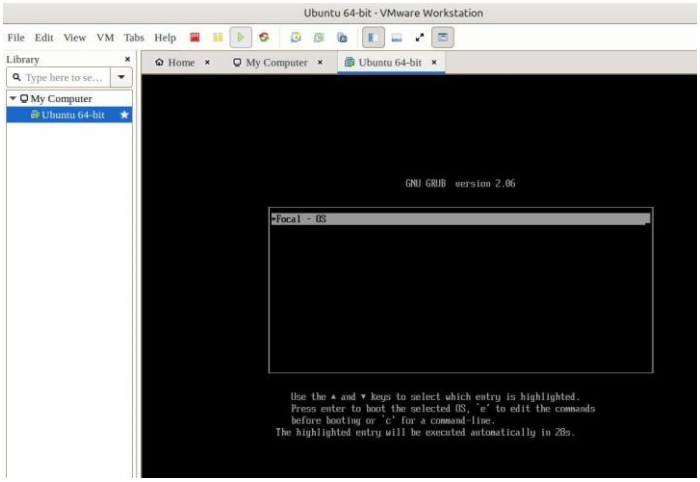
After completing the VM setup, we open the VM settings. Now, **we change the firmware type to UEFI**:



After these changes, we keep the rest of the settings to the default ones:



Now, let's start the client machine:



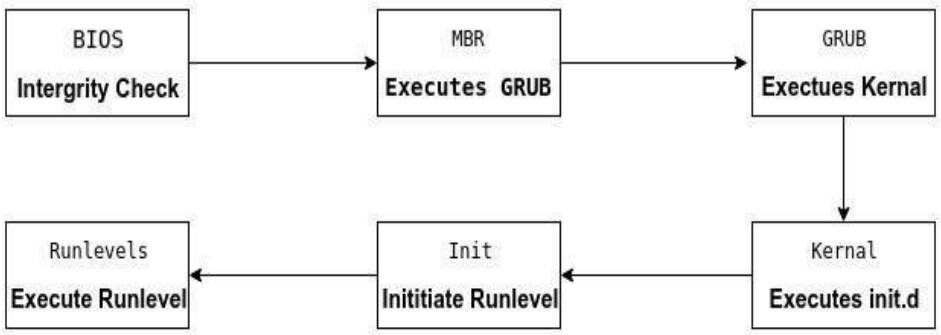
Thus, **the VM boots from the network**. Finally, we can start the Ubuntu installation process.

2.2.3 Conclusion

This guide details the setup of a network installation server for RHEL 8, enabling client machines to boot and install RHEL remotely. This method supports multiple simultaneous installations and eliminates the need for physical media.

2.3 The Linux Booting Process - 6 Steps - BMGKIR

An operating system (OS) is the low-level software that manages resources, controls peripherals, and provides basic services to other software. The **booting process** in Linux involves several tightly integrated steps that bring the system from a powered-off state to a fully functional operating system. Understanding each step is essential for troubleshooting, system administration, and deeper Linux knowledge.



- **Step 1: BIOS/UEFI (Firmware Initialization)**
- **Purpose:**

The system firmware (BIOS for legacy systems or UEFI for modern systems) initializes hardware and locates a bootable device.

- **What Happens:**

- On power-on, the firmware performs a **Power-On Self-Test (POST)** to check basic hardware (CPU, RAM, keyboard, etc.).
- If POST succeeds, it searches for a **bootable disk**.
- **BIOS** checks the **Master Boot Record (MBR)** (first 512 bytes) on disks like /dev/sda.
- **UEFI** looks in the **EFI System Partition (ESP)** for a boot loader using **GPT** partitioning.

🔸 **Key Differences:**

BIOS	UEFI
Uses MBR	Uses GPT
Supports disks ≤ 2TB	Supports disks > 2TB
No secure boot	Supports Secure Boot
Legacy systems	Modern systems

🔹 **Step 2: MBR (Master Boot Record)**

🔸 **Purpose:**

Used by BIOS systems to locate and load the boot loader.

🔸 **What Happens:**

- The **MBR** is the first 512 bytes of the disk and contains:
  - A small program to load the boot loader (e.g., GRUB).
  - A partition table.
- If no valid boot loader is found, the system fails to boot.

Note: UEFI systems **do not use MBR**; they directly use the boot loader from the ESP.

🔹 **Step 3: Boot Loader (e.g., GRUB)**

🔸 **Purpose:**

Loads the Linux **kernel** and **initial RAM disk** into memory and starts the booting process.

🔸 **What Happens:**

- Displays a menu with available kernel versions or operating systems.
- Loads selected **kernel** (vmlinuz) and **initrd/initramfs**.

- Passes necessary **kernel parameters** (e.g., root=/dev/sda1, quiet, ro) to guide system initialization.

🔸 **Popular Boot Loaders:**

Boot Loader	Description
GRUB (most common)	BIOS & UEFI support, menu interface, multiboot
LILO	Older, simple, BIOS-only
SYSLINUX	Lightweight, ideal for USB/CD boots
EFISTUB	UEFI systems, loads kernel directly from firmware

🔹 **Step 4: Kernel Initialization**

🔸 **Purpose:**

Takes over control, initializes system hardware, and starts the first user-space process.

🔸 **What Happens:**

- The kernel uses **initrd/initramfs** as a temporary root filesystem.
- Loads **kernel modules** (drivers) to access storage, network, and other devices.
- Mounts the real root filesystem (/).
- Launches **/sbin/init** (or its alternatives like systemd) — always process **PID 1**.

🔸 **Example Kernel Parameters:**

Parameter	Function
ro	Mount root filesystem read-only initially
quiet	Suppresses boot messages
nomodeset	Disables advanced graphics modes (troubleshooting)

🔹 **Step 5: Init System (e.g., systemd, SysVinit)**

🔸 **Purpose:**

Manages system initialization, services, and sets the system's operational state (runlevel or target).

🔸 **What Happens:**

- In traditional **SysVinit**, /etc/inittab defines runlevels (e.g., 3 = multi-user, 5 = graphical).
- In modern **systemd**, runlevels are replaced with **targets** (e.g., multi-user.target).
- Starts background services (e.g., networking, login manager) based on configuration.

🔹 **Step 6: Runlevel Programs / Services**

🔸 **Purpose:**

Start or stop services based on the system's runlevel or target.

🔸 **What Happens:**

- Scripts located in directories like /etc/rc5.d/ (runlevel 5) or systemd units.
- Scripts prefixed with:
  - S = Start a service.
  - K = Kill (stop) a service.
- Services launched include GUI login, networking, firewall, etc.

2.3.2 Integration of Key Components

🔸 **1. Kernel Initialization**

- Central to **Step 4**.
- Performs:
  - CPU/memory check
  - Device discovery
  - Root filesystem mount
  - Launch of /sbin/init

🔸 **2. Boot Loaders**

- **Step 3** responsibility.
- Loads kernel & initrd/initramfs, passes parameters.
- Offers recovery tools (e.g., GRUB rescue shell).

🔸 **3. Kernel Modules**

- Loaded during **Step 4**, extended in **Step 5**.

- Drivers for disks, network, USB, etc.
- Managed with tools like modprobe.

◆ **4. Configuration**

- Modify /boot/grub/grub.conf or /etc/grub.d/.
- Customize:
  - Default kernel
  - Timeout
  - Kernel parameters
- Example GRUB entry:

```
menuentry 'RHEL 8' {
    linux /vmlinuz-4.18.0 ro root=/dev/sda1 quiet
    initrd /initramfs-4.18.0.img
}
```

- Rebuild initramfs (e.g., after adding drivers):

```
dracut -f /boot/initramfs-4.18.0.img 4.18.0
```

2.3.3 How Components Work Together

Component	Role	Step
BIOS/UEFI	Initializes hardware	Step 1
MBR	Loads boot loader (BIOS only)	Step 2
Boot Loader (GRUB)	Loads kernel + initramfs	Step 3
Kernel	Initializes core systems	Step 4
Init	Starts services and user-space processes	Step 5
Runlevel Programs	Final system setup	Step 6

2.4 User Management in Linux

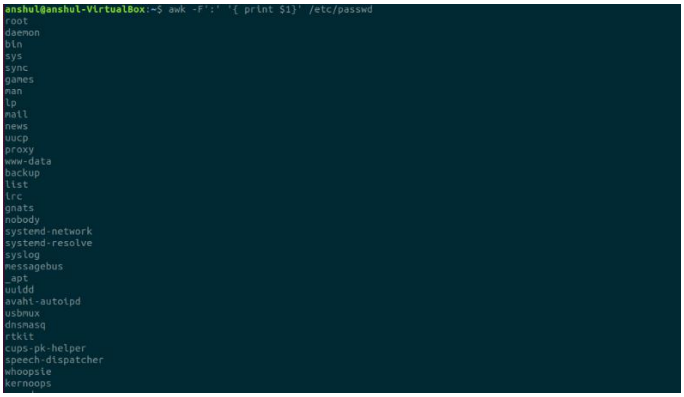
A user is an entity, in a Linux operating system, that can manipulate files and perform several other operations. Each user is assigned an ID that is unique for each user in the operating system. In this post, we will learn about users and commands which are used to get information about the users. After installation of the operating system, the **ID 0 is assigned to the root**

user and the IDs 1 to 999 (both inclusive) are assigned to the system users and hence the ids for local user begins from 1000 onwards.

In a single directory, we can create 60,000 users. Now we will discuss the important commands to manage users in Linux.

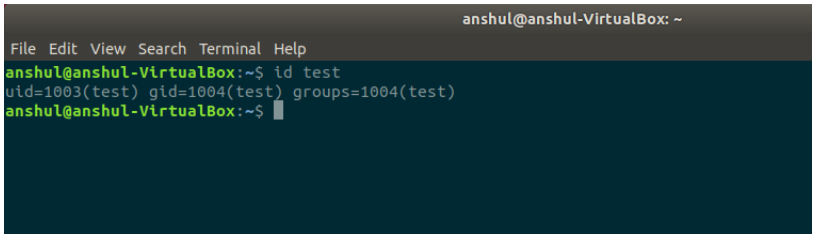
1. To **list out all the users in Linux**, use the awk command with -F option. Here, we are accessing a file and printing only first column with the help of print $1 and awk.

```
awk -F':' '{print $1}' /etc/passwd
```



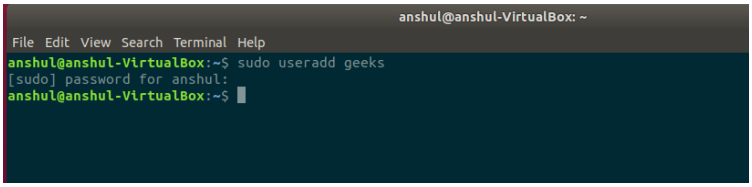
2. Using id command, you can get the ***ID of any username***. Every user has an id assigned to it and the user is identified with the help of this id. By default, this id is also the group id of the user.

```
id username

Example: id test
```



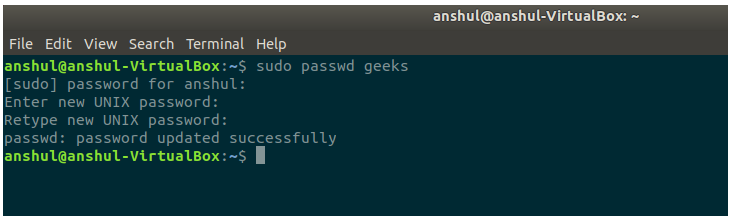
3. The command to add a user. *useradd command* adds a new user to the directory. The user is given the ID automatically depending on which category it falls in. The username of the user will be as provided by us in the command.

```
sudo useradd username

Example: sudo useradd geeks
```



4. Using passwd command to assign a password to a user. After using this command, we have to enter the new password for the user and then the password gets updated to the new password.

```
passwd username

Example: passwd geeks
```

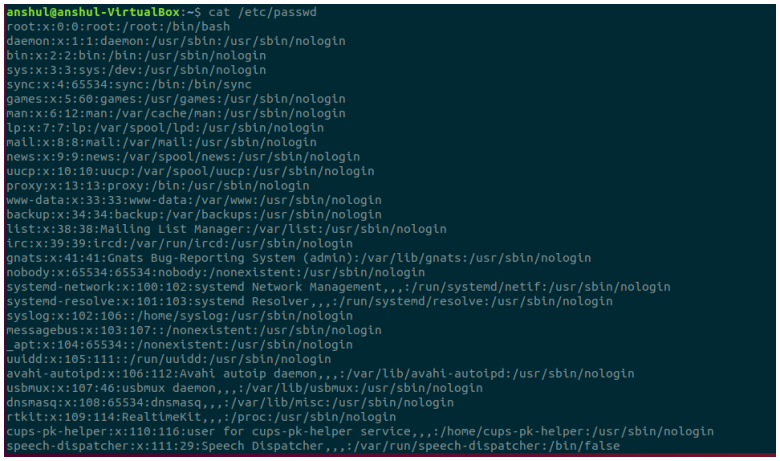


5. ***Accessing a user configuration file***.

```
cat /etc/passwd
```

This command prints the data of the configuration file. This file contains information about the user in the format.

username : x : user id : user group id : : /home/username : /bin/bash



Now we will go through the commands to modify information.

6. The command to **change the user ID for a user**.

```
usermod  -u new_id username
```

This command can change the user ID of a user. The user with the given username will be assigned with the new ID given in the command and the old ID will be removed.

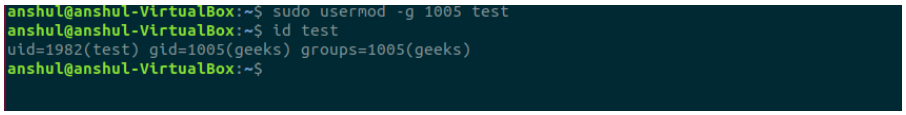
```
Example: sudo usermod -u 1982 test
```

7. Command to **Modify the group ID of a user**.

```
usermod -g  new_group_id username
```

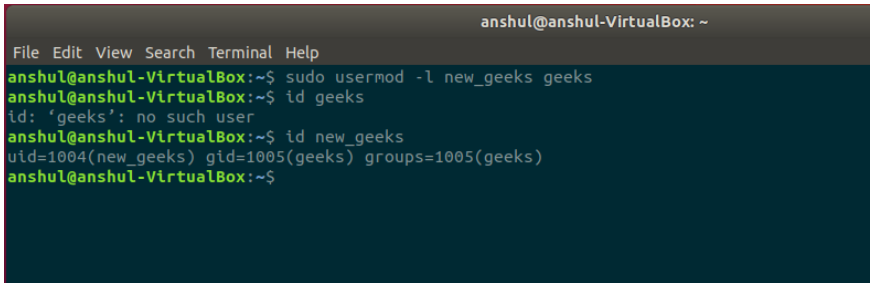
This command can change the group ID of a user and hence it can even be used to move a user to an already existing group. It will change the group ID of the user whose username is given and sets the group ID as the given new_group_id.

```
Example: sudo usermod -g 1005 test
```



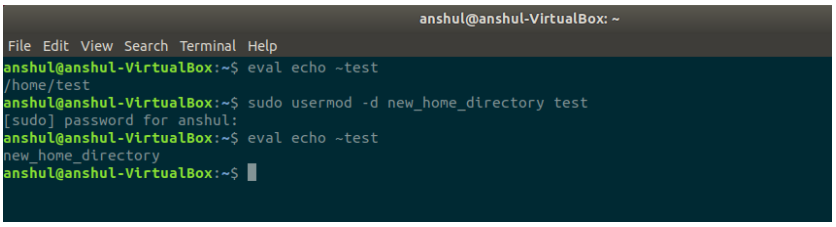
8. You can **change the user login name** using *usermod* command. The below command is used to change the login name of the user. The old login name of the user is changed to the new login name provided.

```
sudo usermod -l new_login_name old_login_name

Example: sudo usermod -c John_Wick John_Doe
```



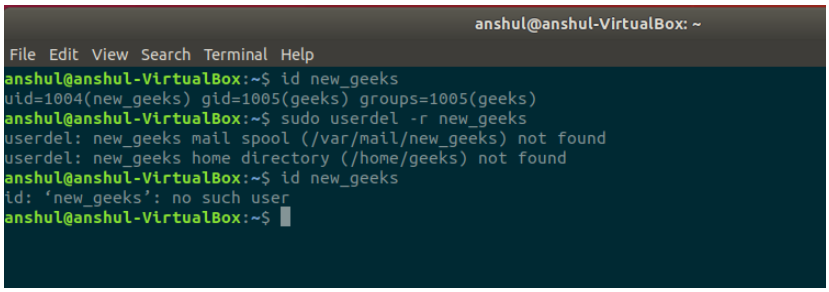
9. The **command to change the home directory**. The below command changes the home directory of the user whose username is given and sets the new home directory as the directory whose path is provided.

```
usermod -d new_home_directory_path username

Example: usermod -d new_home_directory test
```



10. You can also **delete a user name**. The below command deletes the user whose username is provided. Make sure that the user is not part of a group. If the user is part of a group then it will not be deleted directly, hence we will have to first remove him from the group and then we can delete him.

```
sudo userdel -r username

Example: sudo userdel -r new_geeks
```



2.5 Group Management in Linux

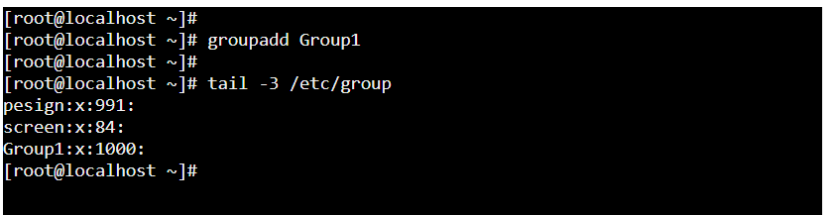
There are 2 categories of groups in the Linux operating system i.e. **Primary** and **Secondary** groups. The *Primary Group* is a group that is automatically generated while creating a user with a unique user ID simultaneously a group with an ID the same as the user ID is created and the user gets added to the group and becomes the first and only member of the group. This group is called the primary group. A *secondary group* is a group that can be created separately with the help of commands and we can then add users to it by changing the group ID of users.

2.5.1 How to Create/Add Users in Linux

1. Creating a Secondary Group

The below command created a group with the name provided. The group while creating gets a group ID and we can get to know everything about the group as its name, ID, and the users present in it in the file "/etc/group".

```
groupadd group_name

Example: groupadd Group1
```



2. Setting the Password for the Group

Below command is used to set the password of the group. After executing the command, we have to enter the new password which we want to assign to the group. The password has to be given twice for confirmation purposes.

```
gpasswd group_name

Example:

gpasswd Group1
```

```
[root@localhost ~]#
[root@localhost ~]# gpasswd Group1
Changing the password for group Group1
New Password:
Re-enter new password:
[root@localhost ~]#
```

3. Command to Display the Group Password File:

To access information about groups and their passwords, you can view the password file, /etc/gshadow. However, keep in mind that this file is not intended for regular viewing. To gather more comprehensive information about groups.

```
cat /etc/gshadow
```

```
[root@localhost ~]#
[root@localhost ~]# cat /etc/gshadow
root:::
bin:::
daemon:::
sys:::
adm:::
tty:::
disk:::
lp:::
mem:::
```

4. Adding a User to an Existing Group

To add a user to an existing group, you can utilize the usermod command. By specifying the group name, you can add a user to the desired group. However, note that when a user is added to a new group, they are automatically removed from their previous groups.

```
usermod -G group_name  username

usermod -G group1 John_Doe
```

```
[root@localhost ~]#
[root@localhost ~]# usermod -G Group1 John_Wick
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# tail -3 /etc/group
Group1:x:1000:John_Doe,John_Wick
John_Doe:x:1001:
John_Wick:x:1002:
[root@localhost ~]#
```

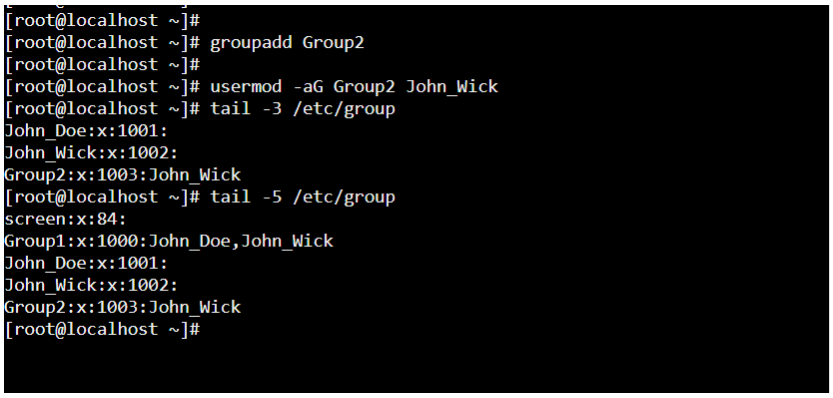
Note:

If we add a user to a group then it automatically gets removed from the previous groups, we can prevent this by the command given below.

5. Command to Add User to Group Without Removing from Existing Groups

This command is used to add a user to a new group while preventing him from getting removed from his existing groups.

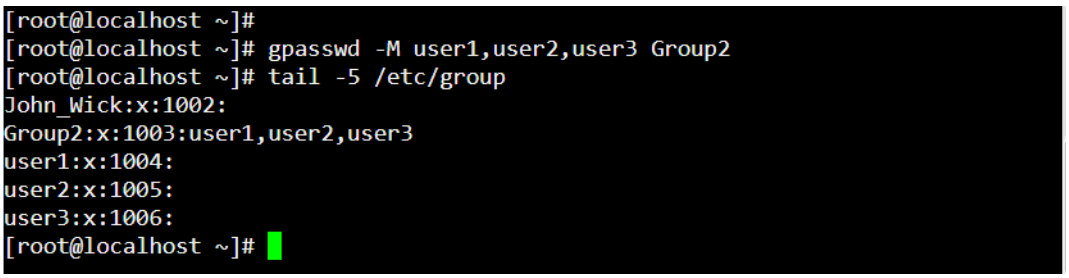
```
usermod -aG *group_name  *username

Example:

usermod -aG group1 John_Doe
```



6. Command to Add Multiple Users to a Group at once

To add multiple users to a group simultaneously, you can utilize the gpasswd command with the -M option. This command allows you to specify a list of usernames separated by commas**.**

```
gpasswd -M *username1, *username2, *username3 ...., *usernamen
                        *group_name
                          Example:

          gpasswd -M Person1, Person2, Person3 Group1
```



7. Deleting a User from a Group

Below command is used to delete a user from a group. The user is then removed from the group though it is still a valid user in the system but it is no longer a part of the group. The user remains part of the groups which it was in and if it was part of no other group then it will be part of its primary group.

```
gpasswd -d *username1  *group_name

Example:

gpasswd -d Person1  Group1
```

```
[root@localhost ~]# gpasswd -d user1 Group2
Removing user user1 from group Group2
[root@localhost ~]#
```

8. Command to Delete a Group

To delete a group from the system, use the groupdel command. This action removes the group while retaining the users who were members of the group. They will revert to their primary groups if they are not part of any other groups.

```
groupdel *group_name

Example:

groupdel  Group1
```

```
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# groupdel Group2
[root@localhost ~]# tail -5 /etc/group
John_Doe:x:1001:
John_Wick:x:1002:
user1:x:1004:
user2:x:1005:
user3:x:1006:
[root@localhost ~]#
```

2.6 Password Policy for Linux Devices

To enforce a password policy on Linux systems based on RHEL (Red Hat Enterprise Linux), you should configure the PAM (Pluggable Authentication Modules) system and adjust system-wide password aging rules.

Here's how to do it:

1. Install Required Packages

Ensure the `pam_pwquality` or `libpwquality` package is installed:

```
sudo dnf install libpwquality
```

---

2. Configure Password Complexity

Edit `/etc/security/pwquality.conf`:

Set or adjust the following values:

```
minlen = 12              # Minimum total length
dcredit = -1             # At least one digit
ucredit = -1             # At least one uppercase letter
lcredit = -1             # At least one lowercase letter
ocredit = -1             # At least one special character
```

```
maxrepeat = 3             # Max repeated characters
minclass = 4              # Require at least 4-character
classes
```

These settings enforce stronger passwords.

---

3. Enforce Policy in PAM

Edit the following file:

```
sudo nano /etc/pam.d/system-auth
```

Look for the password section and ensure this line exists (or add it):

```
password    requisite     pam_pwquality.so retry=3
```

This ensures the password quality module is called when setting passwords.

---

4. Configure Password Expiration Policy

Edit `/etc/login.defs`:

```
sudo nano /etc/login.defs
```

Set these values:

```
PASS_MAX_DAYS   90     # Max days a password is valid
PASS_MIN_DAYS   7      # Min days between changes
PASS_WARN_AGE   14     # Days before expiration to warn
```

Then, apply to existing users using chage:

```
sudo chage --maxdays 90 --mindays 7 --warndays 14 <username>
```

---

5. Lock Accounts After Failed Attempts

To prevent brute-force attacks, configure `pam_faillock`:

```
sudo nano /etc/pam.d/system-auth
```

Add or update:

```
auth required pam_faillock.so preauth silent audit deny=5
unlock_time=900
auth [default=die] pam_faillock.so authfail audit deny=5
unlock_time=900
```

This locks the user for 15 minutes after 5 failed attempts.

6. Optional: Force Password Change on First Login

Use:

```
sudo chage -d 0 <username>
```

Test the Policy

Try creating a new user and changing their password:

```
sudo adduser testuser
sudo passwd testuser
```

Try various password combinations to ensure enforcement.

2.7 File Permissions in Linux

In Linux, file permissions are rules that determine who can access, modify, or execute files and directories. They are foundational to Linux security, ensuring that only authorized users or processes can interact with your data. Here's a breakdown:

1. The Three Basic Permissions

Every file or directory has three types of permissions:

- **Read (r):** View the file's contents or list a directory's files.
- **Write (w):** Modify a file or add/delete files in a directory.
- **Execute (x):** Run a file as a program/script or enter a directory.

Letters	Definition
'r'	"read" the file's contents.
'w'	"write", or modify, the file's contents.
'x'	"execute" the file. This permission is given only if the file is a program.

2. Ownership and Permission Groups

Permissions are assigned to three categories of users:

- **User (Owner):** The person who created the file.
- **Group:** Users belonging to a shared group (e.g., "developers" or "admins").
- **Others:** Everyone else on the system.

 # File Permission: Operation Chart

Operators	Definition
`+`	Add permissions
`-`	Remove permissions
`=`	Set the permissions to the specified values

Note: *All these permissions are being granted at three different levels based on their group.*

 # What are Permission Groups in Linux

First, you must think of those nine characters as three sets of three characters (see the box at the bottom). Each of the three "rwx" characters refers to a different operation you can perform on the file.

1. **Owners:** These permissions apply exclusively to the individuals who own the files or directories.
2. **Groups:** Permissions can be assigned to a specific group of users, impacting only those within that particular group.
3. **All Users:** These permissions apply universally to all users on the system, presenting the highest security risk. Assigning permissions to all users should be done cautiously to prevent potential security vulnerabilities.

--- --- ---
rwx rwx rwx
user group other

 # User, Group, and others Option in Linux File Permission

Reference	Class	Description
`u`	**user**	The user permissions apply only to the owner of the file or directory, they will not impact the actions of other users.
`g`	**group**	The group permissions apply only to the group that has been assigned to the file or directory, they will not affect the actions of other users.
`o`	**others**	The other permissions apply to all other users on the system, this is the permission group that you want to watch the most.
`a`	**All three**	All three (owner, groups, others)

 # How to Check the Permission of Files in Linux?

1. The "Trusty Command"

Here's the command to execute it within the terminal. Let's show you with an example:

Input:

We're taking 'NarX' as a default file name:

ls -l NarX.txt

Output:

-rw-r--r-- 1 user group 46 Apr 14 16:37 NarX.txt

The above command represents these following information:

1. The first character = **'-'**, which means it's a file **'d'**, which means it's a directory.
2. The next nine characters = (rw-r–r–) show the security
3. The next column shows the owner of the file.
4. The next column shows the group owner of the file. (which has special access to these files)
5. The next column shows the size of the file in bytes.
6. The next column shows the date and time the file was last modified.

2. The 'namei' Command

The 'namei' command is used to check the file path through layer of folder's path. Here's the command to execute it within Terminal:

Here, we've taken 'path' as " root@anonymous-VirtualBox:~# " and file name as "hoops"

namei -l /path/to/your/file

3. The 'stat' Command

Unlike 'ls -l' command, the "stat" command is used to pin point the file location. Here's how you can do it:

We're taking file name as "hoops"

stat hoops

Output:

File: example.txt

Size: 2210 Blocks: 8 IO Block: 4096 regular file

Device: 802h/2050d Inode: 1288496 Links: 1

Access: 2024-11-18 10:50:56.000000000 +0000

Modify: 2024-11-18 10:50:56.000000000 +0000

Change: 2024-11-18 10:50:56.000000000 +0000

Birth: -

 # How to Change Permissions in Linux?

The command you use to change the security permissions on files is called "**chmod**", which stands for "change mode" because the nine security characters are collectively called the security "mode" of the file. You can modify permissions using **symbolic notation** or **octal notation**.

1. Symbolic Notation

Symbolic notation allows you to **add, remove,** or **set permissions** for specific users. Let's understand this using different example below:

Example 1: To Change File Permission in Linux

If you want to give "execute" permission to the world ("other") for file "xyz.txt", you will start by typing.

chmod o

Now you would type a '+' to say that you are "adding" permission.

chmod o+

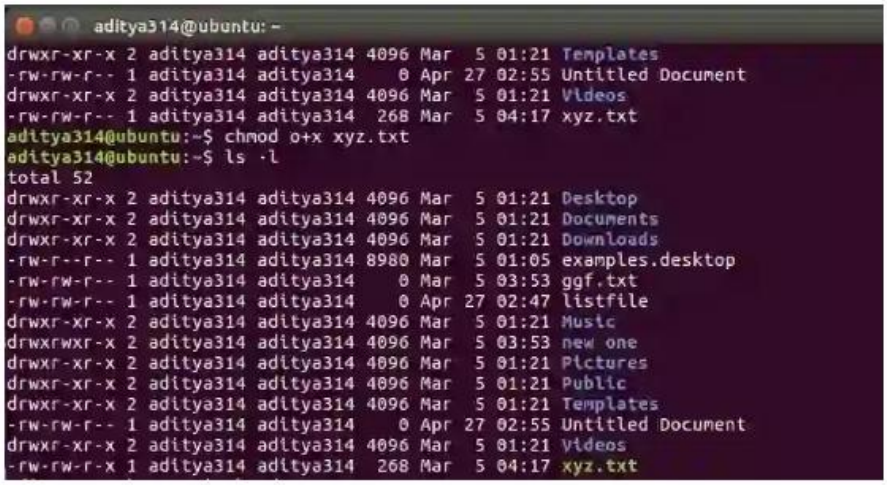
Then you would type an 'x' to say that you are adding "execute" permission.

chmod o+x

Finally, specify which file you are changing.

chmod o+x xyz.txt

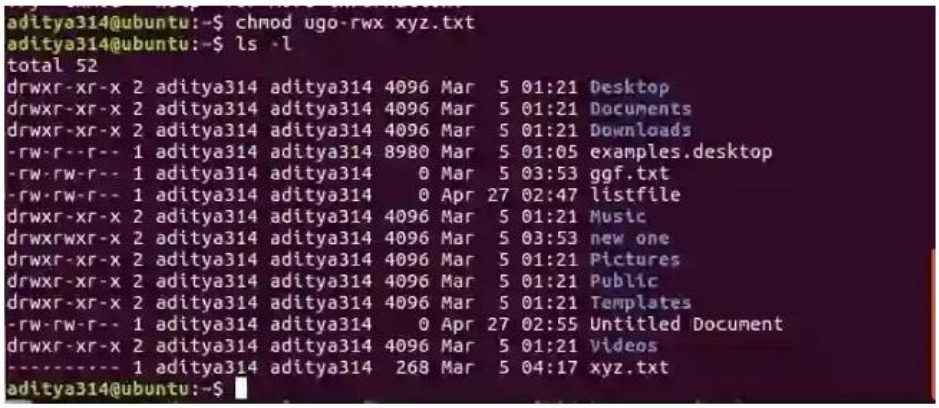
You can see the change in the picture below.



You can also change multiple permissions at once. For example, if you want to take all permissions away from everyone, you would type.

chmod ugo-rwx xyz.txt

The code above revokes all the read(r), write(w), and execute(x) permission from all user(u), group(g), and others(o) for the file xyz.txt which results in this.



chmod ugo

Example 2:

The code adds read(r) and write(w) permission to both user(u) and group(g) and revoke execute(x) permission from others(o) for the file abc.mp4.

chmod ug+rw,o-x abc.mp4

Something like this:

chmod ug=rx,o+r abc.c

Assigns read(r) and execute(x) permission to both user(u) and group(g) and add read permission to others for the file abc.c.

There can be numerous combinations of file permissions you can invoke revoke and assign. You can try some on your Linux system.

2. Octal Notations Permissions in Linux

The octal notation is used to represent file permission in Linux by using three user group by denoting 3 digits i.e.

- user
- group
- other users

Here's how to permissions are mapped:

- **Read (r) = 4**
- **Write (w) = 2**
- **Execute (x) = 1**

Permissions for **owner**, **group**, and **others** are represented by a three-digit octal value. The sum of permissions for each group gives the corresponding number.

You can also use octal notations like this.

Octal	Binary	File Mode
0	000	---
1	001	--x
2	010	-w-
3	011	-wx
4	100	r--
5	101	r-x
6	110	rw-
7	111	rwx

octal notations

Using the octal notations table instead of 'r', 'w', and 'x'. Each digit octal notation can be used for either of the group 'u', 'g', or'o'.

So, the following work is the same.

chmod ugo+rwx [file_name]

chmod 777 [file_name]

Both of them provide full read write and execute permission (code=7) to all the group.

The same is the case with this.

chmod u=r,g=wx,o=rx [file_name]

chmod 435 [file_name]

Both the codes give read (code=4) user permission, write and execute (code=3) for the group and read and execute (code=5) for others.

And even this…

chmod 775 [file_name]

chmod ug+rwx,o=rx [file_name]

Both the commands give all permissions (code=7) to the user and group, read and execute (code=5) for others.

Security Permissions in Linux

The combination for the permissions are r,w,x, and -. Let's understand this briefly in elaborative way:

For example: "rw- r-x r–"

- **"rw-"**: the first three characters `rw-`. This means that the owner of the file can "read" it (look at its contents) and "write" it (modify its contents). We cannot execute it because it is not a program but a text file.
- **"r-x"**: the second set of three characters "r-x". This means that the members of the group can only read and execute the files.
- **"r–"**: The final three characters "r–" show the permissions allowed to other users who have a UserID on this Linux system. This means anyone in our Linux world can read but cannot modify or execute the files' contents.

2.8 Access Control Lists (ACL) in Linux – Implementation Process is in Chapter 5

Access Control Lists (ACLs) provide an extended, more flexible permission mechanism for Linux file systems, allowing administrators to set specific permissions for individual users or groups. This guide will walk you through the key ACL commands, options, and use cases.

 # Why Use ACLs?

Think of a scenario in which a particular user is not a member of a group created by you but still, you want to give some read or write access, how can you do it without making the user a member of the group, here comes in picture Access Control Lists, ACL helps us to do this trick. Basically, ACLs are used to make a flexible permission mechanism in Linux. From Linux man pages, ACLs are used to define more fine-grained discretionary access rights for files and directories.

Syntax

The two main commands for managing ACLs are:

setfacl: Used to set ACL entries.

getfacl: Used to retrieve and display ACL entries.

For example:

getfacl test/declarations.h

Output:

 # file: test/declarations.h

 # owner: mandeep

 # group: mandeep

user::rw-

group::rw-

other::r--

 # List of commands for setting up ACL

1. To add permission for user

setfacl -m "u:user:permissions" /path/to/file

2. To add permissions for a group

setfacl -m "g:group:permissions" /path/to/file

3. To allow all files or directories to inherit ACL entries from the directory it is within

setfacl -dm "entry" /path/to/dir

4. To remove a specific entry

setfacl -x "entry" /path/to/file

5. To remove all entries

setfacl -b path/to/file

For Example:

setfacl -m u:mandeep:rwx test/declarations.h

Modifying ACL using setfacl:

To add permissions for a user (user is either the user name or ID):

 # setfacl -m "u:user:permissions"

To add permissions for a group (group is either the group name or ID):

 # setfacl -m "g:group:permissions"

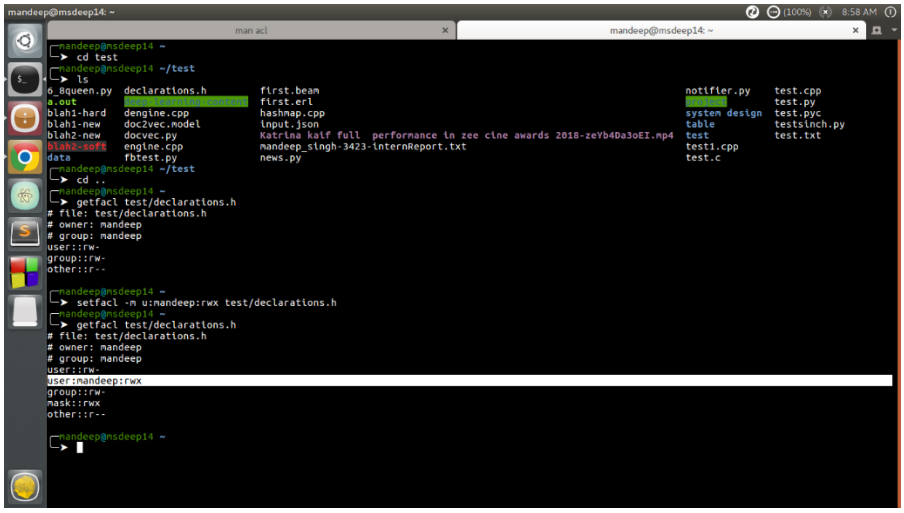
To allow all files or directories to inherit ACL entries from the directory it is within:

 # setfacl -dm "entry"

For Example:

setfacl -m u:mandeep:r-x test/declarations.h

See below image for output:



setfacl and getfacl

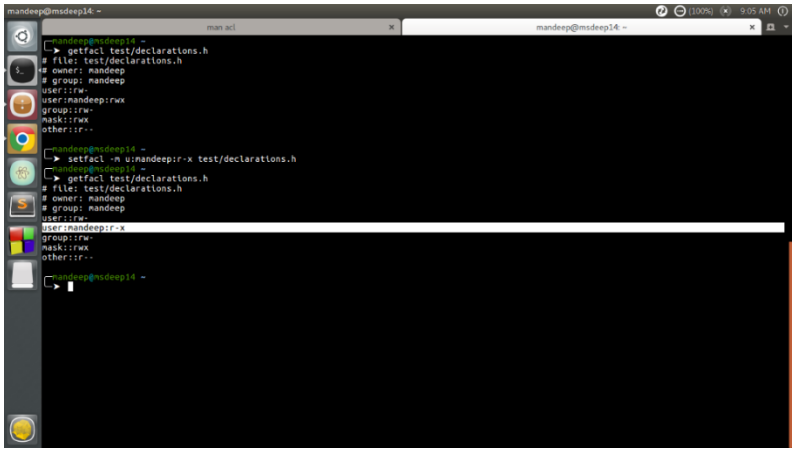
View ACL:

To show permissions:

 # getfacl filename

Observe the difference between output of **getfacl** command before and after setting up ACL permissions using **setfacl** command. There is one extra line added for user mandeep which is highlighted in image above.

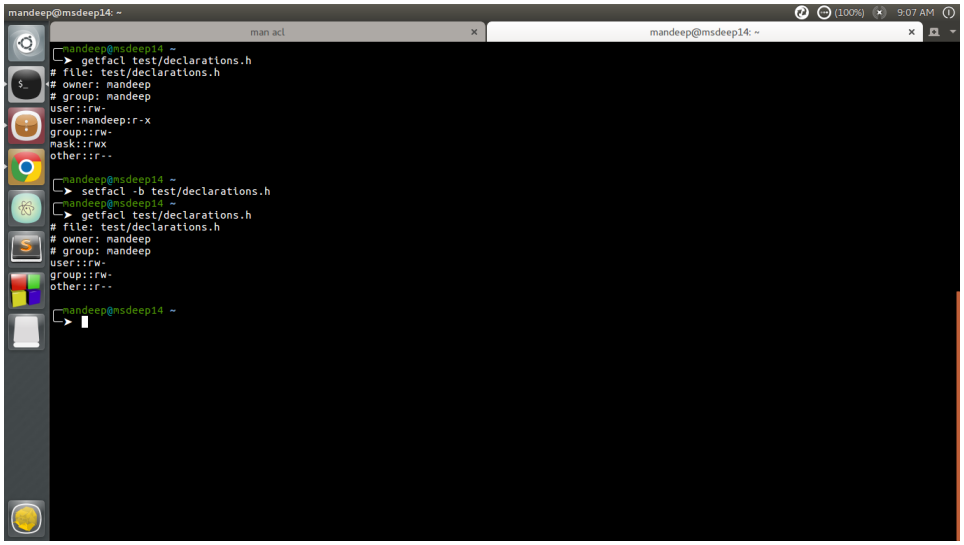
Output:



change permissions

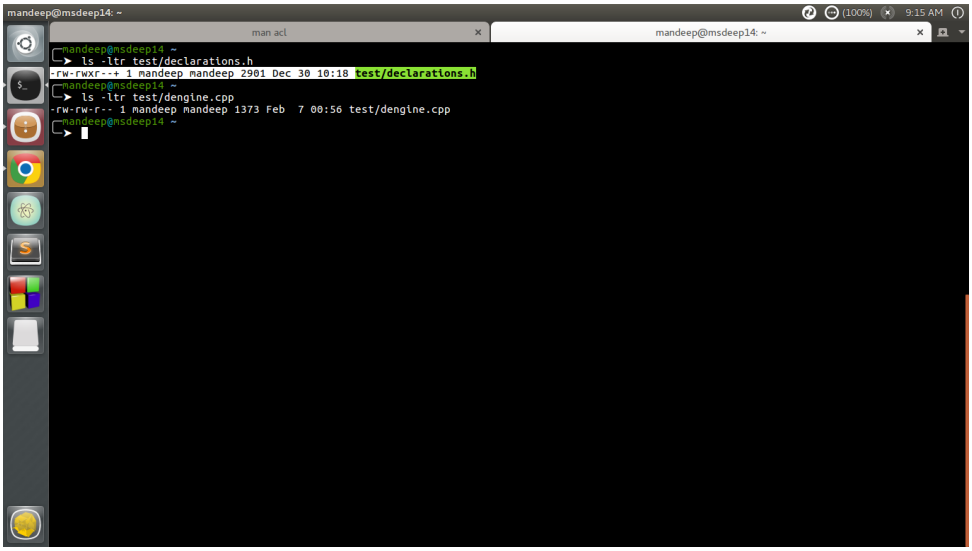
The above command change permissions from **rwx** to **r-x Remove ACL:**

If you want to remove the set ACL permissions, use setfacl command with -b option. For example:



remove set permissions

If you compare output of getfacl command before and after using setfacl command with **-b option**, you can observe that there is no particular entry for user mandeep in later output. You can also check if there are any extra permissions set through ACL using ls command.



check set acl with ls

Observe the first command output in image, there is extra "+" sign after the permissions like **-rw-rwxr–+**, this indicates there are extra ACL permissions set which you can check by getfacl command.

Using Default ACL

The default ACL is a specific type of permission assigned to a directory, that doesn't change the permissions of the directory itself, but makes so that specified ACLs are set by default on all the files created inside of it.

Let's demonstrate it: first we are going to create a directory and assign default ACL to it by using the -d option:

$ mkdir test && setfacl -d -m u:dummy:rw test

Key Options Commonly Used with ACL Commands

Option	Description
-m	Modify or add an ACL entry for a user or group
-x	Remove an ACL entry
-b	Remove all ACL entries
-d	Set default ACL for a directory (inheritance for files)

Unit 3: Disk Quotas, Storage Management, and Network Configuration

3.1 User Quotas in Linux

The control of user quotas in Linux is a very effective facility that provides system administrators with the ability to limit disks usage of particular users or groups of users in order to protect against disk overloading. When quotas are set, it will be easier to stop any user or even a group from using most of the disks hence causing a DOS. Thirdly, quotas can serve a social purpose as it instills positive disk usage behavior among the users and prevents abuse of the available system resources.

3.1.1 Understanding User Quotas

User quotas in Linux are typically defined by two main limits: the soft or flexible deadline and the hard or rigid deadline. Soft limit is the maximum amount of disk usage that is allowed in a customer's account before generating an email warning to the customer. After crossing the soft limit, the user can continue writing on disks but new files or new data cannot be created after the limit just crossed unless the user frees the space and comes below soft limit value.

3.1.2 Enabling Quota Support in Linux

What Is a Quota?

A **quota** limits how much disk space or how many files a user or group can use.

- **Hard limit**: The strict maximum; no data can be written beyond this point.
- **Soft limit**: A flexible threshold; users can exceed it temporarily during a **grace period**.

Step 1: Check Filesystem Support

Most modern RHEL filesystems like **ext4** and **XFS** support quotas. You need to check whether quotas are already enabled on the target filesystem.

For ext4 Filesystem

Command:

```
sudo dumpe2fs -h /dev/sda1 | grep -i quota
```

Explanation:

- dumpe2fs -h: Displays the superblock info of ext2/3/4.
- grep -i quota: Filters for quota-related info.
- Replace /dev/sda1 with your actual device name.

Sample Output:

Filesystem quota usage on /dev/sda1: enabled

If not enabled, you'll need to remount with quota options or enable during filesystem creation.

For XFS Filesystem

Command:

```
sudo xfs_quota -x -c 'stat -' /mnt
```

Explanation:

- xfs_quota: Manages quotas on XFS.
- -x: Expert mode.
- -c 'stat -': Shows current quota status.
- /mnt: The mount point of the XFS filesystem.

Sample Output:

```
Path                    Kern         Enabled
/mnt                    -------      Quota enabled
```

Enable if Disabled:

```
sudo xfs_quota -x -c 'enable' /mnt
```

This command runs silently if successful.

3.1.3 Setting Up Quotas for Users and Groups

Once quotas are enabled, you can assign limits using tools like edquota or setquota.

Method 1: Using edquota (for ext2/3/4)

Command:

```
sudo edquota -u username
```

Explanation:

- Opens a text editor where you can set:
    - **blocks** (disk space)
    - **inodes** (number of files)
    - soft and hard limits

Sample Editor Output:

```
Disk quotas for user username (uid 1000):
Filesystem       blocks   soft   hard   inodes   soft   hard
/dev/sda1          100      500    600     10       0      0
```

Method 2: Using setquota (non-interactive)

Command:

```
sudo setquota -u username 500M 600M 0 0 /home
```

Explanation:

- -u username: Set quota for this user.
- 500M 600M: Soft and hard disk limits.
- 0 0: No limits on inodes.
- /home: Target filesystem.

No output if the command is successful.

3.1.4 Monitoring and Managing Quotas

To check current usage:

Command:

quota -u username

Sample Output:

```
Disk quotas for user username (uid 1000):
Filesystem  blocks   quota   limit   grace   files   quota
limit   grace
/dev/sda1   100      500M    600M            10      0
0
```

Automated Monitoring

- Set up **cron jobs** to generate periodic quota reports.
- You can email users when they approach their limits.

3.1.5 How Quota Violations Are Handled

- **Soft Limit**:
  - A **grace period** is given (e.g., 7 days).
  - Users can exceed the limit temporarily.
- **Hard Limit**:
  - **Strictly enforced**.
  - Any write beyond the hard limit is instantly denied.

3.1.6 Best Practices for Quota Management in RHEL

- Set **realistic limits** that align with user roles and needs.
- Monitor usage frequently and adjust as required.
- **Educate users** on quota policies and how to stay within limits.
- Automate reporting to keep users and admins informed.
- Use a **dedicated partition** for user data (like /home) for easier quota setup.
- Perform periodic **cleanup** of obsolete data to free up space.

Q. How does disk quota management contribute to data security and system performance?

Disk quota management plays a critical role in both **data security** and **system performance** by enforcing limits on how much disk space users or groups can consume. Here's how it contributes to each:

---

1. System Performance

- **Prevents Disk Exhaustion**: By setting quotas, administrators ensure that no single user or process can consume all available disk space, which could crash services or the entire system.
- **Stabilizes Applications**: Applications that rely on disk I/O (like databases, mail servers, etc.) perform consistently when there's guaranteed free space.
- **Improves Predictability**: System resources are allocated fairly, so performance remains stable even with many users on the system.

---

2. Data Security

- **Limits Abuse**: Quotas prevent users (especially on multi-user systems) from intentionally or accidentally storing excessive data, which could lead to denial-of-service scenarios.
- **Helps Detect Unusual Activity**: Sudden quota usage spikes may indicate malicious behavior like unauthorized backups, log flooding, or malware activity.
- **Supports Auditing and Compliance**: Disk usage limits help maintain logs and user data within expected bounds, making auditing simpler and reducing the risk of non-compliance.

Disk quota management isn't just about storage—it's about **control**, **fair use**, and **preventing problems before they occur**. It ensures that all users have access to necessary resources without stepping on each other's toes, keeping the system secure and responsive.

3.2 Configuring RAID Arrays on Linux for Data Redundancy

RAID (Redundant Array of Independent Disks) is a powerful technology that enhances data redundancy, increases storage capacity, and optimizes performance in Linux environments. Configuring RAID arrays on Linux can be a daunting task for beginners, but it is a crucial step for ensuring high availability and data protection in any production or personal server setup. This guide will take you through the process, from understanding RAID levels to setting up and managing RAID arrays on Linux.

3.2.1 Introduction

Data redundancy and fault tolerance are essential elements of modern computing systems. RAID provides a solution by combining multiple physical disks into a single logical unit, allowing for redundancy, improved performance, or both, depending on the RAID level chosen. For administrators and power users who need to manage large amounts of critical data, configuring RAID arrays on Linux is a key skill.

We will explore different types of RAID configurations, explain their benefits, and provide a step-by-step guide on how to configure RAID arrays on Linux using the **mdadm** tool. Additionally, we will cover RAID maintenance, performance optimization, and troubleshooting.

3.2.2 What is RAID?

RAID is a data storage virtualization technology that combines multiple physical disks into a single unit for the purposes of data redundancy or improved performance. There are several RAID levels, each offering different balances between performance, redundancy, and storage capacity.

3.2.3 Understanding RAID Levels

RAID comes in several levels, each catering to different needs. The most common RAID configurations are:

1. **RAID 0 (Striping)**

RAID 0 distributes data across multiple disks without redundancy, significantly improving read/write speeds. However, it offers no protection against disk failures—if one disk fails, all data is lost. RAID 0 is ideal when performance is critical and data redundancy is not a concern.

2. **RAID 1 (Mirroring)**

RAID 1 creates an exact copy (or mirror) of data on two or more disks. This provides excellent redundancy since data is preserved as long as at least one disk remains operational. However, RAID 1 reduces usable storage by half, as every byte is duplicated.

3. **RAID 5 (Striping with Parity)**

RAID 5 offers a balance between performance, redundancy, and storage efficiency. Data and parity information are striped across at least three disks, allowing the array to recover from a single disk failure. While RAID 5 offers redundancy, it has slower write speeds due to the parity calculations.

4. **RAID 6 (Striping with Double Parity)**

RAID 6 is similar to RAID 5 but with added redundancy, allowing the array to survive two simultaneous disk failures. RAID 6 requires at least four disks and is a good choice for systems where uptime is critical.

5. **RAID 10 (Mirroring and Striping)**

RAID 10, or RAID 1+0, combines the benefits of RAID 1 and RAID 0 by striping data across mirrored pairs of disks. This offers high performance and redundancy but requires at least four disks.

3.2.4 Prerequisites for Configuring RAID Arrays on Linux

Before we dive into RAID configuration, ensure you meet the following prerequisites:

1. **Linux System**: The tutorial assumes you're running a Linux distribution - RHEL.
2. **Root Privileges**: You need root or superuser privileges to configure RAID arrays.
3. **Multiple Disks**: RAID requires at least two disks, though RAID 5, RAID 6, and RAID 10 require more. The disks can be either physical hard drives or virtual disks.
4. **mdadm Tool**: We will use **mdadm**, a powerful tool for managing RAID arrays in Linux.

 # Installing mdadm

Install the mdadm package, which is used to create and manage RAID arrays on RHEL.

Command:

```
sudo yum install mdadm -y
```

Verify the installation:

```
mdadm --version
```

 # Preparing the Disks

Before creating a RAID array, ensure the disks are unmounted and free of partitions.

Step 1: List Available Disks

Identify available disks on the system.

Command:

```
lsblk
```

Note the disk names (e.g., /dev/sdb, /dev/sdc) to be used for the RAID array.

Step 2: Wipe Disks

Remove existing data and partitions from the disks to prepare them for RAID.

Command:

```
sudo wipefs -a /dev/sdX
```

Replace /dev/sdX with the actual disk name (e.g., /dev/sdb). Repeat for each disk.

Step 3: Partition Disks (Optional)

Partitioning is unnecessary for most RAID setups unless you need specific partitions on the RAID array. If required, use fdisk:

Command:

```
sudo fdisk /dev/sdX
```

Follow prompts to create partitions, then repeat for other disks.

 # Creating the RAID Array

Create a RAID array using mdadm. This example configures a **RAID 1** array (mirroring) with two disks.

Step 1: Create RAID Array

Create a RAID 1 array named /dev/md0 using two disks.

Command:

```
sudo mdadm --create --verbose /dev/md0 --level=1 --raid-devices=2 /dev/sdX /dev/sdY
```

- /dev/md0: The RAID device name.
- --level=1: Specifies RAID 1 (mirroring).
- --raid-devices=2: Uses two disks.
- /dev/sdX /dev/sdY: The disks (e.g., /dev/sdb, /dev/sdc).

The --verbose flag provides detailed output.

Step 2: Verify the RAID Array

Check the RAID array status to ensure it is created and syncing.

Command:

```
cat /proc/mdstat
```

Output shows the array state and synchronization progress (e.g., [UU] indicates both disks are active).

Step 3: Save RAID Configuration

Persist the RAID configuration to ensure it survives reboots.

Command:

```
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm.conf
```

 # Formatting and Mounting the RAID Array

Format the RAID array with a filesystem and mount it for use.

Step 1: Format the RAID Array

Format the array with the xfs filesystem (common on RHEL).

Command:

```
sudo mkfs.xfs /dev/md0
```

Alternatively, use ext4:

```
sudo mkfs.ext4 /dev/md0
```

Step 2: Create a Mount Point

Create a directory for mounting the RAID array.

Command:

```
sudo mkdir /mnt/raid1
```

Step 3: Mount the RAID Array

Mount the RAID array to the created directory.

Command:

```
sudo mount /dev/md0 /mnt/raid1
```

Step 4: Verify the Mount

Confirm the array is mounted correctly.

Command:

```
df -h
```

Look for /dev/md0 mounted at /mnt/raid1.

Step 5: Persistent Mounting

Add the RAID array to /etc/fstab for automatic mounting on boot.

1. Get the UUID of the RAID array:

   ```
   sudo blkid /dev/md0
   ```

2. Edit /etc/fstab:

   ```
   sudo nano /etc/fstab
   ```

3. Add the following line, replacing your-uuid-here with the actual UUID:

   ```
   UUID=your-uuid-here /mnt/raid1 xfs defaults 0 0
   ```

4. Test the fstab configuration:

   ```
   sudo mount -a
   ```

 # Managing and Monitoring RAID Arrays

Monitor and manage the RAID array to ensure reliability and handle disk failures.

*** Checking RAID Array Status**

View detailed information about the RAID array.

Command:

```
sudo mdadm --detail /dev/md0
```

Output includes disk status, array size, and RAID level.

*** Adding a New Disk to the RAID Array**

Replace a failed disk in the RAID array.

1. **Mark the Failed Disk**:

   ```
   sudo mdadm --manage /dev/md0 --fail /dev/sdX
   ```

2. **Remove the Failed Disk**:

   ```
   sudo mdadm --manage /dev/md0 --remove /dev/sdX
   ```

3. **Add the New Disk**:

   ```
   sudo mdadm --manage /dev/md0 --add /dev/sdY
   ```

*** Monitor rebuilding progress:**

```
cat /proc/mdstat
```

 # Performance Optimization for RAID Arrays

Optimize RAID performance based on workload requirements.

*** Stripe Size Adjustment**

For RAID levels with striping (e.g., RAID 5), adjust the stripe size. For example, set a 64 KB stripe size for a RAID 5 array.

Command:

```
sudo mdadm --create --verbose /dev/md0 --level=5 --raid-
devices=3 --chunk=64 /dev/sdX /dev/sdY /dev/sdZ
```

*** Caching and Read-Ahead Settings**

Improve sequential read performance by adjusting read-ahead settings.

Command:

```
sudo blockdev --setra 4096 /dev/md0
```

This sets read-ahead to 4096 blocks (2 MB).

 # Common RAID Array Issues and Troubleshooting

*** Degraded RAID Array**

Symptoms: /proc/mdstat shows [U_] or similar, indicating a failed disk.

Fix: Follow the "Adding a New Disk to the RAID Array" steps to replace the failed disk and rebuild the array.

*** RAID Array Not Mounting After Reboot**

Symptoms: The RAID array is not available after reboot.

Steps:

1. Verify /etc/fstab:

   ```
   cat /etc/fstab
   ```

   Ensure the UUID and mount point are correct.

2. Check RAID assembly:

   ```
   sudo mdadm --assemble --scan
   ```

3. Remount:

   ```
   sudo mount -a
   ```

4. Update mdadm.conf if needed:

   ```
   sudo mdadm --detail --scan | sudo tee -a /etc/mdadm.conf
   ```

*** Slow RAID Performance**

Symptoms: Slow read/write speeds or high latency.

Fixes:

- Adjust stripe size or read-ahead as described in "Performance Optimization."
- Check disk health:

  ```
  sudo smartctl -a /dev/sdX
  ```

- Ensure no background processes (e.g., rebuilding) are consuming resources:

  ```
  cat /proc/mdstat
  ```

 # Conclusion

Configuring RAID arrays on RHEL with mdadm enhances data reliability and performance. By installing mdadm, preparing disks, creating and mounting a RAID array, and optimizing settings, you can build a robust storage solution. Regular monitoring with mdadm --detail and proactive troubleshooting ensure the array remains operational. This guide provides RHEL-specific steps to manage RAID effectively, complementing secure system configurations like ACLs, SSL/TLS, SSH, and firewalls.

3.3 LVM in Linux

LVM deals with the storage in a way that is, by far, more efficient than traditional disk management. With standard disk partitioning, the storage capacity is based on the individual disk capacity, but with LVM, the storage space is managed by combining all the available physical hard drives as if they are part of a pool, making them usable as a whole, instead of handling them individually.

Let's say we have four 1TB drives, with a traditional disk scheme you would handle them individually but with LVM, these four 1TB drives would be considered to be this one 4TB single chunk or aggregated storage capacity. This gives us greater flexibility and control over the disk layout and allows us to manipulate disks in an easier way. One of the main benefits of using LVM is the ability to effortlessly grow the filesystem.

3.3.1 Three core components of LVM in Linux

To understand and use LVM we need to comprehend three main components, they are interconnected and together make what we call a *Logical Volume,* these components are:

- Physical Volume
- Volume Group
- Logical Volume

1. Physical Volume: These are the base block used to create an LVM, physical volumes or "PVs" are simply your **physical storage devices**, whether it is SSD or HDD drives. For a hard drive to be considered a physical volume, it has to be initialized marked as a physical volume so it can be used by the LVM.

2. Volume Group: We can think of a volume group "VG" as a pool that is comprised of physical volumes. Let's say we three 1TB SSD hard drives that are part of a volume group, this "VG" will show up as having a consolidated storage capacity of 3TB, which will then be used to create logical volumes.

3. Logical Volumes: When our VG is created, we can finally create a logical volume. We can carve one or more logical volumes from a single volume group. A logical volume will be treated as a traditional partition that would be then mounted on a directory and be in use.

The figure below illustrates the structure of a Logical Volume:

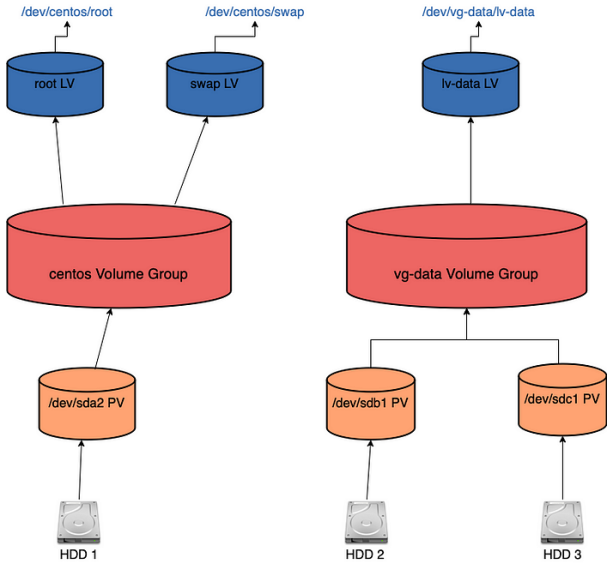


Figure 1: LVM structure

(In this particular example, we happen to have multiple Logical Volumes and Volume groups as it is how the server we use for the demo is configured.)

It starts from physical devices, the hard drives, that are used to create our Physical Volumes or PVs. In this example, we have three separate HDDs, each is used to create one Physical Volume. The partition /dev/sda2 will make our first PV, /dev/sdb1 the second and sdc1 the third.

Moving up we have two separate Volume groups and each of these have their distinctive Logical Volumes, as we said earlier, LVs are carved out from Volume groups, we may have one or many Logical Volumes coming from one VG.

The final layer in this abstraction is the Logical Volume, for example, lv-data that comes from the vg-data volume group, once lv-data is ready it can be formatted and used as a mount point, in this case: **/dev/vg-data/lv-data**.

3.3.2 LVM in Linux: Quick Steps with Short Descriptions

Step 1: Create Physical Volumes (PV)

Command:

```
sudo pvcreate /dev/sdb /dev/sdc
```

Description:
Initializes raw disks (/dev/sdb, /dev/sdc) for use with LVM.

View PVs:

```
sudo pvdisplay
```

Shows details of physical volumes.

Step 2: Create Volume Group (VG)

Command:

```
sudo vgcreate vg_data /dev/sdb /dev/sdc
```

Description:
Combines physical volumes into a storage pool called vg_data.

View VGs:

```
sudo vgdisplay
```

Displays detailed info about volume groups.

Extend VG:

```
sudo vgextend vg_data /dev/sdd
```

Adds another disk to the volume group.

Step 3: Create Logical Volume (LV)

Command:

```
sudo lvcreate -n lv_storage -L 10G vg_data
```

Description:
Creates a 10 GB logical volume named lv_storage inside vg_data.

View LVs:

```
sudo lvdisplay
```

Shows information about created logical volumes.

Step 4: Format Logical Volume

Command:

```
sudo mkfs.ext4 /dev/vg_data/lv_storage
```

Description:
Formats the logical volume with the ext4 file system.

Step 5: Mount the Logical Volume

Command:

```
sudo mkdir /mnt/lv1
sudo mount /dev/vg_data/lv_storage /mnt/lv1
```

Description:
Creates a directory and mounts the LV so it can be used.

Auto-mount on boot:
Add this line to /etc/fstab:

```
/dev/vg_data/lv_storage  /mnt/lv1  ext4  defaults  0 0
```

Resizing Logical Volumes

Extend Logical Volume

Command:

```
sudo lvextend -L +5G /dev/vg_data/lv_storage
```

Description:
Adds 5 GB more to the logical volume.

Resize Filesystem

Command:

```
sudo resize2fs /dev/vg_data/lv_storage
```

Description:
Adjusts the filesystem to use the new space.

One-liner for both:

```
sudo lvextend -r -L +5G /dev/vg_data/lv_storage
```

Use All Free Space in VG

```
sudo lvextend -l +100%FREE /dev/vg_data/lv_storage
```

Delete Logical Volume

Command:

```
sudo umount /mnt/lv1
sudo lvremove /dev/vg_data/lv_storage
```

Description:
Unmounts and deletes the logical volume. **Data is lost.**

Working with LVM Snapshots

Create a Snapshot

Command:

```
sudo lvcreate -s -n lv_snap -L 1G /dev/vg_data/lv_storage
```

Description:
Creates a snapshot called lv_snap of lv_storage.

Mount the Snapshot

Command:

```
sudo mount /dev/vg_data/lv_snap /mnt/snapshot
```

Description:
Access the snapshot like a normal volume to check or copy data.

Restore from Snapshot

Command:

```
sudo umount /mnt/lv1
sudo lvconvert --merge /dev/vg_data/lv_snap
```

Description:
Reverts the original LV back to the state when the snapshot was taken.

Delete a Snapshot

Command:

```
sudo lvremove /dev/vg_data/lv_snap
```

Description:
Deletes the snapshot and finalizes any changes made to the original LV.

Cheat Sheet Table

Task	Command Example	Description
Create PV	pvcreate /dev/sdb	Initialize disk for LVM
Create VG	vgcreate vg1 /dev/sdb	Pool disks into volume group
Extend VG	vgextend vg1 /dev/sdc	Add disk to volume group
Create LV	lvcreate -n lv1 -L 5G vg1	Create logical volume

Format LV	mkfs.ext4 /dev/vg1/lv1	Format with ext4
Mount LV	mount /dev/vg1/lv1 /mnt	Mount volume
Extend LV	lvextend -L +5G /dev/vg1/lv1	Add more space
Resize FS	resize2fs /dev/vg1/lv1	Resize file system
Snapshot	lvcreate -s -n snap -L 1G /dev/vg1/lv1	Create snapshot
Restore Snapshot	lvconvert --merge /dev/vg1/snap	Revert LV to snapshot
Remove LV	lvremove /dev/vg1/lv1	Delete logical volume

3.4 What is an inode?

An **inode (index node)** is a fundamental data structure in Linux and UNIX-based filesystems. It stores important **metadata about each file and directory**, excluding the file name and content.

Each file or directory in a filesystem is assigned a **unique inode number**, which helps the operating system manage and access it efficiently.

3.4.1 What Does an Inode Store?

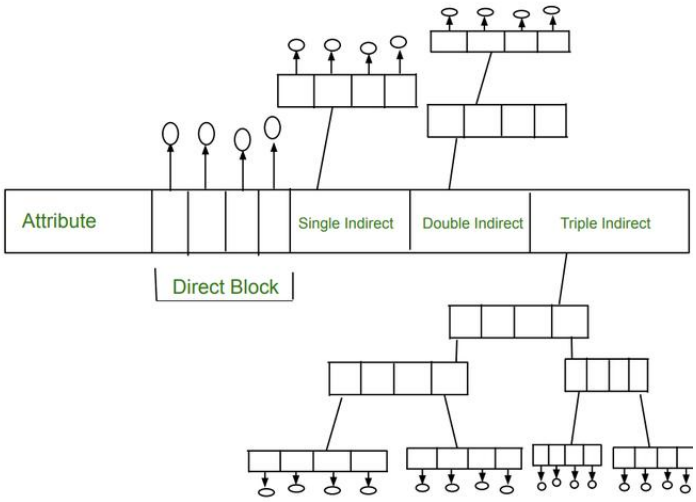
Inodes **do not store the file name or data**, but they **do contain**:

- File type (regular file, directory, etc.)
- File size
- Owner (UID) and group (GID)
- Permissions (read, write, execute)
- Timestamps (last access, modification, change)
- Block pointers (location of data blocks)

3.4.2 How Inodes Work When Files Are Created or Moved

- **New file creation or copying** → gets a **new inode number**
- **Moving a file** within the same filesystem → **inode stays the same**
- **Moving across filesystems → inode changes**

3.4.3 Inode Structure :



3.4.4 How Many Inodes Are There?

- A typical system can have **millions to billions of inodes**
- The **default ratio** is roughly **1 inode per 16 KB of disk space**
- Use df -i to check inode usage in your filesystem

3.4.5 Inode Limit – Why It Matters

When the inode limit is reached, you **cannot create new files**, even if disk space is still available. You might see an error like:

```
No space left on device
```

This can happen if:

- You're using many small files or symbolic links
- Filesystems are created with small block sizes (e.g., ext3)
- Running container-based workloads

3.4.6 Common Inode Problems

High inode usage can lead to:

- System errors and file creation failures
- App crashes or server restarts
- Missed cron jobs or scheduled tasks

Best Practices:

- Regularly clean up:
    - Cache and temp files
    - Old logs and emails
    - Unused directories

3.4.7 How Inodes Help Organize Data

On Linux, files are stored in **data blocks**. Inodes keep track of:

- Where these blocks are on the disk
- The file's size, owner, and timestamps
- Permissions and storage locations

Renaming or moving a file within the same filesystem doesn't change its inode because the actual file stays in place.

3.4.8 Inode Challenges in Large Files

- A single large file may use **multiple inodes**
- Small files in large quantities can **exhaust inodes quickly**
- Changing the inode count requires **reformatting the disk** using the mkfs command

Note: Use mkfs -i option to adjust inode density during filesystem creation

---

3.4.9 Useful Commands for Inodes

Purpose	Command	Example
View file inode	stat <filename>	stat /var/log/lastlog
List inode number	ls -i <file/dir>	ls -i /var/log/lastlog
View directory inode	ls -idl <directory>	ls -idl /var/log
Check inode usage	df -i	df -i /dev/sda1
Count files/inodes in dir	find <dir> | wc -l	`find /var/log

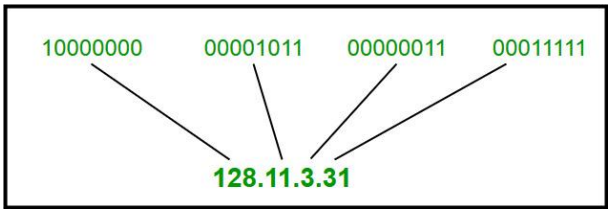
3.5 IPv4 and IPv6 Addresses

3.5.1 What is IPv4?

IPv4, or Internet Protocol version 4, is the original addressing system of the Internet, introduced in 1983. It uses a 32-bit address scheme, which theoretically allows for over 4 billion unique addresses (2^32). IPv4 addresses are typically displayed in decimal format, divided into four octets separated by dots. For example, 192.168.1.1 is a common IPv4 address you might find in a home network.

3.5.2 IPv4 Address Format

IPv4 Address Format is a 32-bit Address that comprises binary digits separated by a dot (.).



IPv4 Address Format

3.5.3 Network Section

The network section of an IP address identifies the class the IP address belongs. There are 3 distinct classes of IP addresses used in computer networks: **Class A**, **Class B**, and **Class C**.

1. What is IPv4 Class A

In **Class A** type of network, the first 8 bits (octet) define the network, while the remaining 24 bits are reserved for the hosts in the network.

- The Public IP addresses range from 1.0.0.0 to 127.0.0.0.
- The Private IP addresses range from 10.0.0.0 to 10.255.255.255.

Addresses **127.0.0.0** to **127.255.255.255** are reserved for loopback and other diagnostic purposes, and hence are not allocated to hosts in a network.

The default subnet mask of class A is **255.0.0.0** with the first 8 bits used to identify the network. The remaining 24 bits are designated for hosts. This class is used in networks that command a large number of hosts. It yields a maximum of **16,777,214** hosts and **126** networks.

2. What is IPv4 Class B

In **Class B**, the first two octets, or 16 bits are used to define the network ID.

- The Public IP addresses range from 128.0.0.0 to 191.255.0.0.
- The private IP range is from 172.16.0.0 to 172.31.255.255.

The default subnet mask is **255.255.0.0** where the first 16 bits define the network ID. This class of IP is typically used for medium-large networks and yields **65,534** hosts per network with a total of **16,382** networks.

3. What is IPv4 Class C

This class of IP is mostly used for small networks such as a home network or a small office or business.

In a **Class C** network, the first two network bits are set to 1 while the third is set to 0, i.e. 1 1 0. The remaining 21 bits of the first three octets define the network ID, and the last octet defines the number of hosts.

As such, Class C IP address produces the highest number of networks amounting to **2,097,150**, and the least number of hosts per network which is **254** hosts.

- The public IP addresses range from 192.0.0.0 to 223.255.255.0.
- The private IP range is from 192.168.0.0 to 192.168.255.255.

The subnet mast is 255.255.255.0.

4. Host Portion (Class D & Class E)

The remaining section of the IP address is the host portion, which is the section that determines the number of hosts in a network. This part uniquely identifies a host in a network. All hosts in the same network share the same network portion.

For example, the following host IP addresses belong to the same network.

192.168.50.15

192.168.50.100

192.168.50.90

3.5.4 Characteristics of IPv4

- **32-bit address length**: Allows for approximately 4.3 billion unique addresses.
- **Dot-decimal notation**: IP addresses are written in a format of four decimal numbers separated by dots, such as 192.168.1.1.
- **Packet structure**: Includes a header and payload; the header contains information essential for routing and delivery.
- **Checksum fields**: Uses checksums in the header for error-checking the header integrity.
- **Fragmentation**: Allows packets to be fragmented at routers along the route if the packet size exceeds the maximum transmission unit (MTU).
- **Address Resolution Protocol (ARP)**: Used for mapping IP network addresses to the hardware addresses used by a data link protocol.
- **Manual and DHCP configuration**: Supports both manual configuration of IP addresses and dynamic configuration through DHCP (Dynamic Host Configuration Protocol).

- **Limited address space**: The main limitation which has led to the development of IPv6 to cater to more devices.
- **Network Address Translation (NAT)**: Used to allow multiple devices on a private network to share a single public IP address.
- **Security**: Lacks inherent security features, requiring additional protocols such as IPSec for secure communications.

3.5.5 Drawbacks of IPv4

- **Limited Address Space**: IPv4 has a limited number of addresses, which is not enough for the growing number of devices connecting to the internet.
- **Complex Configuration**: IPv4 often requires manual configuration or DHCP to assign addresses, which can be time-consuming and prone to errors.
- **Less Efficient Routing**: The IPv4 header is more complex, which can slow down data processing and routing.
- **Security Issues**: IPv4 does not have built-in security features, making it more vulnerable to attacks unless extra security measures are added.
- **Limited Support for Quality of Service (QoS)**: IPv4 has limited capabilities for prioritizing certain types of data, which can affect the performance of real-time applications like video streaming and VoIP.
- **Fragmentation**: IPv4 allows routers to fragment packets, which can lead to inefficiencies and increased chances of data being lost or corrupted.
- **Broadcasting Overhead**: IPv4 uses broadcasting to communicate with multiple devices on a network, which can create unnecessary network traffic and reduce performance.

3.5.6 What is IPv6?

Another most common version of the Internet Protocol currently is IPv6. The well-known IPv6 protocol is being used and deployed more often, especially in mobile phone markets. IPv6 was designed by the Internet Engineering Task Force (IETF) in December 1998 with the purpose of superseding IPv4 due to the global exponentially growing internet of users.

IPv6 stands for Internet Protocol version 6. IPv6 is the new version of Internet Protocol, which is way better than IPv4 in terms of complexity and efficiency. IPv6 is written as a group of 8 hexadecimal numbers separated by colon (:). It can be written as 128 bits of 0s and 1s.

3.5.7 IPv6 Address Format

IPv6 Address Format is a 128-bit IP Address, which is written in a group of 8 hexadecimal numbers separated by colon (:).

3.5.8 To switch from IPv4 to IPv6, there are several strategies:

- **Dual Stacking**: Devices can use both IPv4 and IPv6 at the same time. This way, they can talk to networks and devices using either version.
- **Tunneling**: This method allows IPv6 users to send data through an IPv4 network to reach other IPv6 users. Think of it as creating a "tunnel" for IPv6 traffic through the older IPv4 system.
- **Network Address Translation (NAT)**: NAT helps devices using different versions of IP addresses (IPv4 and IPv6) to communicate with each other by translating the addresses so they understand each other.

3.5.9 Characteristics of IPv6

- IPv6 uses 128-bit addresses, offering a much larger address space than IPv4's 32-bit system.
- IPv6 addresses use a combination of numbers and letters separated by colons, allowing for more unique addresses.
- The IPv6 header has fewer fields, making it more efficient for routers to process.
- IPv6 supports Unicast, Multicast, and Anycast, but no Broadcast, reducing network traffic.
- IPv6 allows flexible subnetting (VLSM) to divide networks based on specific needs.
- IPv6 uses Neighbor Discovery for MAC address resolution instead of ARP.
- IPv6 uses advanced routing protocols like OSPFv3 and RIPng for better address handling.
- IPv6 devices can self-assign IP addresses using SLAAC, or use DHCPv6 for more control.
- IPv6 handles fragmentation at the sender side, not by routers, improving speed.

3.5.10 Difference Between IPv4 and IPv6

IPv4	IPv6
IPv4 has a 32-bit address length	IPv6 has a 128-bit address length
It Supports Manual and DHCP address configuration	It supports Auto and renumbering address configuration

IPv4	IPv6
In IPv4 end to end, connection integrity is Unachievable	In IPv6 end-to-end, connection integrity is Achievable
It can generate 4.29×10 9 address space	The address space of IPv6 is quite large it can produce 3.4×10 38 address space
The Security feature is dependent on the application	IPSEC is an inbuilt security feature in the IPv6 protocol
Address representation of IPv4 is in decimal	Address representation of IPv6 is in hexadecimal
Fragmentation performed by Sender and forwarding routers	In IPv6 fragmentation is performed only by the sender
In IPv4 Packet flow identification is not available	In IPv6 packet flow identification are Available and uses the flow label field in the header
In IPv4 checksum field is available	In IPv6 checksum field is not available
It has a broadcast Message Transmission Scheme	In IPv6 multicast and anycast message transmission scheme is available
In IPv4 Encryption and Authentication facility not provided	In IPv6 Encryption and Authentication are provided
IPv4 has a header of 20-60 bytes.	IPv6 has a header of 40 bytes fixed
IPv4 can be converted to IPv6	Not all IPv6 can be converted to IPv4
IPv4 consists of 4 fields which are separated by addresses dot (.)	IPv6 consists of 8 fields, which are separated by a colon (:)
IPv4's IP addresses are divided into five different classes. Class A , Class B, Class C, Class D , Class E.	IPv6 does not have any classes of the IP address.
IPv4 supports VLSM(Variable Length subnet mask).	IPv6 does not support VLSM.
Example of IPv4: 66.94.29.13	Example of IPv6: 2001:0000:3238:DFE1:0063:0000:0000:FEFB

3.5.11 Benefits of IPv6 over IPv4

The recent Version of IP IPv6 has a greater advantage over IPv4. Here are some of the mentioned benefits:

- **Larger Address Space:** IPv6 has a greater address space than IPv4, which is required for expanding the IP Connected Devices. IPv6 has 128 bit IP Address rather and IPv4 has a 32-bit Address.

- **Improved Security:** IPv6 has some improved security which is built in with it. IPv6 offers security like Data Authentication, Data Encryption, etc. Here, an Internet Connection is more Secure.
- **Simplified Header Format:** As compared to IPv4, IPv6 has a simpler and more effective header Structure, which is more cost-effective and also increases the speed of Internet Connection.
- **Prioritize:** IPv6 contains stronger and more reliable support for QoS features, which helps in increasing traffic over websites and increases audio and video quality on pages.
- **Improved Support for Mobile Devices:** IPv6 has increased and better support for Mobile Devices. It helps in making quick connections over other Mobile Devices and in a safer way than IPv4.

3.5.12 Why IPv4 is Still in Use?

1. **Infrastructure Compatibility** Many systems and devices are built for IPv4 and require significant updates to support IPv6, including routers, switches, and computers.
2. **Cost of Transition** – Switching to IPv6 can be expensive and complex, involving hardware updates, software upgrades, and training for personnel.
3. **Lack of Immediate Need** – Techniques like NAT (Network Address Translation) help extend the life of IPv4 by allowing multiple devices to share a single public IP address, reducing the urgency to switch to IPv6.
4. **Coexistence Strategies** – Technologies that allow IPv4 and IPv6 to run simultaneously make it easier for organizations to adopt IPv6 gradually while maintaining their existing IPv4 systems.
5. **Slow Global Adoption** – The adoption of IPv6 varies significantly around the world, which necessitates the continued support of IPv4 for global connectivity.
6. **Lack of Visible Benefits** – Many users and organizations don't see immediate improvements with IPv6 if they don't face an IP address shortage, reducing the incentive to upgrade.

3.6 Configuring IPv4 and IPv6 Addresses in Linux

3.6.1 Dynamic (DHCP) and Static IP Configuration

IP allocation on client machines or any end-point devices connected to a network is done either using the **DHCP** protocol or manual configuration where IP addresses are statically allocated.

DHCP IP Address

DHCP (**Dynamic Host Configuration Protocol**) is a client-server protocol that dynamically allocates IP addresses to client systems on a network. The **DHCP** server, which in most cases is a router, contains a pool of addresses that it leases out to client devices on a network for a

certain period of time. Thus, it simplifies and makes the configuration of IP addresses more efficient. Once the lease time lapses, the client acquires a new IP address.

Most systems, by default, are configured to obtain an IP automatically using the DHCP protocol. This eliminates the possibility of IP conflicts in a network where two devices share the same IP address.

The drawback of DHCP is that the IP addresses change once the lease expires. If a server is set to acquire an IP via DHCP, this will lead to connectivity issues once the IP address changes. And this is where static IP configuration comes in.

3.6.2 Static IP Address

In static IP configuration, IP addresses are manually configured on a client system, especially servers. Unlike dynamically allocated addresses, statically configured IP addresses remain the same and do not change.

However, the static configuration requires a lot of work from network admins. They have to manually log in and configure the static IP along with other details such as subnet mask, DNS servers, and gateway IP. In addition, they need to keep track of all the client systems with static IP addresses.

In this tutorial, we will focus on how to statically configure IP addresses on various systems.

3.6.3 Configure IPv4 Address on RHEL

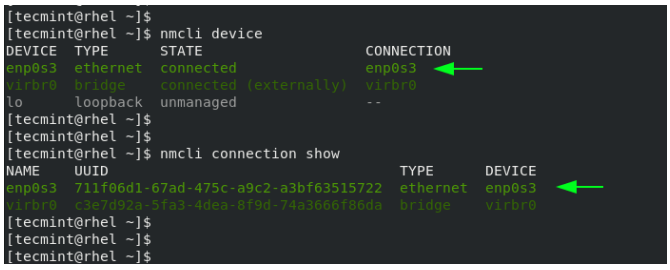
In Red Hat distributions, the nmcli (NetworkManager Command Line Interface) command-line tool is one of the most preferred ways of configuring an IPv4 address. It does so using the NetworkManager service.

To view the network interface name attached to your system, execute the command:

```
$ nmcli device
```

To display the active connection, run the command:

```
$ nmcli connection show
```



Find Network Interface in RHEL

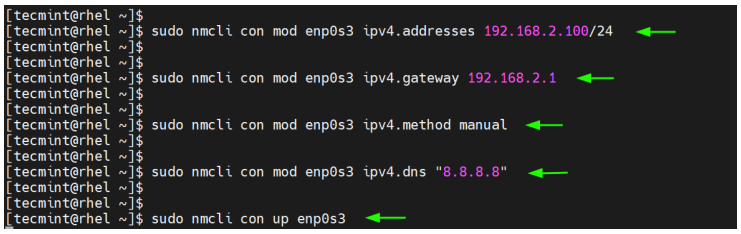
In **RHEL 9** and other Red Hat distributions based on RHEL, the network configuration file resides in the **/etc/sysconfig/network-scripts** directory. In our case, the configuration file is **ifcfg-enp0s3**.

We will assign a static IPv4 address on the interface '**enp0s3**' as shown:

```
IP: 192.168.2.100
netmask: 255.255.255.0
gateway: 192.168.2.1
dns: 8.8.8.8
```

To do so, we will run the following commands:

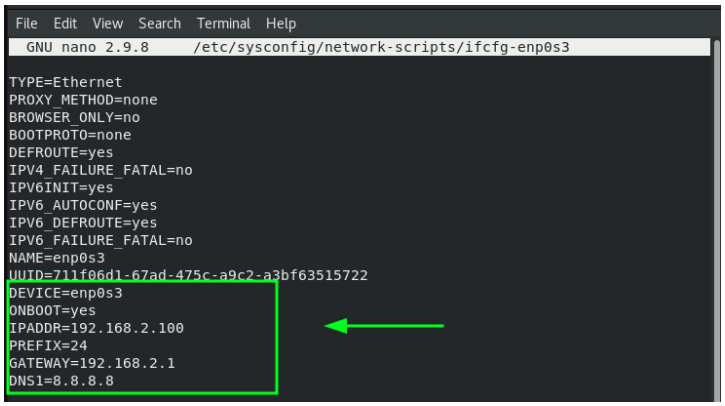
```
$ sudo nmcli con mod enp0s3 ipv4.addresses 192.168.2.100/24
$ sudo nmcli con mod enp0s3 ipv4.gateway 192.168.2.1
$ sudo nmcli con mod enp0s3 ipv4.method manual
$ sudo nmcli con mod enp0s3 ipv4.dns "8.8.8.8"
$ sudo nmcli con up enp0s3
```



Set Static IP Address in RHEL

The commands save the changes inside the associated network configuration file. You can view the file using your preferred text editor

```
$ sudo nano etc/sysconfig/network-scripts ifcfg-enp0s3
```



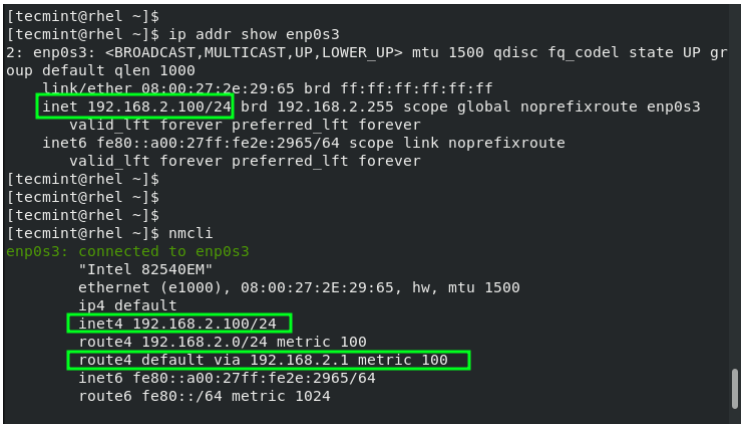
RHEL Network Configuration

To confirm the new IP address, run the following command

```
$ ip addr show enp0s3
```

You can also run the **nmcli** command without any command-line options and the active interface will be displayed at the top.

```
$ nmcli
```



Find IP Address in RHEL

3.6.4 How to Configure Hostname in Linux

A well-configured system should be able to resolve its hostname or domain name to the IP address configured. Usually, the hostname and IP address mapping is done in the **/etc/hosts** file.

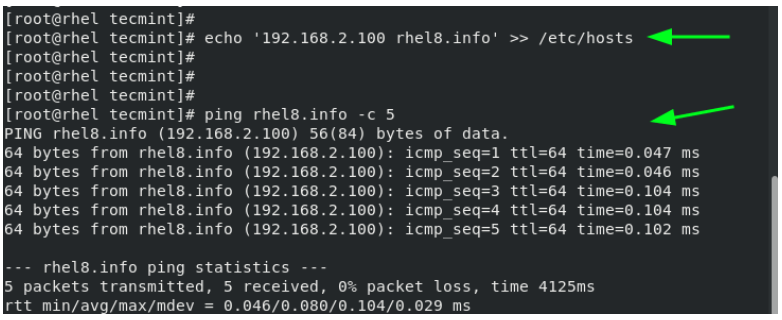
To configure hostname resolution, add a host's entry to the **/etc/hosts** file. This entry includes the host's IP address and the hostname as shown.

```
$ echo '192.168.2.150  debian-11' >> /etc/hosts
```

Be sure to update the **/etc/hosts** file on every Linux system that you intend to connect to the system on the same local network.

Once done, you can successfully ping the hostname of the Linux machine.

```
$ ping debian-11 -c 5
```



Ping Hostname in Linux

3.7 Network troubleshooting Tools in Linux

Networks are the foundation of modern communication and business operations in the digital world of today. The exchange of data and information between devices and users is made

possible by networks, which range from local area networks (LANs) to wide area networks (WANs) and the Internet. However, problems with the network are inevitable and have the potential to disrupt normal operations, resulting in downtime, productivity loss, and revenue loss. Tools for diagnosing and troubleshooting networks come into play at this point.

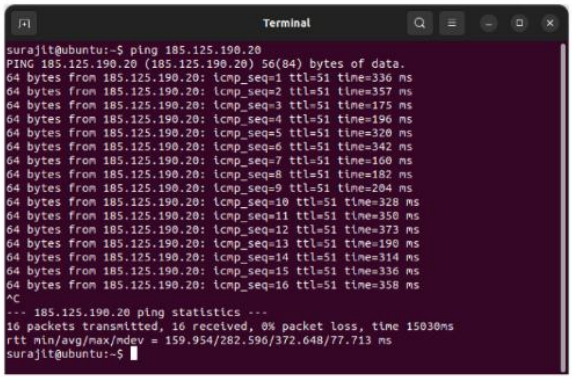
3.7.1 Basic Network Troubleshooting Commands

There are several network troubleshooting commands in Linux. We have listed the most important ones below. Let's find out.

1. ping

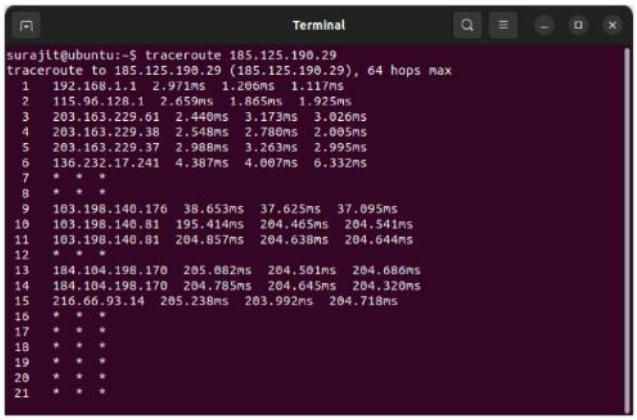
ping is a simple yet essential network troubleshooting command that is utilized to test the reachability of a host or network device and measure the round-trip time (RTT) for data packets that goes out from the source to the destination and back. ping waits for ICMP Echo Reply messages from the target host after sending ICMP Echo Request messages to it. Network administrators can determine the packet loss rate, the latency or delay in network communication, and whether a host or network device is online by analyzing the ping results.

Basically ping command is used to diagnose problems with network connectivity, such as determining if a host can be reached, evaluating the response time of a remote server or website, identifying network congestion or high latency, and confirming that the network connects various network segments.



2. traceroute

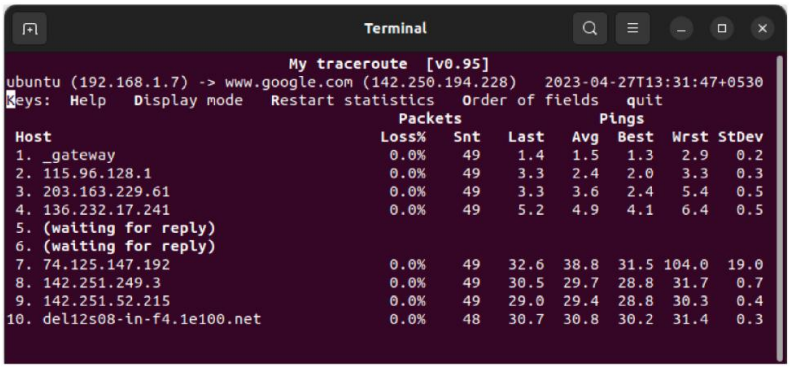
An important network troubleshooting command, traceroute, assists network administrators in tracing the path that packets take from the source to the destination host or IP address. traceroute shows the hops or switches that the data packets go through and the response time for each hop. This permits sysadmins to distinguish the path taken by the data packets and pinpoint any network segments or switches that are creating delays or packet laws.



3. mtr

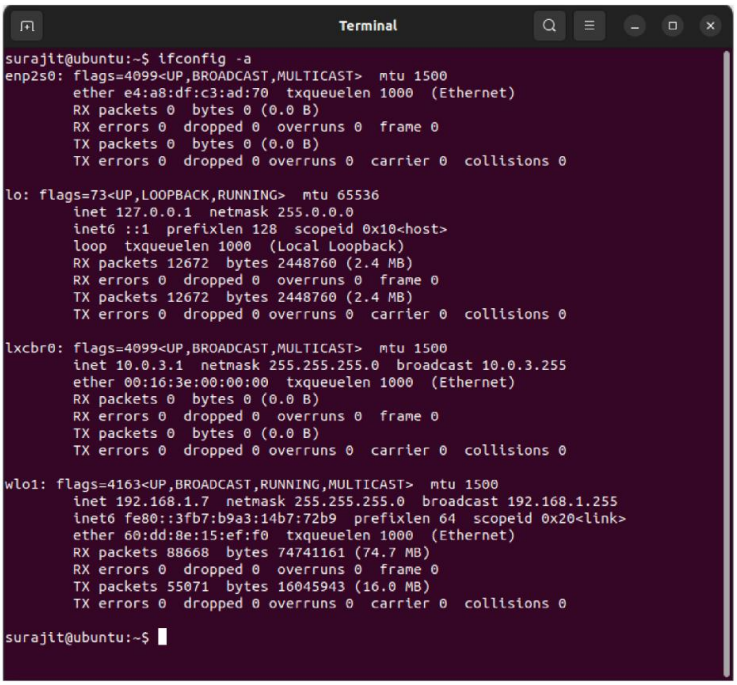
mtr (My traceroute) is a command-line utility in Linux that combines the functions of the traceroute and ping commands. mtr provides network administrators with a comprehensive view of the network path between a source and destination IP address by continuously sending packets and analyzing the responses.

mtr generates a graphical display of network statistics such as packet loss, latency, and route changes, allowing administrators to quickly identify network problems and troubleshoot connectivity issues. mtr can also be used to generate reports and export data in various formats, making it a powerful tool for network performance analysis and monitoring.



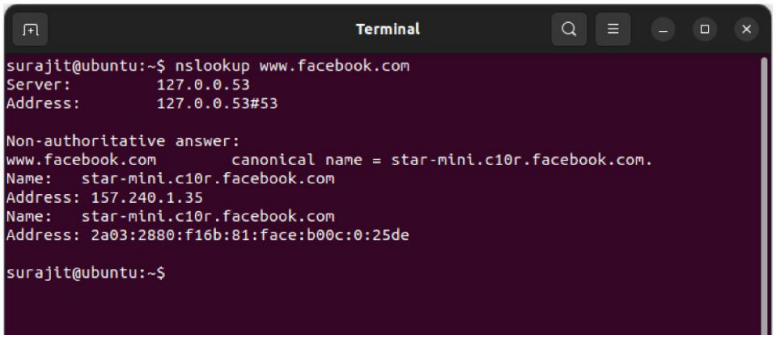
4. ifconfig

The basic network troubleshooting command ipconfig (Unix/Linux) displays a host's IP configuration and network settings. It provides details about the host's MAC address, IP address, default gateway, DNS servers, and other network configuration details. Network administrators can use this information to check a host's network settings, resolve IP addressing, DNS resolution, network connectivity issues, and more.



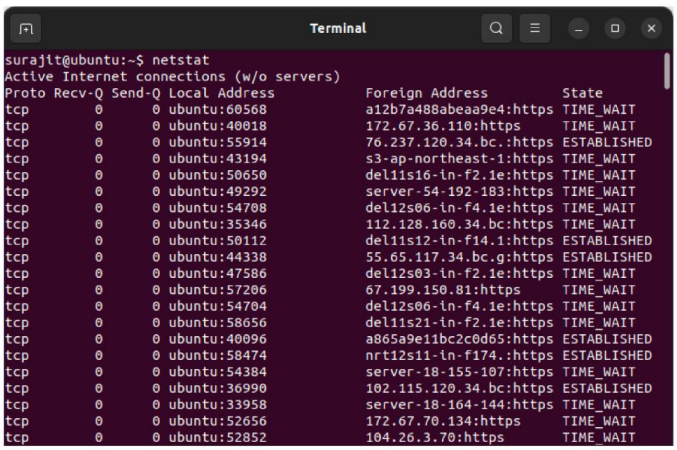
5. nslookup

nslookup is an essential command for troubleshooting networks that queries DNS (Domain Name System) servers to convert IP addresses into domain names and vice versa. nslookup gives data about the IP address related to a domain name, the legitimate DNS server for the domain, and other DNS-related information. This data is helpful for network admins to confirm DNS resolution, detect DNS-related issues, and confirm appropriate name resolution in the network.



6. netstat

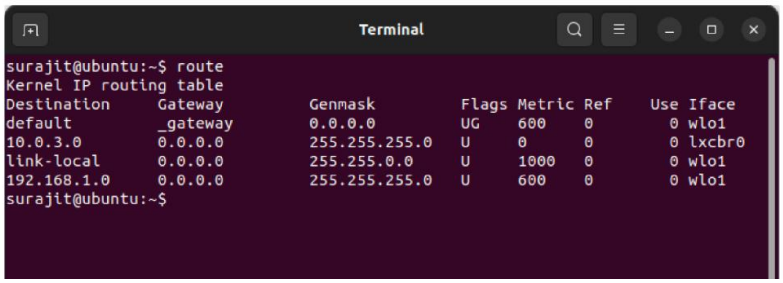
netstat (Network Insights) is an essential network troubleshooting command that shows active network connections, listening ports, and statistical network information of a host. The active network connections, the protocols used (TCP or UDP), the local and remote IP addresses and port numbers, and the connections' current status are all displayed by the netstat command.



7. route

route is a critical command for troubleshooting networks, which displays the host's routing table. The routing table contains data about the routes and paths used to exchange data packets from the source to the destination. The interface used, the destination network or IP address, the gateway or next hop, and other routing details are all provided by route.

This data is helpful for network administrators to check the routing setup of a host, investigate routing issues, and confirm proper routing in the network. Troubleshooting routing loops, identifying incorrect routes, identifying routing misconfigurations, and verifying the entries in the routing table are all common uses of route.



3.7.2 Advanced Network Troubleshooting Tools for Network Administrators

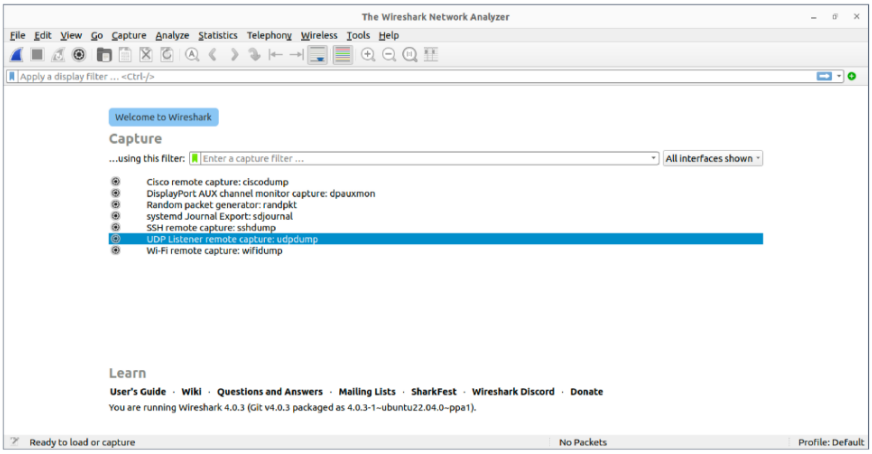
In addition to the basic network troubleshooting commands mentioned above, there are advanced network troubleshooting tools that offer more comprehensive features for in-depth network analysis and monitoring. Some of the popular advanced network troubleshooting tools are mentioned below. Let's find out.

1. Uptrends Uptime Monitor

Uptrends Uptime Monitor is a network monitoring tool that provides real-time monitoring of websites, servers, APIs, and other network resources from multiple global locations. It offers features such as uptime monitoring, performance monitoring, error monitoring, and alerting. Uptrends Uptime Monitor allows network administrators to monitor network resources' availability, response time, and performance, detect real-time issues, and receive notifications in case of failures or performance degradation.

2. Wireshark

Wireshark is a popular open-source network protocol analyzer that allows network administrators to capture, analyze, and troubleshoot network traffic in real time. Wireshark supports various protocols, including TCP/IP, UDP, HTTP, DNS, FTP, and many others, and provides detailed information about each packet, including source and destination IP addresses, port numbers, protocol type, packet size, and payload content. Wireshark also offers powerful filtering and analysis capabilities, allowing network administrators to drill down into network traffic, detect abnormalities, and identify and resolve network issues.



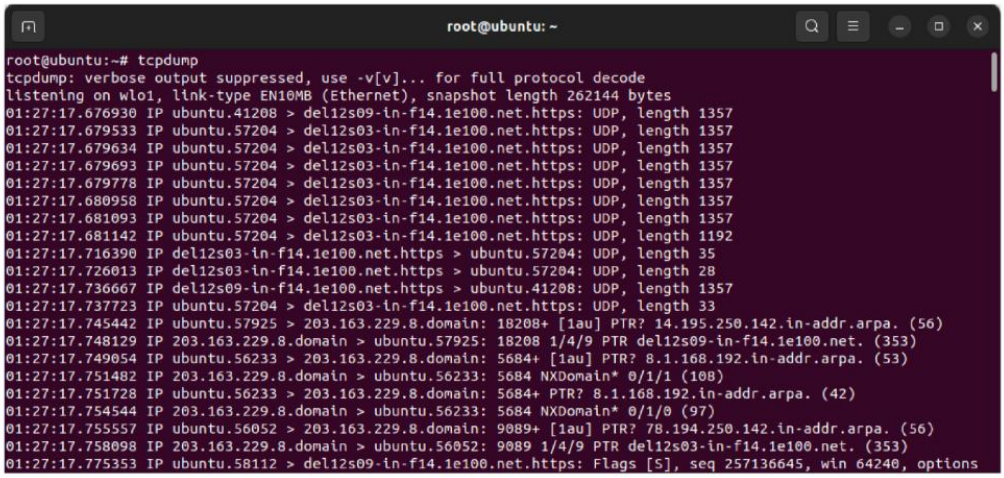
3. Wifi Explorer

Wifi Explorer is a network diagnostic tool specifically designed for analyzing and troubleshooting wireless networks. It provides detailed information about wireless networks, including signal strength, channel utilization, channel interference, MAC addresses, encryption methods, and other network parameters. Wifi Explorer also offers graphical visualizations of wireless networks, allowing network administrators to identify overlapping channels, and interference sources, and optimize the placement of access points.

4. Tcpdump

tcpdump is a command-line tool for network packet capture and analysis. It is used to capture and display network traffic passing through a network interface in real-time or to save the captured packets to a file for later analysis. tcpdump can capture packets at the network layer and analyze the contents of those packets at the application layer. It supports a wide range of protocols and can diagnose network issues, troubleshoot connectivity problems, and perform network security analysis.

tcpdump is available on most Unix-like operating systems, including Linux, macOS, and FreeBSD, and is typically installed by default. It is a powerful tool that requires some expertise to use effectively, but it can be very useful for network administrators, security professionals, and other technical users.



5. Subnet and IP Calculator

A subnet and IP calculator is a tool that helps network administrators calculate IP address ranges, subnet masks, network addresses, broadcast addresses, and other IP address parameters. It also provides information about the number of host addresses, usable IP addresses, and subnet classifications based on different IP addressing schemes. Subnet and IP calculators help design and configure IP networks, verify IP addressing schemes, and troubleshoot IP addressing issues.

Subnet and IP calculators are commonly used for designing and configuring IP networks, verifying IP addressing schemes, troubleshooting IP addressing issues, and optimizing IP address allocation in complex network environments.

6. Datadog Network Performance Monitoring

Datadog Network Performance Monitoring is a comprehensive network monitoring tool that provides real-time visibility into network performance, traffic patterns, and network device health. It offers features such as network traffic monitoring, network device monitoring, network latency monitoring, and alerting.

Datadog Network Performance Monitoring allows network administrators to monitor network performance metrics, detect network anomalies, troubleshoot network issues, and optimize network performance. It is also used in optimizing network performance in large-scale and complex network environments.

7. Nagios

Nagios is a popular open-source network monitoring tool that allows network administrators to monitor the availability, performance, and status of network resources, including servers, routers, switches, applications, and services. Nagios offers features such as uptime monitoring, performance monitoring, event logging, alerting, and reporting.

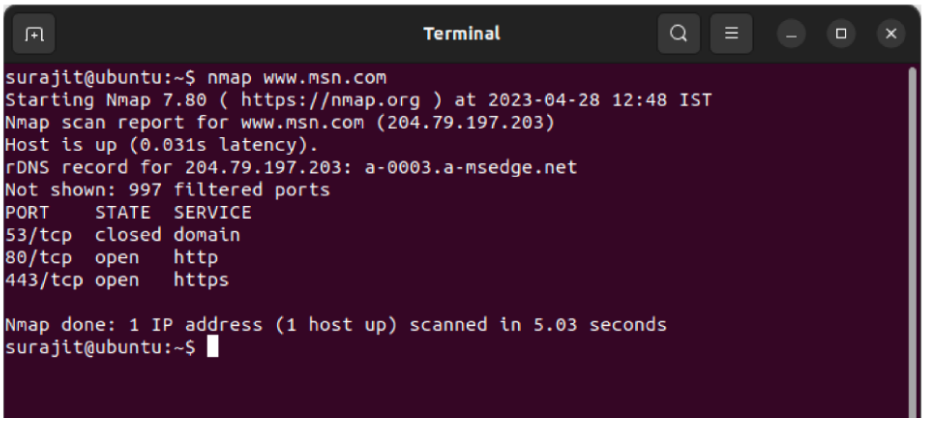
It supports plugins and extensions, allowing network administrators to customize monitoring configurations and integrate with other network management tools. Nagios falls under the list

of commonly used network troubleshooting tools for ensuring optimal network operation in small to large-scale network environments.

8. Nmap

nmap (Network Mapper) is a powerful network scanning tool that allows network administrators to discover hosts, services, and open ports in a network. nmap offers various scanning techniques, including ping scanning, port scanning, version detection, and OS detection. It provides detailed information about the hosts, services, and ports discovered, allowing network administrators to identify potential security vulnerabilities, detect network devices, and troubleshoot network connectivity issues.

nmap also comes under the list of commonly used Network troubleshooting tools for network reconnaissance, network mapping, network security checks, and troubleshooting network connectivity issues. It is a versatile tool that provides valuable information about the network topology, devices, and services running on the network.



Q. Suppose you are a system administrator at Rastriya Banijya Bank (RBB), RBB decides to open a new branch in Pokhara with 120 staffs providing branded PC for each of them. How will you manage IP address configuration details to avoid IP conflict, and time management?

As the system administrator, my primary goals here are:

1. **Avoid IP conflicts** in a network of 120+ devices.
2. **Ensure accurate time management** across all systems for logging, transactions, and security.

Here's how I'd approach it:

---

1. IP Address Configuration — Avoiding IP Conflicts

✅ **Use a Centralized DHCP Server**

- Set up a **DHCP server** (could be on a dedicated appliance, router, or Linux server).
- The DHCP server will:
    - Dynamically assign IP addresses from a defined pool.
    - Avoid duplicate IPs by tracking issued leases.
    - Automatically configure subnet mask, gateway, and DNS for all 120 PCs.

✅ **Define a Proper IP Addressing Scheme**

- Example subnet: 192.168.50.0/24 (supports up to 254 hosts).
- DHCP range: 192.168.50.100 – 192.168.50.200 for PCs.
- Reserve specific IPs **outside** this range for:
    - Servers (e.g., 192.168.50.10)
    - Printers, switches, CCTV, etc.

✅ **Use DHCP Reservations for Key Systems**

- For critical systems (like file servers, printers), bind their MAC addresses to fixed IPs via DHCP reservation.
- This keeps their IP consistent without needing static config on the device itself.

✅ **Network Segmentation (Optional, for future scale)**

- Use VLANs to segment traffic (e.g., admin, teller, CCTV, guest Wi-Fi).
- Helps in managing broadcast domains and improves security.

---

2. Time Management — Ensuring Accurate Time Across All Devices

✅ **Set Up an NTP Server (Network Time Protocol)**

- Deploy an **internal NTP server** on a Linux system (e.g., using chronyd or ntpd).
- Configure all PCs, servers, and network equipment to sync with this internal NTP server.

✅ **Sync Internal NTP Server with Public NTP Sources**

- The internal server syncs with reliable upstream sources (like time.nist.gov or pool.ntp.org).
- This way, all devices are aligned to the same accurate time source.

✅ **Apply Group Policy (for Windows PCs, if applicable)**

- Use Active Directory Group Policy to push time server settings to all Windows clients.
- Ensures even if DHCP is down, time settings are enforced.

Summary:

Task	Method
IP Address Assignment	DHCP Server with well-defined IP pool and reservations
Avoiding IP Conflicts	Central management via DHCP + reserved IPs
Time Synchronization	Internal NTP server syncing with external sources
Ensuring Consistency	All clients point to internal NTP via DHCP or GPO

By automating IP allocation and time synchronization, I ensure a **stable, conflict-free network** and **accurate system logs**, which is critical for banking operations like auditing, transaction tracking, and security compliance.

Q. How do DHCP and DNS servers contribute to network functionality?

DHCP and DNS are two core network services that play very different but complementary roles in keeping a network functional, automated, and user-friendly.

---

1. DHCP (Dynamic Host Configuration Protocol)

Role: Automatically assigns IP addresses and other network configuration details to devices on the network.

How it contributes:

- **Reduces manual work**: No need to configure IPs manually on every device.
- **Prevents conflicts**: Ensures that each device gets a unique IP address.
- **Provides essential info**: Sends out subnet mask, default gateway, and DNS server addresses.
- **Supports dynamic environments**: Devices can join and leave the network without admin intervention.

Example: When a laptop connects to Wi-Fi, the DHCP server gives it an IP address, gateway, and DNS settings on the spot.

---

2. DNS (Domain Name System)

Role: Translates human-readable domain names (like google.com) into IP addresses (like 142.250.190.14).

How it contributes:

- **Makes the internet usable**: Users don't need to remember IP addresses.
- **Speeds up connections**: DNS caching improves response time when visiting frequently accessed sites.
- **Supports scalability**: As services move or scale (e.g., load balancing), DNS ensures clients reach the right destination without changes on the user end.

Example: When you open a browser and type example.com, your system queries a DNS server to get the IP address, then connects to that server to load the site.

Conclusion:

- **DHCP** handles **automatic network configuration**.
- **DNS** handles **human-friendly name resolution**.

Together, they allow devices to join the network quickly and access services easily, keeping everything smooth and scalable without constant admin input.

Unit 4: Network Services and File Sharing

4.1 DNS and DHCP Configuration,

4.1.1 What is DNS?

DNS or Domain Name System is a distributed database system that translates website (Domain) names into IP addresses. When a user enters a website name into their web browser, the browser sends a DNS query to a DNS server. The DNS server then looks up the IP address associated with the domain name and returns it to the browser, allowing the browser to connect to the desired website or service.

DNS uses a hierarchical naming system, with each domain name consisting of one or more labels separated by dots. The highest level of the DNS hierarchy is the root domain, followed by top-level domains such as .com, .org, and .net, for example google.com or facebook.com.

4.1.2 What is DHCP?

DHCP stands for Dynamic Host Configuration Protocol. It is a network protocol used to automatically assign IP addresses and other network configuration parameters, such as subnet mask, default gateway, and DNS server information, to devices on a network.

When a device joins a network that uses DHCP, it sends a broadcast message requesting network configuration information. A DHCP server on the network responds to the request by offering an IP address and other configuration parameters to the device. If the device accepts the offer, it will be assigned an IP address and can then communicate with other devices on the network.

DHCP makes it easier to manage and configure a large network with many devices, as it eliminates the need to manually configure IP addresses on each device. It also allows for more efficient use of IP addresses, as devices can be assigned temporary addresses from a pool of available addresses, rather than permanently reserving a specific address for each device.

4.1.3 DNS Server Configuration (BIND DNS) in Linux

You'll configure:

- An **Apache web server** hosting a simple website.
- A **BIND DNS server** resolving www.myweb.com to your IP (192.168.20.128).
- Client settings to use your DNS server for name resolution within your LAN.

---

1. Set Up the Web Server

1. **Install Apache**:

```
sudo dnf install httpd
```

2. **Start and enable Apache**:

```
sudo systemctl start httpd
sudo systemctl enable httpd
```

3. **Verify** it's running:

```
sudo systemctl status httpd
```

4. **Deploy a test page**:
    - Place a regular HTML file (e.g., index.html) in /var/www/html/.
5. **Adjust firewall**:

```
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --reload
```

6. **Test locally:** Visit http://localhost/ or http://192.168.20.128/.

---

2. Install & Start BIND DNS

1. **Install BIND**:

```
sudo dnf install bind bind-utils
```

2. **Start and enable named**:

```
sudo systemctl start named
sudo systemctl enable named
```

3. **Confirm it's up**:

```
sudo systemctl status named
```

---

3. Configure BIND (/etc/named.conf)

1. **Backup config**:

```
sudo cp /etc/named.conf /etc/named.bak
```

2. **Edit file**:

```
sudo vim /etc/named.conf
```

3. **Under options**, ensure:

```
listen-on port 53 { 127.0.0.1; 192.168.20.128; };
listen-on-v6 port 53 { ::1; };
allow-query { any; };
```

4. Add zone definitions at bottom:

```
   zone "myweb.com" IN {
        type master;
```

```
        file "myweb.com.fwd";
        allow-update { none; };
        allow-query { any; };
    };

    zone "20.168.192.in-addr.arpa" IN {
        type master;
        file "myweb.com.rev";
        allow-update { none; };
        allow-query { any; };
    };
```

---

4. Create Forward Zone File

1. **Create/edit** /var/named/myweb.com.fwd:

```
$TTL 1d
@ IN SOA ns.myweb.com. admin.myweb.com. (
        2025071801 ; Serial
        4h         ; Refresh
        1h         ; Retry
        1w         ; Expire
        1d         ; Minimum TTL
)
@               IN NS    ns.myweb.com.
ns              IN A     192.168.20.128
www             IN A     192.168.20.128
ftp             IN CNAME www.myweb.com.
```

5. Create Reverse Zone File

1. Create/edit `/var/named/myweb.com.rev`:

```
    $TTL 1d
    @ IN SOA ns.myweb.com. admin.myweb.com. (
            2025071801 ; Serial
            4h
            1h
            1w
            1d
```

```
    )
    @       IN NS    ns.myweb.com.
    ns      IN A     192.168.20.128
    128     IN PTR   ns.myweb.com.
```

---

6. Set Permissions & Restart Named

```
sudo chown named:named /var/named/myweb.com.fwd
sudo chown named:named /var/named/myweb.com.rev

sudo named-checkconf
sudo named-checkzone myweb.com /var/named/myweb.com.fwd
sudo named-checkzone 20.168.192.in-addr.arpa /var/named/myweb.com.rev

sudo systemctl restart named
```

Enable DNS in firewall:

```
sudo firewall-cmd --add-service=dns --permanent
sudo firewall-cmd --reload
```

---

7. Configure Client to Use DNS

On the client, add your DNS server IP to /etc/resolv.conf or network config:

```
nameserver 192.168.20.128
```

Then restart the Bind Server

```
sudo systemctl restart named
sudo systemctl status named
```

---

8. Test DNS Resolution

From the client, run:

```
nslookup www.myweb.com
dig www.myweb.com
dig -x 192.168.20.128
```

Expected results:

- Forward lookup returns 192.168.20.128.
- Reverse lookup returns ns.myweb.com.
- Using a browser with DNS set to your server: http://www.myweb.com/

✅ **Summary Table**

Component	IP Address	Domain
Web Server	192.168.20.128	hosts website
DNS Server	192.168.20.128	resolves domains
Zones	Forward & Reverse	myweb.com zones
TTL	1 day (1d)	in zone files
Refresh	4h; Retry 1h; Expire 1w; Min 1d	in zone files

4.2 Hostname Resolution

When we want to connect to a website or another computer over a network, we often use *hostnames* (like example.com). However, computers communicate using IP addresses. So, the system must *resolve* a hostname to an IP address. Normally, this is done via **DNS (Domain Name System)**.

But in Linux, there's a special local file called **/etc/hosts**, where you can define your own mappings between hostnames and IP addresses. This file is checked *before* contacting any DNS server, which makes it useful for:

- Testing websites locally
- Redirecting domains
- Quick hostname lookups without internet

---

2. Setting Up /etc/hosts

To manually map a hostname:

1. Open the /etc/hosts file using a text editor like nano, vim, or gedit:

```
sudo nano /etc/hosts
```

2. Add a line like this at the end:

```
127.0.0.1   example.com
```

This tells your system to resolve example.com to the *localhost* IP 127.0.0.1.

3. Save and exit the editor.
4. Optionally, to test IPv6, you can add:

```
::1    example.com
```

⚠️ Don't forget to remove these lines after testing to avoid future confusion.

---

3. Resolving Hostnames with getent

The getent command retrieves entries from system databases like hosts, passwd, and group.

Using ahosts (Shows IPv4 and IPv6):

```
getent ahosts example.com
```

Output:

```
127.0.0.1     STREAM example.com
127.0.0.1     DGRAM
127.0.0.1     RAW
```

Using hosts (Shows only one IP version):

```
getent hosts example.com
```

If you added both 127.0.0.1 and ::1 for example.com, this may return only:

```
::1    example.com
```

Key Point:

- ahosts = shows all address types (IPv4 & IPv6)
- hosts = might return just one

---

4. Using ping to Resolve Hostnames

Ping not only checks if a host is reachable but also reveals the resolved IP address.

```
ping example.com
```

Example Output:

```
PING example.com (127.0.0.1) 56(84) bytes of data.
```

This shows example.com is resolved to 127.0.0.1.

Press Ctrl + C to stop the ping.

---

5. Resolving Hostnames with Python

If Python 3 is installed, run this one-liner:

```
python3 -c 'import socket;
print(socket.getaddrinfo("example.com","http")[0][4][0])'
```

Output:

```
127.0.0.1
```

This uses Python's socket module to get the IP address.

---

6. Resolving Hostnames with Perl

If Perl is installed:

```
perl -e 'use Socket; print inet_ntoa(inet_aton("example.com")) .
"\n";'
```

Output:

```
127.0.0.1
```

This only returns the IPv4 address.

---

7. Conclusion

You now know several ways to resolve hostnames in Linux while prioritizing the /etc/hosts file. Here's a summary:

Method	Reads /etc/hosts	Supports IPv6	Simple Output
getent hosts	✅	❌ (one only)	✅
getent ahosts	✅	✅	Moderate
ping	✅	✅	Verbose
Python script	✅	❌	✅
Perl script	✅	❌	✅

Using /etc/hosts is very effective for local testing and custom mappings. Just remember to clean up the file after use.

4.3 DNS Queries

When a user types a website address (like gatech.edu) into a browser, that request must be translated into an IP address so the computer can find and connect to the appropriate server.

This translation process is known as **DNS resolution**. DNS, or **Domain Name System**, is a hierarchical and decentralized naming system that converts human-readable domain names into machine-understandable IP addresses.

To understand DNS resolution, it is essential to explore how the query travels through different levels of the DNS infrastructure before arriving at the final IP address. This process involves several key components:

🧩 Key Components of the DNS Hierarchy

1. **Root Level Domain (RLD)**
   At the top of the DNS hierarchy is the root domain, represented by a single dot (.). Root servers do not contain actual domain names but instead respond with the locations of **Top-Level Domain (TLD)** name servers.
2. **Top Level Domain (TLD)**
   These servers handle domains like .com, .edu, .org, .net, and country-specific domains like .np, .uk, .in. For a domain like gatech.edu, the relevant TLD is .edu.
3. **Second Level Domain (SLD)**
   This is the registered domain name within a TLD—for example, gatech in gatech.edu.
4. **Fully Qualified Domain Name (FQDN)**
   A complete domain name including hostname and domain levels, such as www.gatech.edu.

🔄 DNS Resolution Process – Step by Step

1. A DNS query begins when the browser or operating system asks the **Local DNS Resolver** for the IP address of a domain like gatech.edu.
2. If the resolver has the answer in its **cache**, it immediately returns the IP address.
3. If not cached, the resolver sends the query to a **Root DNS Server**, which directs it to the relevant TLD server (.edu in this case).
4. The **TLD DNS Server** responds with the name and address of the **Authoritative DNS Server** for the domain (gatech.edu).
5. The **Authoritative Server** responds with the final **A Record** (IPv4 address) or **AAAA Record** (IPv6 address) for the domain.
6. This IP address is returned to the resolver, and eventually to the client. It is also **cached** by both the resolver and the client for future use, based on the **TTL (Time to Live)** value defined in the DNS records.

📁 Common DNS Record Types

DNS (Domain Name System) uses various types of records to store and serve information about domains and how they should be handled. These records are stored in a DNS zone file and are critical for mapping domain names to their respective services, like websites, email servers, or network locations.

---

1. A Record (Address Record)

- **Purpose**: Maps a domain name to an **IPv4 address**.
- **Use Case**: When someone types example.com, the A record provides the IP address (like 192.0.2.1) where the website is hosted.

Example:

```
example.com.   IN   A   192.0.2.1
```

Explanation:
This tells DNS that example.com should resolve to the IP address 192.0.2.1. IPv4 addresses are 32-bit, written as four decimal numbers separated by dots.

---

2. AAAA Record (IPv6 Address Record)

- **Purpose**: Maps a domain name to an **IPv6 address**.
- **Use Case**: Similar to A records, but used for newer IPv6 addresses (which are 128-bit).

Example:

```
example.com.   IN   AAAA   2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

Explanation:
This tells DNS that example.com can be reached at the IPv6 address 2001:db8::7334. IPv6 provides a vastly larger address space than IPv4.

---

3. CNAME Record (Canonical Name Record)

- **Purpose**: Creates an **alias** of one domain to another.
- **Use Case**: Used to point one domain (like www.example.com) to another (like example.com), simplifying management.

Example:

```
www.example.com.   IN   CNAME   example.com.
```

Explanation:
This means that www.example.com will use the same DNS records (especially A or AAAA) as example.com. It's useful when multiple subdomains should resolve to the same server.

Note: A CNAME must always point to a domain name, not an IP address, and cannot coexist with other records on the same name.

---

4. MX Record (Mail Exchange Record)

- **Purpose**: Specifies **mail servers** responsible for receiving emails for a domain.
- **Use Case**: Directs email traffic to the correct mail server.

Example:

```
example.com.    IN    MX    10 mail1.example.com.
example.com.    IN    MX    20 mail2.example.com.
```

Explanation:
These records say that emails for @example.com should first try to be delivered to mail1.example.com (priority 10), and if that fails, try mail2.example.com (priority 20). Lower numbers indicate higher priority.

---

5. NS Record (Name Server Record)

- **Purpose**: Specifies the **authoritative name servers** for a domain.
- **Use Case**: Tells the DNS system which servers are responsible for managing DNS records for a domain.

Example:

```
example.com.    IN    NS    ns1.examplehost.com.
example.com.    IN    NS    ns2.examplehost.com.
```

Explanation:
This means that ns1.examplehost.com and ns2.examplehost.com are authoritative for example.com, and any changes to the DNS records must be made on these servers.

---

6. PTR Record (Pointer Record)

- **Purpose**: Used for **reverse DNS lookups** — mapping an IP address back to a domain name.
- **Use Case**: Mainly used in server verification, email validation, and diagnostics.

Example:

```
1.2.0.192.in-addr.arpa.   IN   PTR   example.com.
```

Explanation:
This means the IP address 192.0.2.1 resolves **back** to the domain example.com. It's the reverse of an A record and is stored in a special domain (in-addr.arpa) for reverse mapping.

🔁 Summary Table

Record Type	Purpose	Maps To	Example Value
A	Domain to IPv4 address	IPv4	192.0.2.1
AAAA	Domain to IPv6 address	IPv6	2001:db8::7334
CNAME	Domain alias to another domain	Domain	example.com.
MX	Email routing	Mail server (domain)	mail1.example.com.
NS	Delegation to authoritative server	Name server domain	ns1.examplehost.com.
PTR	IP to domain (reverse lookup)	Hostname	example.com.

🔧 Demonstrating DNS in Linux

To analyze and troubleshoot DNS queries on a Linux system, several powerful command-line tools are available. Two of the most informative are dig and traceroute.

---

1. dig – Domain Information Groper

The dig command is used to query DNS name servers and display the response details. It can retrieve various record types (A, NS, MX, etc.).

Example:

```
dig +trace gatech.edu
```

- The +trace option enables **recursive tracing**, which reveals each step of the DNS resolution starting from the **root servers**, down to the **authoritative servers**.
- This command simulates the full DNS resolution process, helping to visualize how a DNS query is delegated from one level to the next.

Output Highlights:

- First, root servers (a.root-servers.net, b.root-servers.net, etc.) return the .edu TLD servers.
- Then, .edu servers (a.edu-servers.net, b.edu-servers.net, etc.) provide the authoritative name servers for gatech.edu.
- Finally, dns1.gatech.edu, dns2.gatech.edu, etc., respond with the final **A record**, e.g., 3.214.16.8.

Each DNS response includes a **TTL value**, defining how long this data is valid for caching.

---

2. traceroute – Trace the Network Path

While dig focuses on domain name resolution, traceroute maps the **network path** between your computer and a target server.

Example:

```
traceroute gatech.edu
```

- This tool shows how a packet travels through various routers (hops) to reach the destination.
- It displays IP addresses of intermediate routers and the time it takes for packets to travel through each hop.
- It is valuable for identifying **latency**, **routing issues**, or **packet loss** during DNS-related connectivity problems.

4.4 Implementing Servers

4.4.1 What is a Linux Server?

A Linux server is a computer system running a Linux operating system that provides services or applications over a network. It can be hosted physically or virtually and is commonly used for web hosting, file sharing, databases, cloud services, and more. Linux's open-source nature and resource efficiency make it a popular choice for server environments.

4.4.2 Advantages of Linux Servers

1. **Free & Cost-Effective:** No licensing fees.
2. **Stable & Reliable:** Ideal for high-uptime environments.
3. **Customizable:** Wide range of distributions and configurable components.
4. **Secure:** Regular updates, strong access control, and fewer malware threats.

5. **Scalable:** Handles everything from small setups to enterprise workloads.
6. **High Performance:** Efficient on both low-end and high-end hardware.
7. **Strong Community Support:** Extensive documentation and forums.

4.4.3 Common Use Cases

- **Web Hosting:** Apache, Nginx for sites, CMSs like WordPress.
- **Database Management:** Hosts databases like MySQL, PostgreSQL.
- **Application Hosting:** Email, ERP, CRM, and collaboration tools.
- **Cloud Infrastructure:** Powers AWS, Azure, and private clouds.
- **DevOps & CI/CD:** Used for automation with Jenkins, Docker, Kubernetes.
- **Networking & Security:** Acts as firewall, VPN, NAS.
- **Virtualization/Containers:** Supports KVM, Docker, LXC.
- **Big Data:** Runs frameworks like Hadoop, Spark.
- **Educational & Development:** Used in labs for training and coding.
- **IoT/Embedded Systems:** Powers smart devices and automation.

4.4.4 Security & Performance Considerations

Security:

- **User Access:** Limit root access, use sudo and strong passwords.
- **Firewall:** Use iptables/firewalld to allow only necessary ports.
- **Updates:** Regularly patch OS and software using apt/yum.
- **SSH:** Disable root login, use key-based authentication.
- **Filesystem:** Encrypt sensitive data, monitor integrity with tools like AIDE.

Performance:

- **Resource Monitoring:** Use top, htop, vmstat.
- **Tuning:** Optimize system settings and kernel parameters.
- **Caching:** Use Redis, Memcached for speed.
- **Load Balancing:** Use Nginx, HAProxy to manage traffic.
- **Filesystem Choices:** Select optimal filesystems (ext4, XFS) based on workload.

4.4.5 Six Key Steps to Set Up a Linux Server

Setting up a Linux server involves a series of essential steps to ensure security, performance, and reliability. These six key steps provide a foundational checklist for deploying a new Linux server, applicable across most Linux distributions:

Step 1: User and Network Configuration

- **Secure the Root Account:** Change the root password and follow strong password policies (minimum 8 characters with uppercase, lowercase, numbers, and symbols).
- **Create User Accounts:** Disable direct root login and set up non-privileged users with sudo access.
- **Configure Network Settings:** Assign a static IP address, hostname, domain, and DNS information. Disable IPv6 if not in use. Place the server appropriately within VLAN or network segments.

Step 2: Package Management and System Updates

- **Install Required Packages:** Use the distribution's package manager (e.g., apt, yum, or dnf) to install only necessary software.
- **Remove Unnecessary Packages:** Reduce the system's attack surface and footprint by removing unused packages.
- **Update the System:** Ensure the OS and all installed software are fully updated, including the Linux kernel.
- **Enable Automatic Updates:** If appropriate for your use case, configure automatic security and package updates.

Step 3: NTP (Network Time Protocol) Configuration

- **Sync System Time:** Configure the server to synchronize time using internal or external NTP servers.
- **Avoid Clock Drift:** Proper time synchronization is critical for logging, security, and authentication services.

Step 4: Firewall and Port Configuration

- **Set Up Firewalls:** Use tools like iptables, firewalld, or ufw to configure firewall rules.

- **Allow Essential Services Only:** Open only the necessary ports required for the server's role (e.g., HTTP, SSH).
- **Review Default Rules:** Ensure the default firewall policy is restrictive and adjusted as needed.

Step 5: Secure SSH and Manage Services

- **Harden SSH Access:**
  - Disable root login over SSH.
  - Change the default SSH port to a non-standard port.
  - Use SSH key-based authentication instead of passwords.
- **Control Services (Daemons):**
  - Disable unneeded services.
  - Configure essential services to start automatically at boot.

Step 6: Enable SELinux, Hardening, and Logging

- **Enable SELinux (on RHEL-based systems):** Provides mandatory access control. Test applications under SELinux policies to ensure compatibility.
- **Harden Applications:** Follow best practices for securing services like Apache, MySQL, etc.
- **Set Up Logging:** Ensure logging is configured for all critical services. Use tools for log management, monitoring, and alerting (e.g., rsyslog, Logwatch, ELK Stack).

4.5 How to setup and configure an FTP server in Linux?

An **FTP (File Transfer Protocol)** server enables the transfer of files between a server and clients over a network. It allows users to upload, download, and manage files remotely. Common use cases include hosting a **public FTP server** for downloads or setting up a **private FTP server** for internal file transfer within an organization. While FTP is widely used for its simplicity, it is important to secure it properly—especially if exposed over the internet—by using SSL/TLS encryption to protect credentials and data.

In Red Hat Enterprise Linux (RHEL), we typically use **vsftpd (Very Secure FTP Daemon)**, a fast and secure FTP server. Below are the step-by-step instructions to install and configure vsftpd on a RHEL system.

4.5.2 FTP Server Configuration Steps on RHEL

Step 1: Install vsftpd

```
sudo dnf install vsftpd -y
sudo systemctl enable --now vsftpd
sudo systemctl status vsftpd
```

Step 2: Open Required Firewall Ports

```
sudo firewall-cmd --permanent --add-port=20-21/tcp
sudo firewall-cmd --permanent --add-port=990/tcp
sudo firewall-cmd --permanent --add-port=5000-10000/tcp
sudo firewall-cmd --reload
```

Step 3: Create FTP User

```
sudo adduser ftpuser
sudo passwd ftpuser
```

To disable SSH access:

```
echo "DenyUsers ftpuser" | sudo tee -a /etc/ssh/sshd_config
sudo systemctl restart sshd
```

Step 4: Create FTP Directory and Set Permissions

```
sudo mkdir /ftp
sudo chown youradminuser:youradminuser /ftp
```

Step 5: Configure vsftpd

Edit config file:

```
sudo vi /etc/vsftpd/vsftpd.conf
```

Ensure these settings are present or uncommented:

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=0002
local_root=/ftp

pasv_enable=YES
pasv_min_port=5000
```

```
pasv_max_port=10000


chroot_local_user=YES
chroot_list_enable=YES
chroot_list_file=/etc/vsftpd/chroot_list
allow_writeable_chroot=YES
```

Create and edit chroot list:

```
sudo touch /etc/vsftpd/chroot_list
echo "youradminuser" | sudo tee -a /etc/vsftpd/chroot_list
```

Step 6: Enable SSL/TLS Encryption

Generate a self-signed certificate:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/vsftpd.pem -out
/etc/ssl/private/vsftpd.pem
```

Update the config file:

```
sudo vi /etc/vsftpd/vsftpd.conf
```

Add at the end:

```
rsa_cert_file=/etc/ssl/private/vsftpd.pem
rsa_private_key_file=/etc/ssl/private/vsftpd.pem
ssl_enable=YES
allow_anon_ssl=NO
force_local_data_ssl=YES
force_local_logins_ssl=YES
ssl_tlsv1=YES
ssl_sslv2=NO
ssl_sslv3=NO
require_ssl_reuse=NO
ssl_ciphers=HIGH
```

Restart the service:

```
sudo systemctl restart vsftpd
```

Step 7: Connect Using FTP Client

Use **FileZilla** or another FTP client. Enter the server's IP, username, password, and port (21 for FTP, 990 for FTPS). You can now upload/download files securely.

4.5.1 FTP server commands

You can also connect to your FTP server on the terminal and operate it with FTP commands. A list of a few of them is given below.

Command	Function
pwd	print the current working directory
cwd	change working directory
dele	delete the specified file
cdup	change to the parent directory
help	displays help information
cd	change the working directory
get filename	download the specified file
put filename	uploads the specified file
bye	end FTP session

4.6 NFS and Samba in Linux

Network File System (NFS) and Samba are two of the most widely used protocols for sharing files over a network in a Linux environment. This guide will provide an overview of each, explain their differences, and show you how to set them up.

4.6.1 What is NFS?

In modern enterprise or collaborative development environments, **sharing files across multiple systems** is essential. Imagine a scenario where two Linux servers—let's call them **Server A** and **Server B**—are on the same network. We want Server A to **access and modify** a directory (/server/apps) located in Server B, as if the directory were local to Server A.

To achieve this, we use **NFS (Network File System)**—a distributed file system protocol that allows users on a client system to access files over a network much like they access local storage. NFS is especially helpful in cases like:

- Shared development environments
- Centralized application deployment
- Multi-node systems needing consistent access to shared files
- Lightweight backup and syncing systems

Let's now walk through how to **configure NFS on RHEL**, both on the **server-side (file provider)** and **client-side (file consumer)**.

Part 1: Server-Side Configuration (Server B)

✅ **Step 1: Install NFS Packages**

Install necessary NFS utilities:

```
sudo dnf install nfs-utils nfs4-acl-tools -y
```

✅ **Step 2: Enable and Start Required Services**

Start and enable NFS services:

```
sudo systemctl start nfs-server rpcbind
sudo systemctl enable nfs-server rpcbind
```

You may also start these for user ID mapping:

```
sudo systemctl start nfs-idmapd
sudo systemctl enable nfs-idmapd
```

✅ **Step 3: Create and Prepare the Shared Directory**

```
sudo mkdir -p /server/apps
sudo chmod -R 777 /server/apps
```

You can give stricter permissions based on your use-case. 777 is used here for simplicity and testing.

✅ **Step 4: Export the Directory**

Edit the NFS exports file:

```
sudo nano /etc/exports
```

Add this line (replace * with a specific IP range for better security):

```
/server/apps *(rw,sync,no_root_squash)
```

Save and exit, then export the shares:

```
sudo exportfs -r
```

✅ **Step 5: Allow NFS Through the Firewall (optional but recommended)**

```
sudo firewall-cmd --permanent --add-service=nfs
sudo firewall-cmd --reload
```

Or disable the firewall temporarily (not recommended for production):

```
sudo systemctl stop firewalld
```

Part 2: Client-Side Configuration (Server A)

✅ **Step 1: Install NFS Client Utilities**

```
sudo dnf install nfs-utils -y
```

✅ **Step 2: Start RPC Bind (if not already running)**

```
sudo systemctl start rpcbind
```

(Optional) Disable firewall if needed for testing:

```
sudo systemctl stop firewalld
```

✅ **Step 3: Create Mount Point**

```
sudo mkdir -p /mnt/apps
```

✅ **Step 4: Mount the NFS Share**

Assuming server B's IP is 192.168.1.100:

```
sudo mount 192.168.1.100:/server/apps /mnt/apps
```

✅ **Step 5: Verify the Mount**

Create a file:

```
cd /mnt/apps
touch testfile_from_client.txt
```

Now check on the server (/server/apps)—you'll see the same file there.

(Optional) Step 6: Permanent Mount via /etc/fstab

To mount the share permanently even after reboot, add this line to /etc/fstab on the client:

```
192.168.1.100:/server/apps  /mnt/apps  nfs  defaults  0  0
```

Use Case Recap

Using NFS, your server-side directory is **accessible and editable** in real time by the client as though it's local. This setup is particularly useful for:

- Multi-user Linux environments
- Shared web server content
- Centralized logging or data collection
- Simplified backups or CI/CD pipelines

4.6.2 What is Samba?

In multi-platform environments, seamless file sharing between Linux and Windows systems is a common requirement. For example, a company might operate several Linux servers for backend services while using Windows workstations for day-to-day operations. In such scenarios, **Samba** serves as a bridge, enabling file and printer sharing across these platforms using the **SMB (Server Message Block)** or **CIFS (Common Internet File System)** protocols. Samba is particularly helpful for:

- Sharing folders between a RHEL Linux server and Windows PCs.
- Mounting shared folders from Linux to Linux.
- Managing access with authentication and security controls.

Samba Server-Side Configuration on RHEL

Step 1: Install Required Packages

Install Samba and related tools:

```
sudo dnf install samba samba-client samba-common -y
```

Step 2: Create Shared Directory

Create a directory that you want to share:

```
sudo mkdir -p /samba/apps
sudo touch /samba/apps/sample.txt
```

Step 3: Set Permissions

Give necessary permissions:

```
sudo chmod -R 777 /samba/apps
```

Step 4: Configure SELinux (if enabled)

Allow the directory to be accessed over the network:

```
sudo chcon -t samba_share_t /samba/apps -R
```

Step 5: Configure Samba (smb.conf)

Edit the Samba configuration file:

```
sudo nano /etc/samba/smb.conf
```

Add or modify these sections:

```
[global]
   workgroup = WORKGROUP
```

```
   server string = Samba Server %v
   netbios name = rhelserver
   security = user
   map to guest = Bad User
   dns proxy = no

[apps]
   path = /samba/apps
   writable = yes
   browsable = yes
   guest ok = yes
   read only = no
```

Save and exit the file.

Step 6: Enable Firewall for Samba

If firewalld is active:

```
sudo firewall-cmd --permanent --zone=public --add-service=samba
sudo firewall-cmd --reload
```

Step 7: Enable and Start Samba Services

```
sudo systemctl enable smb nmb
sudo systemctl start smb nmb
```

Check service status:

```
sudo systemctl status smb
sudo systemctl status nmb
```

Client-Side Configuration

1. From Windows Client

Step 1: Open File Explorer

In the address bar, enter:

```
\\<Linux-Server-IP>\apps
```

Step 2: Access Shared Folder

If guest ok = yes is enabled, it will open directly. Otherwise, enter Samba credentials if prompted.

Step 3: Test File Operations

Create a file from Windows and verify it appears on the Linux side and vice versa.

---

2. From Linux Client (Another RHEL Machine)

Step 1: Install CIFS Utilities

```
sudo dnf install cifs-utils -y
```

Step 2: Create a Mount Point

```
sudo mkdir -p /mnt/samba/apps
```

Step 3: Mount the Share

```
sudo mount -t cifs //192.168.x.x/apps /mnt/samba/apps -o guest
```

Replace 192.168.x.x with your Samba server IP.

To mount using credentials:

```
sudo mount -t cifs //192.168.x.x/apps /mnt/samba/apps -o
username=<samba_user>,password=<password>
```

Step 4: Test

Read/write files to `/mnt/samba/apps` to confirm it's working.

Optional: Securing the Samba Share with Authentication

Step 1: Create a Samba Group and User

```
sudo groupadd smbgrp

sudo useradd -M -d /samba/secure -s /sbin/nologin -G smbgrp
testuser

sudo passwd testuser
```

Step 2: Add the User to Samba

```
sudo smbpasswd -a testuser

sudo smbpasswd -e testuser
```

Step 3: Create a Secured Directory

```
sudo mkdir -p /samba/secure

sudo chown testuser:smbgrp /samba/secure
```

```
sudo chmod 2770 /samba/secure
sudo chcon -t samba_share_t /samba/secure -R
```

Step 4: Update smb.conf

Add this block:

```
[secure]
   path = /samba/secure
   valid users = @smbgrp
   guest ok = no
   writable = yes
   browsable = yes
```

Step 5: Restart Samba

```
sudo systemctl restart smb nmb
```

Step 6: Access from Windows

Access via:

```
\\<Linux-Server-IP>\secure
```

Enter testuser and the password.

Verification & Maintenance

- Use testparm to verify configuration:

```
testparm
```

- To check mounted shares on Linux client:

```
df -h | grep samba
```

This configuration allows you to:

- Share files from RHEL to both Windows and Linux.
- Set up both open (guest) and restricted (authenticated) shares.
- Control access using Linux file permissions and Samba authentication.

NFS vs. Samba: When to Use Which?

- **NFS**: Best for Linux-to-Linux file sharing due to its seamless integration with Linux permissions and performance.

- **Samba**: Ideal for environments with both Linux and Windows systems, enabling cross-platform file sharing.

4.7 Directory Access

4.7.1 Understanding File Permissions

In Linux, every file and folder have *permissions* that decide who can access it and what they can do. Permissions are like rules for a locked box: they say who can open it, change what's inside, or use it.

4.7.2 Types of Permissions

There are three types of permissions:

- **Read (r)**: Lets you view a file's contents or see what's inside a folder.
- **Write (w)**: Lets you change a file or add/delete items in a folder.
- **Execute (x)**: Lets you run a file (like a program) or enter a folder.

4.7.3 Who Gets Permissions?

Permissions are set for three groups:

- **Owner**: The person who created the file or folder (usually you).
- **Group**: A team of users who can share access.
- **Others**: Everyone else on the computer.

4.7.4 Viewing Permissions

To see permissions, use the ls -l command in a terminal. For example:

```
$ ls -l
-rw-r--r-- 1 user group 0 Jun 18 12:00 cat.txt
drwxr-xr-x 2 user group 0 Jun 18 12:00 Pets
```

- For cat.txt:
  - `-rw-r--r--` means it's a file (-), the owner can read and write (rw-), and the group and others can only read (r--).
- For Pets:
  - `drwxr-xr-x` means it's a folder (d), the owner can read, write, and execute (rwx), and the group and others can read and execute (r-x).

This shows who can access your files and what they can do.

4.7.5 Setting and Changing Permissions

You can change permissions and ownership to control access to your files and folders. Here's how:

4.7.6 Changing Permissions with chmod

The chmod command lets you set what users can do. There are two ways to use it:

4.7.7 Symbolic Method

- Use letters: u (owner), g (group), o (others), and + (add), - (remove), = (set exactly).
- Examples:
  - `chmod u+x file.txt:` Adds execute permission for the owner.
  - `chmod g-w file.txt:` Removes write permission for the group.
  - `chmod o= file.txt:` Removes all permissions for others.

4.7.8 Numeric Method

- Each permission has a number: read = 4, write = 2, execute = 1.
- Add them up for each group:

```
7 = read (4) + write (2) + execute (1)
6 = read (4) + write (2)
5 = read (4) + execute (1)
4 = read (4)
0 = no permissions
```

- Use three numbers for owner, group, and others.
- Example: chmod 755 file.txt

```
Owner: 7 (read, write, execute)
Group: 5 (read, execute)
Others: 5 (read, execute)
```

Example

To make a file readable and writable only by the owner:

```
chmod 600 secret.txt
```

This sets owner

4.7.9 Changing Ownership

- **Change Owner with** `chown`:
  - Example: `chown newuser file.txt`
  - This makes newuser the owner of file.txt.
- **Change Group with** `chgrp:`
  - Example: `chgrp newgroup file.txt`
  - This assigns file.txt to the newgroup group.

4.7.10 Best Practice: Principle of Least Privilege

- Only give users the permissions they need.
- Example: For a shared folder, set permissions so only the owner and group can access it:

```
chmod 770 shared_folder

chgrp team shared_folder
```

This gives full access to the owner and team group, but no access to others.

4.8 Secure File Sharing Over a Network

Sharing files with someone on another computer requires extra care to keep your data safe. Unprotected sharing is like sending a postcard—anyone can read it. Secure sharing is like sending a locked box that only the recipient can open.

4.8.1 Why Security Matters

- Files sent over a network can be intercepted if not encrypted.
- Sensitive data (e.g., passwords, personal info) must be protected.

4.8.2 Secure File Sharing Methods

Here are beginner-friendly ways to share files securely:

1. SFTP (SSH File Transfer Protocol)

- Uses SSH to encrypt files during transfer.
- Safe for sensitive data.
- You can use an SFTP client like FileZilla or the sftp command.

2. SCP (Secure Copy)

- Also uses SSH to copy files between computers.
- Simple command-line tool.
- Example:

```
scp myfile.txt user@remotehost:/path/to/directory/
```

3. Rsync over SSH

- Efficiently transfers and syncs files using SSH.
- Example:

```
rsync -avzh myfile.txt user@remotehost:/path/to/directory/
```

These methods are secure because they encrypt data, making it unreadable to anyone without the key.

Example: Sharing a File with SCP

Let's say you want to share important.doc with a colleague on a remote server.

Steps

1. Open a terminal.
2. Run:

```
scp important.doc user@remotehost:/home/user/shared/
```

   - Replace user with your username on the remote server.
   - Replace remotehost with the server's address (e.g., 192.168.1.100 or example.com).
   - Replace /home/user/shared/ with the destination path.
3. Enter your password when prompted (or use SSH keys for passwordless access).
4. The file is copied securely to the server.

Your colleague can now access important.doc from the server.

Other Methods (For Future Learning)

- **Samba**: Shares files with Windows or Linux systems but requires careful setup for security.
- **NFS (Network File System)**: Common for Linux-to-Linux sharing but needs secure configuration. These are more complex and best learned after mastering SFTP and SCP.

4.8.3 Best Practices for Secure File Sharing and Directory Access

To keep your files safe, follow these simple tips:

- **Use Encrypted Protocols**: Always use SFTP, SCP, or Rsync over SSH for network transfers. Avoid plain FTP, as it's not secure.
- **Limit Access**:
  - Set permissions to allow only necessary users or groups.
  - Example: chmod 770 shared_dir for owner and group access only.
- **Use Strong Passwords**:
  - For SSH access, use complex passwords or set up key-based authentication.
- **Encrypt Sensitive Data**:
  - Consider encrypting files before sharing if they contain sensitive information.
- **Regularly Check Permissions**:
  - Use ls -l to verify who has access to your files and folders.
- **Be Cautious**:
  - Only share files with trusted people.
  - Avoid sharing sensitive files unless necessary.

4.8.4 Table: Common Secure File Sharing Methods

Method	Description	Example Command	Security Level
SFTP	Transfers files using SSH encryption	sftp user@remotehost	High
SCP	Copies files using SSH encryption	scp file.txt user@remotehost:/path/	High
Rsync over SSH	Syncs files efficiently using SSH	rsync -avzh file.txt user@remotehost:/path/	High
Samba	Shares files with Windows/Linux (complex)	Requires server setup	Medium (if configured properly)
NFS	Linux-to-Linux sharing (complex)	Requires server setup	Medium (if configured properly)

4.8.5 Practice Exercise

Try these steps to practice what you've learned:

1. Create a folder called shared:

```
mkdir shared
```

2. Create a file inside it called test.txt:

```
touch shared/test.txt
```

3. Check the permissions:

```
ls -l shared/test.txt
```

4. Set permissions so only the owner can read and write:

```
chmod 600 shared/test.txt
```

5. Verify the permissions:

```
ls -l shared/test.txt
```

6. (Optional) If you have access to another Linux machine, copy test.txt using SCP:

```
scp shared/test.txt user@remotehost:/path/to/directory/
```

Unit 5: Web, Email, Database Services, and Network Security

5.1 Apache & HTTPD

Apache HTTP Server, commonly referred to as Apache, is one of the most popular and widely used open-source web servers in the world. It was developed and maintained by the Apache Software Foundation. Apache enables a computer to host one or more websites that can be accessed over the Internet using a web browser.

The Apache web server is a reliable and flexible solution for hosting websites and applications on Red Hat Enterprise Linux (RHEL). This guide provides step-by-step instructions to install and configure Apache on RHEL, including testing a simple website and uninstallation steps.

5.1.1 Features of Apache:

1. **Open Source:** Freely available with a large community.
2. **Cross-platform:** Runs on Unix/Linux, Windows, and macOS.
3. **Modular Architecture:** Supports dynamically loadable modules like mod_ssl, mod_rewrite, mod_php.
4. **Virtual Hosting:** Can serve multiple websites from a single server.
5. **Authentication and Authorization:** Controls user access to content.
6. **Logging and Custom Error Messages:** Logs requests and supports customizable error pages.

5.1.2 HTTPD (HTTP Daemon)

The HTTP daemon (httpd) is the software process that runs in the background and serves web content. Apache HTTP Server's binary is often called httpd. It listens for incoming HTTP requests and serves web pages in response.

5.1.3 Configuration Files:

- Main config file: `/etc/httpd/conf/httpd.conf` or `etc/apache2/apache2.conf`
- Document root: `/var/www/html`

Apache also supports SSL/TLS configuration for HTTPS, and URL rewriting for SEO-friendly URLs using `.htaccess`.

5.1.4 How to Install Apache Web Server on RHEL

Step 2: Update Your System

Ensure your system is up-to-date to avoid compatibility issues.

Command:

```
sudo yum update -y
```

This updates all packages on your RHEL system.

Step 3: Install Apache Web Server

Install the Apache web server package (httpd).

Command:

```
sudo yum install httpd -y
```

This installs Apache and its dependencies.

Step 4: Enable the Services

Enable and start the Apache service to ensure it runs on boot and is active.

Command:

```
sudo systemctl enable httpd.service
sudo systemctl start httpd.service
```

Verify the service status.

Command:

```
sudo systemctl status httpd.service
```

You should see an **active (running)** status.

Step 5: Test the Server by Hosting a Simple Website

Create a test website to verify Apache is working.

1. Create a directory for the test website.

Command:

```
sudo mkdir /var/www/html/test_website
```

2. Create an index.html file with sample content.

```
sudo vi /var/www/html/test_website/index.html
```

Command:

```
<html>
<head>
```

```
 <title>Example</title>
</head>
<body>
     <h1>Welcome to RHEL Apache</h1>
     <p>This is a test.</p>
</body>
</html>
```

3. Create a configuration file for the virtual host.

Command:

Using nano:

```
sudo nano /etc/httpd/conf.d/web.conf
```

Step 2: Add the Virtual Host Configuration

Paste the following content into the file:

```
<VirtualHost *:80>
    ServerName web.testingserver.com
    DocumentRoot /var/www/html/test_website
    DirectoryIndex index.html
    ErrorLog /var/log/httpd/test_website_error.log
    CustomLog /var/log/httpd/test_website_requests.log
combined
</VirtualHost>
```

4. Set proper ownership and permissions for the website directory.

Command:

```
sudo chown -R apache:apache /var/www/html/test_website
sudo chmod -R 755 /var/www/html/test_website
```

5. Restart Apache to apply the configuration.

Command:

```
sudo systemctl restart httpd.service
```

6. Test the website by accessing http://localhost/test_website or your server's IP address in a browser. You should see the test page with "Welcome to RHEL Apache" and the test message.

How to Uninstall Apache Web Server

If needed, remove Apache from your RHEL system.

Command:

```
sudo yum remove httpd -y
```

This removes the Apache web server and its associated files.

5.2 How to Set Up a Mail Server with Postfix and Dovecot on RHEL

Setting up a mail server on Red Hat Enterprise Linux (RHEL) is essential for small businesses or personal websites needing reliable email services. This guide details how to configure a mail server using **Postfix** and **Dovecot** on RHEL, enabling secure sending and receiving of emails.

5.2.1 What is Postfix on RHEL?

Postfix is a **Mail Transfer Agent (MTA)** on RHEL that handles email sending and routing using the **Simple Mail Transfer Protocol (SMTP)**. It manages outbound email traffic, ensuring emails are delivered to the recipient's server efficiently.

5.2.2 Features of Postfix:

- Filters outgoing emails to specific servers or recipients.
- Supports customized filters for email management.
- Relays emails between servers.
- Receives emails from clients or servers.

5.2.3 What is Dovecot on RHEL?

Dovecot complements Postfix by handling email reception on RHEL. It uses **Internet Message Access Protocol (IMAP)** and **Post Office Protocol (POP3)** to securely deliver and store incoming emails on the server.

5.2.4 Features of Dovecot:

- Allows email access from multiple devices.
- Enables checking emails without downloading to the local machine.
- Supports deleting emails from the server after downloading.
- Ensures emails are securely stored on the server.

5.2.5 Prerequisites to Set Up Mail Server with Postfix and Dovecot:

- A fully qualified domain name (FQDN) server running RHEL.
- Basic knowledge of DNS server operations.
- Root or sudo access to the RHEL server.
- A subscribed RHEL system with active repositories (e.g., BaseOS and AppStream).

📬 Step-by-Step: Setting Up Postfix on RHEL (Gmail SMTP Relay)

✅ 1. Install Postfix & Mailx

First, make sure your system is up to date and connected to the internet. Then run:

```
sudo dnf install postfix mailx -y
```

This installs both Postfix (for sending mail) and Mailx (a simple command-line mail client to test it).

✅ 2. Configure Postfix to Use Gmail SMTP

Open the Postfix config file:

```
sudo nano /etc/postfix/main.cf
```

Add or update these lines:

```
relayhost = [smtp.gmail.com]:587
myhostname = <your-hostname>  # Replace with your actual hostname
inet_interfaces = all
inet_protocols = all
smtp_sasl_password_maps = hash:/etc/postfix/sasl/sasl_passwd
smtp_sasl_auth_enable = yes
smtp_tls_security_level = encrypt
smtp_sasl_security_options = noanonymous
```

To find your hostname:

```
hostname -f
```

✅ 3. Set Up Gmail Authentication

Create the sasl_passwd file:

```
sudo mkdir -p /etc/postfix/sasl/
sudo nano /etc/postfix/sasl/sasl_passwd
```

Add the following line (replace with your Gmail and app-specific password):

```
[smtp.gmail.com]:587 yourgmail@gmail.com:yourapppassword
```

Now **secure the file**:

```
sudo chown root:root /etc/postfix/sasl/sasl_passwd
sudo chmod 600 /etc/postfix/sasl/sasl_passwd
```

Generate the Postfix hash map:

```
sudo postmap /etc/postfix/sasl/sasl_passwd
```

✅ 4. Enable and Start Postfix

```
sudo systemctl enable postfix
sudo systemctl start postfix
```

To check status:

```
sudo systemctl status postfix
```

✅ 5. Test Sending Emails

Send a basic email:

```
echo "This is a test mail" | mail -s "Postfix Test"
recipient@example.com
```

Send with an attachment:

```
echo "Message body here" | mail -s "With Attachment" -a
/path/to/file.pdf recipient@example.com
```

📥 What is Dovecot and Why Do You Need It?

Postfix handles **sending** emails. But to **receive** emails (like using IMAP or POP3), you need a Mail Delivery Agent (MDA) like **Dovecot**.

Dovecot's role:

- Provides **IMAP** and **POP3** access so users can fetch their emails using apps like Thunderbird or Outlook.
- Works alongside Postfix to deliver incoming mail securely and efficiently.

📦 Setting Up Dovecot on RHEL

✅ 1. Install Dovecot

```
sudo dnf install dovecot -y
```

✅ 2. Enable Basic Protocols

Open the main Dovecot config:

```
sudo nano /etc/dovecot/dovecot.conf
```

Make sure the following lines are there (add if missing):

```
protocols = imap pop3
!include_try local.conf
```

Save and close the file.

✅ 3. Enable and Restart Dovecot

```
sudo systemctl enable dovecot
sudo systemctl restart dovecot
```

Check status:

```
sudo systemctl status dovecot
```

It should show active (running).

✅ 4. Check the Mail

Log into Computer as testuser or switch:

```
su - testuser
```

```
mail
```

Q. Explain the email operation process on both the sender's and receiver's side.

Email Operation Process: Sender Side (Outbound Mail with Postfix)

1. Composing and Sending the Email

- The user composes an email using a mail client (like Thunderbird, Outlook, or even mail from the terminal).
- When the email is sent, the client hands it off to the **Mail Transfer Agent (MTA)** — in this case, **Postfix**.

2. Postfix Accepts the Mail

- Postfix receives the message through **SMTP** (Simple Mail Transfer Protocol).
- It examines the recipient's address and decides how to route it.

3. Authentication and Relay

- If Postfix is configured to relay through **Gmail SMTP**, it connects to smtp.gmail.com using **TLS encryption** on port 587.
- It authenticates using credentials defined in /etc/postfix/sasl/sasl_passwd.

4. Outbound Delivery

- Once authenticated, Postfix relays the email to Gmail's SMTP server.
- Gmail's server then takes over and forwards the message to the **recipient's mail server**, using DNS MX records to figure out where to send it.

Email Operation Process: Receiver Side (Inbound Mail with Dovecot)

1. Mail Arrives at Recipient Server

- The recipient's MTA (could be another Postfix server or something else) receives the mail.
- It performs checks like spam filtering, DKIM/DMARC validation, and then passes the message to the **Mail Delivery Agent (MDA)** — in this setup, Dovecot.

2. Dovecot Delivers the Email

- Dovecot stores the email in the recipient's **mailbox directory** (usually under /var/mail/username or /home/username/Maildir).
- Dovecot doesn't send the mail — it **makes it available** to the user.

3. User Accesses the Mail

- The user opens a mail client and connects to the server using **IMAP** or **POP3**, both provided by Dovecot:
    - **IMAP** lets users access and manages emails **on the server**, syncing across devices.
    - **POP3** downloads emails to the client and may delete them from the server.

4. Secure Retrieval

- Dovecot handles authentication and encryption (SSL/TLS), ensuring only the right user accesses their mailbox.

Q. What are the major email communication protocols?

There are three major protocols that govern how email is sent, received, and accessed over the internet:

---

1. SMTP (Simple Mail Transfer Protocol)

- **Purpose**: Used to **send** emails from a client to a server or between mail servers.
- **Port Numbers**:
    - Port **25** (default, often blocked by ISPs)
    - Port **587** (secure submission from client to server)
    - Port **465** (SMTP over SSL, deprecated but still used)
- **Role in Email**:
    - Outgoing mail only.
    - Used by MTAs like **Postfix** to relay messages to recipients' servers.

---

2. IMAP (Internet Message Access Protocol)

- **Purpose**: Used to **retrieve and manage emails** stored on a mail server.
- **Port Numbers**:
    - Port **143** (default)
    - Port **993** (IMAP over SSL/TLS)
- **Role in Email**:

- Lets users access their email from multiple devices.
- Email stays on the server unless deleted.

---

3. POP3 (Post Office Protocol version 3)

- **Purpose**: Used to **download** emails from the server to a local device.
- **Port Numbers**:
  - Port **110** (default)
  - Port **995** (POP3 over SSL)
- **Role in Email**:
  - Once emails are downloaded, they're usually deleted from the server.
  - Suitable for single-device access.

Summary Table:

Protocol	Purpose	Direction	Ports	Usage Type
SMTP	Send email	Outgoing only	25, 587	Server to Server / Client to Server
IMAP	Access email	Incoming	143, 993	Multi-device sync
POP3	Download email	Incoming	110, 995	One-device storage

5.3 MySQL Administration

MySQL is an open-source relational database management system that is based on SQL queries. Here, **"My" represents the name of the co-founder Michael Widenius's daughter** and **"SQL" represents the Structured Query Language**. MySQL is used for data operations like **querying, filtering, sorting, grouping, modifying, and joining** the tables present in the database.

In this article, we will be going to download and install MySQL on Linux and will verify MySQL installation by creating a database. But Before that Let's see some features of MySQL.

5.3.1 Feature of MySQL

MySQL is the most popular RDBMS, It offers various features like:

- It is **easy to use and free of cost** to download.
- It contains a solid **data security** layer to protect important data.

- It is based on **client and server architecture.**
- It supports **multithreading** which makes it more scalable.
- It is **highly flexible** and **supported by multiple applications**.
- MySQL is **fast, efficient, and reliable.**
- It is **compatible** with many operating systems like **Windows, MacOS, Linux**, etc.

5.3.2 Installing MySQL on RHEL

Step 1: Enable the MySQL Repository

RHEL does not include MySQL in its default repositories, so you need to enable the official MySQL repository.

1. Download the MySQL Yum repository package.

Command:

```
sudo yum install -y https://dev.mysql.com/get/mysql80-community-release-el8-5.noarch.rpm
```

OR,

Download the MySQL Yum repository package from the official website

And then install locally

```
sudo yum localinstall mysql80-community-release-el8-5.noarch.rpm
```

2. Enable the MySQL 8.0 repository.

Command:

```
sudo yum repolist all | grep mysql
```

Disable the default MySQL module

Run this command:

```
dnf module disable mysql -y
```

Then clean up metadata:

```
dnf clean all
dnf makecache
```

Step 2: Install MySQL Server

Install the MySQL server package.

Command:

```
sudo yum install mysql-community-server -y
```

Enter y when prompted to confirm the installation. This downloads and installs MySQL and its dependencies, which may take a few minutes.

Step 3: Start and Enable MySQL Service

Start the MySQL service and enable it to run on boot.

Command:

```
sudo systemctl start mysqld
sudo systemctl enable mysqld
```

5.3.3 Verifying MySQL Installation

Step 4: Check MySQL Version

Verify the installation and check the MySQL version.

Command:

```
mysql --version
```

This should display the installed MySQL version (e.g., mysql Ver 8.0.XX).

Step 5: Retrieve Temporary Root Password

After installation, MySQL generates a temporary root password, which is stored in the MySQL log file.

Command:

```
sudo grep 'temporary password' /var/log/mysqld.log
```

Note the temporary password for the next step.

Protecting and Securing MySQL

Step 6: Run the Security Script

Secure the MySQL installation by running the security script.

Command:

```
sudo mysql_secure_installation
```

Follow these prompts:

1. Enter the temporary root password from Step 5.
2. Press y to set up the VALIDATE PASSWORD component.
3. Choose a password strength level (e.g., 0 for low, 1 for medium, 2 for strong).
4. Enter a new root password and re-enter it to confirm.
5. Press y to continue with the default security settings (e.g., remove anonymous users, disallow root login remotely, remove test database).

This configures MySQL with enhanced security settings.

5.3.4 Starting to Use MySQL

Step 7: Log in to MySQL

Access the MySQL root account using the new password.

Command:

```
sudo mysql -u root -p
```

Enter the root password set in Step 6.

Step 8: Create a Database

Create a sample database to verify functionality.

1. Create a database.

Command (within MySQL prompt):

```
CREATE DATABASE sample_db;
```

2. Verify the database creation.

Command (within MySQL prompt):

```
SHOW DATABASES;
```

This lists all databases, including sample_db.

3. Exit the MySQL prompt.

Command:

```
EXIT;
```

5.4 Securing Systems Using SSL/TLS, Cryptography, SSH, and Firewall

Securing a system is critical to protect data, ensure privacy, and prevent unauthorized access. This guide focuses on securing a Red Hat Enterprise Linux (RHEL) system using **SSL/TLS**, **Cryptography**, **SSH**, and **Firewall** configurations. Each component plays a vital role in enhancing system security, and this note provides practical steps tailored for RHEL.

5.4.1 Securing Systems with SSL/TLS

SSL/TLS (Secure Sockets Layer/Transport Layer Security) encrypts communication between servers and clients, ensuring data confidentiality and integrity. This is essential for web servers, email servers, or any service transmitting sensitive data.

Step 1: Install Required Packages

Install the Apache web server (httpd) and mod_ssl for SSL/TLS support.

Command:
```
sudo yum install httpd mod_ssl -y
```

Step 2: Obtain an SSL/TLS Certificate

Use **Let's Encrypt** to obtain a free SSL/TLS certificate or acquire one from a Certificate Authority (CA).

1. Install the Certbot tool for Let's Encrypt.

Command:
```
sudo yum install certbot python3-certbot-apache -y
```

2. Request a certificate for your domain.

Command:
```
sudo certbot --apache -d yourdomain.com
```

Replace yourdomain.com with your actual domain. Follow the prompts to configure the certificate.

Step 3: Configure Apache for SSL/TLS

Certbot automatically updates the Apache configuration, but you can manually verify or edit the SSL configuration.

1. Edit the SSL configuration file.

Command:
```
sudo nano /etc/httpd/conf.d/ssl.conf
```

2. Ensure the following lines are present (adjust paths if needed):

```
SSLEngine on

SSLCertificateFile
/etc/letsencrypt/live/yourdomain.com/fullchain.pem

SSLCertificateKeyFile
/etc/letsencrypt/live/yourdomain.com/privkey.pem
```

3. Save and close the file.

Step 4: Restart Apache

Apply the changes by restarting the Apache service.

Command:

```
sudo systemctl restart httpd
```

Step 5: Test SSL/TLS

Access your website using https://yourdomain.com in a browser. Verify the certificate using a tool like openssl:

Command:

```
openssl s_client -connect yourdomain.com:443
```

This displays the certificate details, confirming SSL/TLS is active.

5.4.2 Implementing Cryptography

Cryptography protects data at rest and in transit by encrypting sensitive files or communications. RHEL provides tools like **OpenSSL** and **GPG** for cryptographic operations.

Step 1: Install Cryptographic Tools

Ensure OpenSSL and GPG are installed.

Command:

```
sudo yum install openssl gnupg2 -y
```

Step 2: Encrypt Files with OpenSSL

Encrypt sensitive files to protect them from unauthorized access.

1. Encrypt a file (e.g., sensitive.txt).

Command:

```
openssl enc -aes-256-cbc -salt -in sensitive.txt -out
sensitive.txt.enc
```

Enter a password when prompted. The encrypted file is saved as sensitive.txt.enc.

2. Decrypt the file when needed.

Command:

```
openssl enc -aes-256-cbc -d -in sensitive.txt.enc -out sensitive_decrypted.txt
```

Enter the same password to decrypt.

Step 3: Use GPG for Key-Based Encryption

GPG provides public/private key encryption for secure file sharing.

1. Generate a GPG key pair.

Command:

```
gpg --gen-key
```

Follow the prompts to set up your key (name, email, passphrase).

2. Encrypt a file with your public key.

Command:

```
gpg --encrypt --recipient your_email@example.com sensitive.txt
```

This creates sensitive.txt.gpg.

3. Decrypt the file with your private key.

Command:

```
gpg --decrypt sensitive.txt.gpg > sensitive_decrypted.txt
```

Enter your passphrase to decrypt.

Step 4: Secure Database Credentials

For services like MySQL, store encrypted credentials and decrypt them programmatically or manually when needed, reducing exposure of plaintext passwords.

5.4.3 Configuring SSH for Secure Access

SSH (Secure Shell) provides encrypted remote access to your RHEL system. Proper configuration minimizes vulnerabilities.

Step 1: Install and Start SSH

Ensure the SSH server (sshd) is installed and running.

Command:

```
sudo yum install openssh-server -y
```

```
sudo systemctl start sshd
sudo systemctl enable sshd
```

Step 2: Harden SSH Configuration

Edit the SSH configuration file to enhance security.

Command:

```
sudo nano /etc/ssh/sshd_config
```

Update or add the following settings:

```
Port 22
PermitRootLogin no
PasswordAuthentication no
AllowUsers username
```

- Port 2222: Changes the default SSH port (replace 2222 with your preferred port).
- PermitRootLogin no: Disables root login.
- PasswordAuthentication no: Enforces key-based authentication.
- AllowUsers username: Restricts SSH access to specific users (replace username).

Save and close the file.

Step 3: Set Up Key-Based Authentication

1. Generate an SSH key pair on your local machine (if not already done).

Command (on your local machine):

```
ssh-keygen -t rsa -b 4096
```

2. Copy the public key to the RHEL server.

Command (from your local machine):

```
ssh-copy-id -i ~/.ssh/id_rsa.pub username@yourserver.com
-p 22
```

Replace username, yourserver.com, and 2222 with your details.

Step 4: Restart SSH Service

Apply the changes.

Command:

```
sudo systemctl restart sshd
```

Step 5: Test SSH Access

Connect to the server using the new port and key-based authentication.

Command (from your local machine):

```
ssh -p 22 username@yourserver.com
```

5.4.4 Setting Up a Firewall

A **firewall** controls network traffic, allowing only authorized connections. RHEL uses **firewalld** for firewall management.

Step 1: Install and Start Firewalld

Ensure firewalld is installed and running.

Command:

```
sudo yum install firewalld -y
sudo systemctl start firewalld
sudo systemctl enable firewalld
```

Step 2: Configure Firewall Rules

1. Allow necessary services (e.g., HTTP, HTTPS, SSH).

Commands:

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --permanent --add-port=2222/tcp
```

The --add-port=2222/tcp allows the custom SSH port.

2. Reload the firewall to apply changes.

Command:

```
sudo firewall-cmd --reload
```

Step 3: Restrict Unnecessary Ports

List active services and ports to ensure only required ones are open.

Command:

```
sudo firewall-cmd --list-all
```

Remove any unnecessary services or ports.

Example (to remove a service):

```
sudo firewall-cmd --permanent --remove-service=unnecessary_service
```

```
sudo firewall-cmd --reload
```

Step 4: Test Firewall

Attempt to access your server's services (e.g., https://yourdomain.com or SSH on port 2222) to confirm the firewall allows legitimate traffic while blocking unauthorized access.

For Practice in Localhost; Here are the commands:

1. SSL/TLS (Apache + mod_ssl)

```
sudo yum install httpd mod_ssl -y
```

```
# Generate a self-signed certificate:
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
 -keyout /etc/pki/tls/private/selfsigned.key \
 -out /etc/pki/tls/certs/selfsigned.crt
```

```
# Edit Apache SSL config
sudo nano /etc/httpd/conf.d/ssl.conf
```

```
# Update these lines:
SSLCertificateFile /etc/pki/tls/certs/selfsigned.crt
SSLCertificateKeyFile /etc/pki/tls/private/selfsigned.key
```

```
# Restart Apache
sudo systemctl restart httpd
```

🔍 **Test:**

In a **browser on your host**

---

3. SSH (Localhost Access)

✅ **Yes, even without a network.**

You'll connect to **localhost** via SSH from inside the same VM or from your host if port-forwarded.

🔧 **How to simulate:**

```
# Install and start SSH server
sudo yum install openssh-server -y
sudo systemctl enable --now sshd
```

```
# Change default port (optional)
sudo nano /etc/ssh/sshd_config
# Set:
Port 22
PermitRootLogin no
PasswordAuthentication no
AllowUsers your_username
```

```
# Restart SSH
sudo systemctl restart sshd
```

🧪 **Test:**

From within the VM:

```
ssh -p 22 your_username@localhost
```

Or, from host (if port 2222 is forwarded in VMware).

🔑 **Key-Based Authentication:**

On your VM (or from host):

```
ssh-keygen -t rsa -b 4096
ssh-copy-id -i ~/.ssh/id_rsa.pub your_username@localhost
-p 22
```

Q. How do file permissions and Access Control Lists (ACLs) help in securing a Linux system? Explain the process of configuring ACL in Linux.

File permissions and Access Control Lists (ACLs) are essential components of Linux system security. They help control access to files and directories, ensuring that only authorized users can perform specific actions.

1. File Permissions:

Linux uses a permission model based on three categories:

- **Owner**
- **Group**
- **Others**

Each category can be assigned:

- **Read (r):** View file or list directory contents
- **Write (w):** Modify file or directory
- **Execute (x):** Run executable files or access directories

Example:

```
-rw-r--r--  user  group  file.txt
```

This means:

- Owner can read and write
- Group can read
- Others can read

By setting correct permissions, we can prevent unauthorized reading, modification, or execution of files, which strengthens system security.

2. Access Control Lists (ACLs):

ACLs provide more detailed control by allowing permissions to be set for specific users or groups beyond the standard three.

For example:

```
setfacl -m u:ajay:r-- file.txt
```

This gives user ajay read-only access, even if he is not the owner or in the group.

ACLs are useful when:

- Multiple users need different levels of access
- Default permission model is not flexible enough

3. Security Benefits:

- Enforces **principle of least privilege**
- Prevents **unauthorized access or accidental changes**
- Protects **confidential data**
- Allows **fine-grained control** in multi-user environments

5.5 Implementing ACLs and Anti-Spam Measures for Security on RHEL

Securing a Red Hat Enterprise Linux (RHEL) system involves fine-grained access control and protection against unwanted communications. This guide details how to implement **Access Control Lists (ACLs)** for precise file permissions and **anti-spam measures** for mail servers using tools like **SpamAssassin** with **Postfix**. These steps enhance system security by restricting access and filtering malicious or unwanted emails.

5.5.1 Implementing Access Control Lists (ACLs)

Access Control Lists (ACLs) provide a flexible permission mechanism beyond standard Linux permissions, allowing specific access for individual users or groups without altering ownership or group settings.

Step 1: Verify Filesystem Support for ACLs

Ensure your filesystem supports ACLs. Most modern RHEL filesystems (e.g., ext4, xfs) do, but verify the mount options.

Command:

```
sudo tune2fs -l /dev/sdX | grep acl
```

Replace /dev/sdX with your filesystem's device (e.g., /dev/sda1). If acl is listed, proceed. If not, enable it:

Command:

```
sudo tune2fs -o acl /dev/sdX
sudo mount -o remount /mount/point
```

Step 2: Install ACL Tools

Install the acl package to manage ACLs.

Command:

```
sudo yum install acl -y
```

Step 3: Set ACLs for a User

Grant specific permissions to a user for a file or directory. For example, give user johndoe read and execute permissions on /var/www/html/project.

Command:

```
sudo setfacl -m u:johndoe:rx /var/www/html/project
```

Step 4: Set ACLs for a Group

Grant permissions to a group. For example, give the developers group read and write access to /var/www/html/project.

Command:

```
sudo setfacl -m g:developers:rw /var/www/html/project
```

Step 5: Set Default ACLs for Inheritance

Ensure new files in a directory inherit specific ACLs. For example, set default ACLs so new files in /var/www/html/project grant johndoe read access.

Command:

```
sudo setfacl -dm u:johndoe:r /var/www/html/project
```

Step 6: View ACLs

Check the ACLs for a file or directory.

Command:

```
getfacl /var/www/html/project
```

Example Output:

```
# file: project
# owner: apache
# group: apache
user::rwx
user:johndoe:r-x
group::r-x
group:developers:rw-
mask::rwx
```

```
other::r-x
default:user::rwx
default:user:johndoe:r--
default:group::r-x
default:mask::rwx
default:other::r-x
```

Note the + sign in ls -l output (e.g., drwxr-xr-x+), indicating ACLs are set.

Step 7: Remove or Modify ACLs

1. Remove a specific ACL entry.

Command:

```
sudo setfacl -x u:johndoe /var/www/html/project
```

2. Remove all ACLs from a file or directory.

Command:

```
sudo setfacl -b /var/www/html/project
```

Step 8: Test ACLs

Create a test file in /var/www/html/project and verify permissions.

Command:

```
sudo touch /var/www/html/project/test.txt
getfacl /var/www/html/project/test.txt
```

Ensure johndoe has read access as per the default ACL.

5.5.2 Setting Up Anti-Spam Measures

Anti-spam measures protect your mail server from unwanted or malicious emails. This section integrates **SpamAssassin** with **Postfix** to filter spam on a RHEL mail server.

Step 1: Install SpamAssassin

Install SpamAssassin and its dependencies.

Command:

```
sudo yum install spamassassin -y
```

Step 2: Configure SpamAssassin

Edit the SpamAssassin configuration to customize spam filtering rules.

Command:

```
sudo nano /etc/mail/spamassassin/local.cf
```

Add or modify the following settings:

```
rewrite_header Subject *****SPAM*****
required_score 5.0
use_bayes 1
bayes_auto_learn 1
```

- rewrite_header: Marks emails as spam in the subject line.
- required_score: Sets the threshold for marking emails as spam (lower is stricter).
- use_bayes and bayes_auto_learn: Enable Bayesian filtering for improved spam detection.

Save and close the file.

Step 3: Start and Enable SpamAssassin

Start the SpamAssassin service and enable it to run on boot.

Command:

```
sudo systemctl start spamassassin
sudo systemctl enable spamassassin
```

Step 4: Integrate SpamAssassin with Postfix

Configure Postfix to filter emails through SpamAssassin.

1. Install the postfix package if not already installed (refer to previous guides).

Command:

```
sudo yum install postfix -y
```

2. Edit the Postfix configuration to use SpamAssassin.

Command:

```
sudo nano /etc/postfix/master.cf
```

Add the following lines at the end:

```
spamassassin unix - n n - - pipe
  user=spamd argv=/usr/bin/spamc -f -e /usr/sbin/sendmail
-oi -f ${sender} ${recipient}
```

3. Edit the Postfix main configuration to enable content filtering.

Command:

```
sudo nano /etc/postfix/main.cf
```

Add or modify:

```
smtpd_milters   =   unix:/var/run/spamass-milter/spamass-
milter.sock
non_smtpd_milters = unix:/var/run/spamass-milter/spamass-
milter.sock
```

4. Restart Postfix and SpamAssassin.

Command:

```
sudo systemctl restart postfix
sudo systemctl restart spamassassin
```

Step 5: Install and Configure SpamAssassin Milter

Install the SpamAssassin milter for better integration.

Command:

```
sudo yum install spamass-milter -y
```

Start and enable the milter service.

Command:

```
sudo systemctl start spamass-milter
sudo systemctl enable spamass-milter
```

Step 6: Test Anti-Spam Filtering

Send a test email to your mail server (e.g., using mail command or an email client). Check the email headers or subject line for *****SPAM***** if the email exceeds the spam score threshold.

Command (to send a test email):

```
echo "Test email body" | mail -s "Test Subject" user@yourdomain.com
```

Check the mail log for SpamAssassin activity.

Command:

```
sudo tail -f /var/log/maillog
```

Step 7: Additional Anti-Spam Measures

1. **Enable SPF and DKIM**:
   - Install and configure opendkim for DKIM signing.

Command:

```
sudo yum install opendkim -y
sudo nano /etc/opendkim.conf
```

Add your domain and key settings, then generate and publish DKIM keys.

2. **Blacklist/Whitelist**:
   - Update SpamAssassin rules to blacklist known spam domains or whitelist trusted senders.

Command:

```
sudo nano /etc/mail/spamassassin/local.cf
```

Example:

```
blacklist_from spammer@bad-domain.com
whitelist_from trusted@safe-domain.com
```

3. **Update SpamAssassin Rules**:
   - Regularly update SpamAssassin rules for the latest spam patterns.

Command:

```
sudo sa-update
```

5.6 Troubleshooting and Resolving Issues in Servers and

Troubleshooting server and network issues on Red Hat Enterprise Linux (RHEL) requires a systematic approach to identify, diagnose, and resolve problems efficiently. This guide provides practical steps to troubleshoot common server and network issues, focusing on tools and techniques available on RHEL. It covers issues related to services (e.g., Apache, MySQL,

Postfix), connectivity, performance, and security configurations like those previously discussed (ACLs, SSL/TLS, SSH, Firewall, Anti-Spam).

5.6.1 General Troubleshooting Approach

1. **Identify the Problem**: Gather symptoms (e.g., error messages, service failures, slow performance) and user reports.
2. **Reproduce the Issue**: Attempt to replicate the problem to understand its scope and triggers.
3. **Check Logs**: Review system and application logs for errors or warnings.
4. **Isolate the Cause**: Narrow down whether the issue is with the server, network, or configuration.
5. **Test Solutions**: Apply fixes incrementally and verify their impact.
6. **Document Findings**: Record the issue, cause, and resolution for future reference.

5.6.2 Troubleshooting Server Issues

1. Service Not Starting (e.g., Apache, MySQL, Postfix)

Symptoms: Service fails to start, or systemctl status shows failed or inactive.

Steps:

1. **Check Service Status**:

```
sudo systemctl status httpd
```

Look for errors in the output (e.g., port conflicts, missing files).

2. **Review Logs**:
   - Apache: /var/log/httpd/error_log
   - MySQL: /var/log/mysqld.log
   - Postfix: /var/log/maillog

```
sudo tail -n 50 /var/log/httpd/error_log
```

3. **Common Issues and Fixes**:
   - **Port Conflict**: Check if another service is using the same port (e.g., port 80 for Apache).

```
sudo netstat -tulnp | grep :80
```

     Stop or reconfigure the conflicting service.

   - **Configuration Errors**: Validate configuration files.
     - Apache: sudo apachectl configtest
     - Postfix: sudo postconf -n
   - **Insufficient Permissions**: Ensure the service user (e.g., apache, mysql) has access to necessary files.

```
sudo ls -l /var/www/html
sudo chown apache:apache /var/www/html -R
```

4. **Restart Service**:

```
sudo systemctl restart httpd
```

2. High CPU/Memory Usage

Symptoms: Server is slow, processes hang, or top shows high resource usage.

Steps:

1. **Identify Resource-Hogging Processes**:

```
top
```

Note the PID and command of high-CPU/memory processes.

2. **Check Process Details**:

```
ps -aux | grep <pid>
```

3. **Common Issues and Fixes**:
   - **Apache Overload**: Too many connections or misconfigured httpd.conf.
     - Limit connections:

```
sudo nano /etc/httpd/conf/httpd.conf
```

Adjust MaxClients or MaxRequestWorkers (e.g., MaxRequestWorkers 150).

     - Restart Apache: sudo systemctl restart httpd.
   - **MySQL Queries**: Long-running or unoptimized queries.
     - Enable slow query log:

```
sudo nano /etc/my.cnf
Add:
[mysqld]
slow_query_log = 1
slow_query_log_file = /var/log/mysql-slow.log
```

     - Analyze: sudo tail /var/log/mysql-slow.log.
   - **Kill Unnecessary Processes**:

```
sudo kill -9 <pid>
```

4. **Monitor System Resources**: Use htop or vmstat for real-time monitoring.

```
sudo yum install htop -y
htop
```

3. ACL Issues

Symptoms: Users cannot access files despite ACL settings, or permissions are incorrect.

Steps:

1. **Verify ACLs**:

```
getfacl /path/to/file
```

Check for correct user/group entries.

2. **Check Filesystem Support**:

```
sudo tune2fs -l /dev/sdX | grep acl
```

Ensure acl is enabled.

3. **Common Issues and Fixes**:
   - **Incorrect ACL Entry**: Reapply ACL.

```
sudo setfacl -m u:johndoe:rwx /path/to/file
```

   - **Inheritance Not Working**: Set default ACLs.

```
sudo setfacl -dm u:johndoe:r /path/to/dir
```

   - **Mask Issues**: Ensure the ACL mask allows permissions.

```
sudo setfacl -m m:rwx /path/to/file
```

4. **Test Access**: Log in as the affected user and attempt to access the file.

4. SpamAssassin/Anti-Spam Failures

Symptoms: Spam emails are not filtered, or legitimate emails are marked as spam.

Steps:

1. **Check Logs**:

```
sudo tail -n 50 /var/log/maillog
```

Look for SpamAssassin errors or filtering issues.

2. **Verify SpamAssassin Service**:

```
sudo systemctl status spamassassin
```

3. **Common Issues and Fixes**:
   - **Rules Outdated**: Update SpamAssassin rules.

```
sudo sa-update
sudo systemctl restart spamassassin
```

   - **Misconfigured Threshold**: Adjust required_score in /etc/mail/spamassassin/local.cf.

```
sudo nano /etc/mail/spamassassin/local.cf
```

     Set required_score 4.0 for stricter filtering.

   - **Milter Failure**: Ensure spamass-milter is running.

```
sudo systemctl restart spamass-milter
```

4. **Test Filtering**: Send a test email and check headers for SpamAssassin scores.

```
echo "Test spam" | mail -s "Test" user@yourdomain.com
```

5.6.3 Troubleshooting Network Issues

1. Connectivity Issues

Symptoms: Unable to ping servers, SSH fails, or websites are unreachable.

Steps:

1. **Test Basic Connectivity**:

```
ping 8.8.8.8
ping yourdomain.com
If pinging IP works but domain fails, DNS is the issue.
```

2. **Check DNS Resolution**:

```
nslookup yourdomain.com
cat /etc/resolv.conf
```

   Ensure valid nameservers (e.g., nameserver 8.8.8.8).

3. **Check Network Interfaces**:

```
ip addr
nmcli device status
```

Verify interfaces are UP.

4. **Common Issues and Fixes**:
   - **DNS Misconfiguration**: Update /etc/resolv.conf or use NetworkManager.

```
sudo nmcli con mod "System eth0" ipv4.dns "8.8.8.8 8.8.4.4"
sudo nmcli con up "System eth0"
```

   - **Interface Down**: Bring it up.

```
sudo ip link set eth0 up
```

   - **Firewall Blocking**: Check firewall rules.

```
sudo firewall-cmd --list-all
Allow necessary ports (e.g., 80, 443, 2222).
sudo firewall-cmd --permanent --add-port=80/tcp
sudo firewall-cmd --reload
```

2. Slow Network Performance

Symptoms: Slow downloads, high latency, or dropped packets.

Steps:

1. **Test Network Speed**:

```
sudo yum install speedtest-cli -y
speedtest-cli
```

2. **Check Packet Loss**:

```
ping -c 10 8.8.8.8
```

3. **Monitor Traffic**: Use iftop or nload to identify bandwidth usage.

```
sudo yum install iftop -y
sudo iftop -i eth0
```

4. **Common Issues and Fixes**:
   - **Congested Network**: Limit bandwidth-heavy services or prioritize traffic with tc.
   - **MTU Issues**: Check and adjust MTU.

```
ip link show | grep mtu
sudo ip link set eth0 mtu 1400
```

- **Faulty Hardware**: Test with a different cable or NIC.

3. SSH Connection Failures

Symptoms: Connection refused or timeout when connecting via SSH.

Steps:

1. **Verify SSH Service**:

```
sudo systemctl status sshd
```

2. **Check Port and Firewall**: Ensure the SSH port (e.g., 2222) is open.

```
sudo firewall-cmd --list-ports
sudo netstat -tulnp | grep 2222
```

3. **Common Issues and Fixes**:
   - **Port Misconfiguration**: Verify /etc/ssh/sshd_config has the correct Port directive.
   - **Firewall Blocking**: Add the SSH port.

```
sudo firewall-cmd --permanent --add-port=2222/tcp
sudo firewall-cmd --reload
```

   - **Key Issues**: Ensure the client's public key is in ~/.ssh/authorized_keys.

```
sudo cat /home/username/.ssh/authorized_keys
```

4. **Test Connection**:

```
ssh -p 2222 username@yourserver.com
```

5.6.4 Resolving Common Issues

1. SSL/TLS Errors

Symptoms: Browser shows certificate errors, or openssl s_client fails.

Fixes:

- **Expired Certificate**: Renew with Certbot.

```
sudo certbot renew
```

- **Misconfigured Certificate**: Verify paths in /etc/httpd/conf.d/ssl.conf.
- **Protocol Mismatch**: Ensure SSLEngine on and modern protocols (e.g., TLSv1.2, TLSv1.3).

2. Email Delivery Failures

Symptoms: Emails not sent/received, or Postfix/Dovecot logs show errors.

Fixes:

- **SMTP Issues**: Check Postfix configuration.

```
sudo postconf -n
```

- **Dovecot Authentication**: Verify /etc/dovecot/dovecot.conf settings.
- **DNS Records**: Ensure MX, SPF, and DKIM records are correct.

```
nslookup -type=MX yourdomain.com
```

3. Permission Denied Errors

Symptoms: Users cannot access files despite correct ACLs or ownership.

Fixes:

- **SELinux Blocking**: Check SELinux context.

```
ls -Z /path/to/file
sudo restorecon -R /path/to/file
```

- **Incorrect Ownership**: Fix ownership.

```
sudo chown apache:apache /var/www/html -R
```